



Autonomous Traffic Clearance for Emergency Vehicles

A Cooperative Reinforcement Learning Approach

**Academically Supervised by
Dr Mohamed Elshenawy**

**Technically Supervised by
Dr Mohamed Abdo - Dr Hesham Eleraqi
Eng Waleed Mohamed**

**Original Work of
Abdulahdy Ramadan Feteiha, CIE Department
Alaa Hesham Mahmoud, CIE Department
Mohammed EL-Sayed Ahmed, SPC Department
Nadine Amr Mahmoud, SPC Department
Tasneem Basem Omara, SPC Department
Walid Moustafa Awad, CIE Department**

A Graduation Project Thesis Submitted to
The Communications and Information Engineering Department
&
The Aerospace Engineering Department
at
The University of Science and Technology, Zewail City
in partial fulfillment of the
requirements of the
Bachelor of Science for the respective majors

Acknowledgements

- **Prof. Ahmed Zewail** for taking the initiative to build the edifice that we are grateful is located in Egypt.
- **Amr Mamdouh** for his great technical help at the beginning of the project, and his later follow-up.
- **Mohamed Elsayed** for being available for advice and feedback..
- Our friends, namely **Mayar Shahin** and **Toka Tarek**, for helping deliver the hardware from USA; and all our friends who were willing to help deliver the hardware packages.
- **Dr Elsayed Hemayed** for enabling a major-interdisciplinary team, and assigning fund for the project.
- **Esraa Magdy** for helping us communicate with Valeo.
- **Yara Alaa** for giving us part of their project fund
- **Mohamed Ahmed** for his dedication, effort, and giving us valuable feedback on our work .
- **Our parents and Loved ones** for their endless support through one of our most stressful academic years.

Autonomous Traffic Clearance for Emergency Vehicles

A Cooperative Reinforcement Learning Approach

Abstract

Healthcare studies suggest serious effects of longer wait times on patients' mortality rates[1], specially on those with fatal and rapidly deteriorating conditions. Reducing response times is also an essential part of any firefighters or policemen training[2]. Despite such data being not present (or at least not easily accessible) for Egypt, congestion is a very-well-researched and undeniably present problem in Egypt[3], specially in the main cities.

The mentioned points, along with the realization of the time-critical nature of emergency situations, stress on the need for emergency vehicles to reach their destinations in the shortest possible times.

In our project, we propose a solution through allowing cooperation between vehicles via communication such that an emergency vehicle can broadcast its location, arriving lane number, and velocity; ahead of time. Consequently, preceding vehicles, each using a trained reinforcement learning model, have enough time to determine and perform appropriate timed maneuvering actions to immediately clear the emergency vehicle's path.

In order to demonstrate our solution, a platform for training reinforcement learning models on autonomous vehicles was implemented. Results are evaluated on two main fronts: effect on an emergency vehicle's (EV's) travel time, and practicality of the model from a mechanics points of view. For the first goal, several vehicles with the introduced reinforcement learned model are deployed and tested in a traffic system simulator called SUMO[4], observing the resulting EV travel time. For the second goal, necessary autonomous vehicle functions such as mapping, localization, global and local path planning, keeping and changing lanes, as well as communication mechanisms were implemented alongside with our model and put in a mechanical simulator.

Our simple q-learning based model demonstrated great improvement on the vehicle's travel time, and was successfully implemented (along with the mentioned autonomous functionalities) in a mechanical simulator. Our main contributions are the implemented autonomous vehicle on the Gazebo [5] simulator using ROS [6], the learned q-learning model with the demonstrated results, using V2V for tasks otherwise performed using sensors, and two Reinforcement Learning platforms on both of the used simulators (SUMO and Gazebo); introducing all of this in a modular and easily accessible manner that we hope later efforts can build on.

— **Keywords:** Traffic Clearance, Cooperative Agents, Reinforcement Learning, Autonomous Vehicles, V2V

Contents

1	Introduction	10
1.1	Problem Definition	10
1.2	Previous Efforts	10
1.3	Proposed Solution	11
1.3.1	Solution Platforms	11
1.3.2	Main Concepts that Affected the Solution	12
1.4	Document Organization	12
2	Background	14
2.1	Reinforcement learning	14
2.2	Q Learning	15
3	Literature Review	17
3.1	Vehicle Rerouting	17
3.2	Dynamic Signal Control	18
3.3	Independent Vehicle Planning	18
3.4	Reinforcement Learning as an Alternative	19
4	Project Design	20
4.1	Technical Specifications	20
4.1.1	Environment and Agent Model	20
4.1.1.1	Gazebo	20
4.1.1.2	SUMO	24
4.1.2	Autonomous Functionalities	27
4.1.2.1	Mapping and Localization	27
4.1.2.2	Longitudinal Control	28
4.1.2.3	Lateral Control	28
4.1.2.4	Lane Keeping	29
4.1.2.5	Lane Changing	29
4.1.2.6	Path Planning	29
4.1.2.7	Navigation	31
4.1.2.8	External Commands Following	31
4.1.2.9	High-Level Control Master	32
4.1.3	V2V Communication	32
4.1.4	Cooperation Technique	33
4.1.4.1	Training Single Agent with Q-learning in a Single-Agent Environment	33
4.1.4.2	Testing Single Agent with Q-learning in a Multi-agent Environment	34

4.1.5	System Integration	34
4.2	Design Alternatives and Justification	35
4.2.1	Environment and Agent Model	36
4.2.1.1	Gazebo	36
4.2.1.2	SUMO	36
4.2.2	Autonomous Functionalities	38
4.2.2.1	Mapping and Localization	38
4.2.2.2	Longitudinal Control	39
4.2.2.3	Lateral Control	39
4.2.2.4	Lane Keeping	40
4.2.2.5	Lane Changing	41
4.2.2.6	Path Planning	42
4.2.2.7	Navigation	43
4.2.2.8	External Commands Following	44
4.2.2.9	High-Level Control Master	44
4.2.3	V2V Communication	44
4.2.4	Cooperation Technique	45
5	Project Implementation	46
5.1	Autonomous Functionalities	46
5.1.1	Mapping and Localization	46
5.1.2	Longitudinal Control	47
5.1.3	Lateral Control	49
5.1.4	Lane Keeping	51
5.1.5	Lane Changing	52
5.1.6	Path Planning	56
5.1.7	Navigation	61
5.1.8	External Commands Following	63
5.1.9	High-Level Control Master	65
5.2	V2V Communication	66
5.2.1	ID Message Structure	66
5.2.2	Broadcasting ID Messages	67
5.2.3	Receiving ID Messages	68
5.3	Cooperation Technique	69
5.3.1	Single Agent Problem Formulation	69
5.3.1.1	Environment	69
5.3.1.2	States	70
5.3.1.3	Actions	70
5.3.1.4	Reward	70
5.3.1.5	General Problem Formulation Design Considerations	71
5.3.2	Moving the Single Agent to Multi-Agent Environment	72
5.4	System Integration	73
5.4.1	RL Algorithm	75
5.4.2	Environment	75
5.4.3	Master	76

6	Project Testing and Discussion	77
6.1	Autonomous Functionalities	77
6.1.1	Mapping and Localization	77
6.1.2	Longitudinal Control	78
6.1.3	Lateral Control	81
6.1.4	Lane Keeping	83
6.1.5	Lane Changing	83
6.1.6	Path Planning	92
6.1.7	Navigation	98
6.1.8	External Commands Following	109
6.1.9	High-Level Control Master	113
6.2	V2V Communication	116
6.3	Cooperation Technique	119
6.3.1	Single Agent with Q-learning in a Single-Agent Environment .	119
6.3.2	Single Agent with Q-learning in a Multi-agent Environment .	120
6.3.2.1	Experiment Setup	120
6.3.2.2	Averaging and Randomness	121
6.3.2.3	Important Note Before Results	121
6.3.2.4	Results	121
6.4	System Integration	123
7	Conclusion and Future Work	130
7.1	Conclusion	130
7.2	Main Contributions	130
7.3	Future Work	131

List of Figures

4.1	Straight three-lane road simulated in Gazebo.	22
4.2	Agent model spawned in Gazebo environment	23
4.3	270° wide detection angle of the laser scanner.	23
4.4	The geometry of the ackermann robot in the two dimensional space [29].	24
5.1	Geometry associated with the Stanley controller	51
5.2	Applying homography on a raw grayscale image from the simulation, giving us our map (bird's eye view) view.	52
5.3	Detecting the lane on a thresholded bird's eye view image using sequential rectangles (shown in red), followed by generating way-points (blue dots) from the calculated quadratic equations of both the left and right lanes.	53
5.4	Applying inverse homography on the way-points generated in the bird's eye view to retrieve them in the simulation coordinate system.	53
5.5	Lane Change schematic diagram.	53
5.6	Communication Layer Update Algorithm	60
6.1	Resulted map generated from Gmapping.	77
6.2	Longitudinal Control - Test 2: Initial relative vehicle positions.	79
6.3	Longitudinal Control - Test 2: Final relative vehicle positions.	81
6.4	Longitudinal Control - Test 3: Final relative vehicle positions.	82
6.5	Stanley Controller - Test 1: Large initial cross-track error	82
6.6	Stanley Controller - Test 2: Large initial heading error	83
6.7	Way-points generated (in blue) appear at the center of the thresholded lane (in white) at all times in our tests.	84
6.8	Lane Changing - Test 1: Case A - Terminal log - Feasible.	85
6.9	Lane Changing - Test 1: Case A - Feasible.	85
6.10	Lane Changing - Test 2: Case A - Terminal log - Infeasible.	87
6.11	Lane Changing - Test 2: Case A - Infeasible	87
6.12	Lane Changing - Test 3: Indeterminate Case A or B - Terminal log - Infeasible.	88
6.13	Lane Changing - Test 3: Indeterminate Case A or B - Infeasible	88
6.14	Lane Changing - Test 4: Case B - Terminal log - Infeasible.	89
6.15	Lane Changing - Test 4: Case B - Infeasible	89
6.16	Lane Changing - Test 5: Case C - Terminal log - Feasible.	90
6.17	Lane Changing - Test 5: Case C - Feasible	90
6.18	Lane Changing - Test 6: Case D - Terminal log - Feasible.	92
6.19	Lane Changing - Test 6: Case D - Feasible	92

6.20	Communication Layer - Test 1: Ego (red) vehicle's global costmap (RViz).	94
6.21	Communication Layer - Test 1: Ego (red) vehicle's local costmap (RViz).	95
6.22	Communication Layer - Test 2: Communication layer added only to the ego (red) vehicle's global costmap (RViz).	95
6.23	Communication Layer - Test 2: Ego (red) vehicle's global costmap while navigating to goal (RViz).	96
6.24	Communication Layer - Test 3: Communication layer added only to the ego (red) vehicle's local costmap (RViz).	97
6.25	Communication Layer - Test 3: Ego (red) vehicle's local costmap while navigating to goal (RViz).	97
6.26	Test Cases and Results	98
6.27	Test 1: Starting Position	101
6.28	Test 1: First Goal	101
6.29	Test 1: Second Goal	102
6.30	Test 1: Third Goal	103
6.31	Test 1: Fourth Goal	104
6.32	Test 2: Starting Positions	106
6.33	Test 2: First Desired Goals' Positions for both AVs (RViz)	106
6.34	Test 2: First Goal Maneuvers	107
6.35	Test 2: First Goal Reached by both AVs (RViz)	107
6.36	Test 2: First Goal Reached by both AVs, yellow AV's lane = 1, and read AV's lane = 2 (Gazebo)	108
6.37	Test 2: Second Goal Positions (RViz)	108
6.38	Test 2: Second Goal	109
6.39	Test 2: External Commands Following - 9 randomly generated actions	112
6.40	Test 1: High-Level Control Master	114
6.41	Test 2: High-Level Control Master	115
6.42	V2V Communication Test: Relative position of test vehicles in the track.	117
6.43	Travel Time vs Lane Busyness, comparing between two cases: RL (all vehicles on field have the model Q_LEARNING_SINGLE_AGENT) deployed, and Auto(all vehicles on field have the model SUMO_KRAUSS deployed). Both models described in 4.1.1.2.	122
6.44	Travel Time vs RL Percentage for different Lane Busyness's.	124
6.45	First four time-steps in episode 1.	127
6.46	First four time-steps in episode 3.	129

List of Tables

5.1	Lane change time and distance at certain velocities.	54
6.1	Longitudinal Control - Test 1 Results	79
6.2	Longitudinal Control - Test 2 Results	80
6.3	Longitudinal Control - Test 3 Results	81
6.4	Lane Changing velocity and steering angle throughout time - Test 1: Case A - Feasible	86
6.5	Lane Changing velocity and steering angle throughout time - Test 2: Case A - Infeasible	87
6.6	Lane Changing velocity and steering angle throughout time - Test 3: Case Indeterminate A or B - Terminal log - Infeasible	88
6.7	Lane Changing velocity and steering angle throughout time - Test 4: Case B - Terminal log - Infeasible	90
6.8	Lane Changing velocity and steering angle throughout time - Test 5: Case C - Feasible	91
6.9	Lane Changing velocity and steering angle throughout time - Test 6: Case D - Feasible	93
6.10	Path Planning - Test Cases: Scenarios and Results	98
6.11	Navigation - Test 1: Scenario	99
6.12	Navigation - Test 1: Results	99
6.13	Navigation - Test 1: Error between planned goals and actual destinations	100
6.14	Navigation - Test 2: Scenario (Starting Positions)	104
6.15	Navigation - Test 2: Scenario (Desired Goals)	104
6.16	Navigation - Test 2: Results (Actual Destinations)	105
6.17	Navigation - Test 2: Errors in Results between desired goals and actual destinations	105
6.18	External Commands Following - Test: Requested Actions	110
6.19	External Commands Following - Test: Responses	111
6.20	TITLE: Episode 1 - Results	126
6.21	Episode 3 results	128

Chapter 1

Introduction

In this chapter, we introduce our project and the contents of this document. Section 1.1 illustrates the problem at hand; Traffic Clearance for Emergency Vehicles (EVs). Section 1.2 discusses previous efforts and what other researchers have proposed to solve this problem. Section 1.3 discusses our proposed solution. Section 1.4 helps the reader navigate the contents of this document by highlighting its different sections.

1.1 Problem Definition

Given our technical backgrounds and the highlighted congestion problems, the problem we choose to tackle (regarding emergency response times) is reducing the travel time an EV takes to reach its destination. It is undoubtedly crucial that emergency vehicles reach their destinations in the least possible amount of time. Taking the ambulance as a compelling example, we can promptly realize how critical it is for such vehicle to reach targeted patients on time and transport them to their nearest hospitals as fast as possible to avoid any adverse consequent complications or deterioration. Unfortunately, what actually happens is that ambulances get considerably delayed by the severely congested streets; they probably end up either wasting time waiting for vehicles to make way for them or taking relatively longer zigzag paths to overtake vehicles in their way.

In countries like Egypt with no dedicated lanes for emergency vehicles, this problem gets specifically escalated. However, even in countries with such dedicated lanes, drivers sometimes tend to break the law by driving in emergency lanes. As a result, several collisions are actually caused by emergency vehicles crashing with other vehicles illegally driven in those emergency lanes. Consequently, this issue is extremely alluring worldwide and definitely constitutes a hot area of research.

1.2 Previous Efforts

Many approaches tried to solve the problem of traffic clearance. A literature review of them along with state of the art results with the research gaps identified are to be discussed in chapter 3. Previous efforts as documented in sections 3.1, 3.2, and 3.3 are divided into three approaches:

- Vehicle rerouting

- Dynamic signal control
- Independent vehicle planning

Vehicle rerouting algorithms try to find alternative routes for either the emergency vehicle, or the other vehicles to alleviate the congestion problem and plan the fastest road for the vehicle.

On the other hand, Dynamic signal control algorithms rely on local control signal to sense the emergency vehicle, and organise traffic signals in a way that directs the emergency vehicle to the fastest possible road.

Finally, independent vehicle planning takes into consideration the role of autonomous vehicles, AVs. Algorithms in this section suggest equipping AVs with planning systems that sense the existence of an emergency vehicle and try to avoid slowing it down.

1.3 Proposed Solution

The emergency lane evacuation problem is evidently a communication and planning problem. Information -at least- about the approaching emergency vehicle should be ***communicated*** to allow for clearing its path. And, for us to use this piece of information to clear the path, careful ***planning*** is in order.

Our proposed solution tackles the planning task for autonomous vehicles, assuming a perfect-information decentralized communication system, to avoid dealing with the communication problem which is carefully studied in more communication-related papers [7][8]. For the planning part, on the other hand, we explore in Section 4.2 several approaches after which we reside to choosing reinforcement learning as our planning method.

1.3.1 Solution Platforms

Our proof-of-concept (illustrated in this project) demonstrates how communicating RL agents can help build on the current commercially used autonomous functionalities in order to reduce the total emergency vehicle's travel time; helping it reach its destination faster. Our approach is investigated on the following platforms:

1. The first platform is a ***traffic simulation*** platform called SUMO (Simulation of Urban Mobility) [4], on which we demonstrate the results of using communicating autonomous agents that implement reinforcement learning for planning (and cooperatiopn) to evacuate the approaching emergency vehicle's path. We use this platform to demonstrate the aggregate properties of the emerging system, such as reduction in the ambulance travel time, through several experiments (in Chapter 6).
2. The second platform is a ***robotic simulation*** platform called ROS (Robot Operating System) [6], on which we demonstrate the practical feasibility of using an RL agent on top of a communicating autonomous agent.

3. A third **hardware** platform would have been in place, where we were going to implement the resulting agent on a real-world toy autonomous vehicle (one tenth of the size of a real formula car), following F1/10 [9], needless to say, as a result of the COVID-19 pandemic, we were not able to continue working on it.

1.3.2 Main Concepts that Affected the Solution

Throughout the design and implementation phases, several concepts have affected our decisions. Some of which were goals, others were just constraints. Below, we mention a few of them:

1. Decentralization and limited communication range: Our system was built on the premise of having a limited communication range for each vehicle, enabling a real-world decentralized implementation. A decentralized implementation (as opposed to a centralized one) is not only much cheaper to implement on large scale, but also avoids the problem of a single failure point. It also enables the incorporation of non-autonomous vehicles, by assuming the problem's solution depends on the participation of all members, but also accounting for noise during training, thus being not totally failing in case of other non-cooperative vehicles. A limited communication range also accounts for power (and thus range) limitations of real-world communication. This imposed, however, some challenges, as it is much easier to implement a solution that gives the best possible action given the full system information, rather than only a part of the information, but that system inevitably uses much more resources, and thus was avoided for reasons mentioned above.
2. Efficiency: We tried to minimize the amount of communicated and stored information, and use the communicated information as efficiently as possible, to account for the limited communication rates in real-world applications.
3. Safety: Reinforcement learning, despite being able to learn many autonomous driving tasks, has not yet matured enough to be used as a an autonomous driving solution on its own [10]. A reinforcement learning policy with autonomous functionalities for feasibility testing and implementation was thus used; more on that in chapter 5.

1.4 Document Organization

In **Chapter 2**, we give the reader a background about the theory used in this document. In Section 2.1, the user is given a background about the field of Reinforcement learning. In Section 2.2, the Q-learning Algorithm (the main algorithm used in our RL solution) is explained from the literature.

In **Chapter 3**, a literature review of the methods used by fellow researchers to solve our problem is presented. The approach of vehicle rerouting is discussed in Section 3.1, Section 3.2 discusses the dynamic signal control approach, while Section 3.3 discusses independent vehicle planning. Section 3.4 discusses Reinforcement Learning as an alternative to the previously mentioned methods.

In **Chapter 4**, the main tasks of the system as well as the tools used to achieve these tasks are discussed. Section 4.1 lays out the technical specifications of the proposed system by introducing the Gazebo and SUMO environment and their associated agent models. Section 4.2 introduces design alternatives we had and justifications for using the current choices.

In **Chapter 5**, a detailed description of the implementation and the methodology behind each subsystem is introduced. In Section 5.1, the methodologies of all the autonomous functionalities¹ are explained in great detail with figures. In Section 5.2, we illustrate how V2V communication is simulated in our system. In Section 5.3, the Q-Learning with communication cooperation technique is highlighted mainly by discussing the problem formulation designed and how it was used on SUMO. In Section 5.4, the details of integrating the RL and ROS systems are discussed.

In **Chapter 6**, we illustrate how we tested each of our modules and their integration. We also show the results of our experiments and follow them with a discussion. In Section 6.1 we illustrate how each of the autonomous functionalities was tested. In Section 6.2, basic simulations of V2V communications on ROS were performed as tests. In Section 6.3 the results of using the reinforcement learning model on the EV travel time are discussed. Finally, in Section 6.4, we demonstrate how the integration of different system modules was tested.

In **Chapter 7**, we conclude the document by highlighting what we concluded from our experiments and what our work has amounted to. In Section 7.1, we conclude the document as well as summarize and comment on our work. In Section 7.2, we highlight our main contributions presented throughout the document. In Section 7.3, we mention our future plans regarding our work.

¹Mapping, Localization, Longitudinal Control, Lateral Control, Lane Keeping, Lane Changing, Path Planning, Navigation and External Commands Following.

Chapter 2

Background

This section provides a very brief background on Reinforcement Learning in general and Q-Learning in particular. If you are familiar with these concepts, please feel free to skip this section.

2.1 Reinforcement learning

Reinforcement learning is a learning paradigm where an agent sequentially interacts with the environment by receiving observations and performing actions that maximize a reward signal in the long run. [11] This could be modeled as a Markov decision process, MDP, defined by a tuple $\langle S, A, T, R \rangle$. Thus the problem consists of:

- **S**, a finite state space $\{s_1, \dots, s_N\}$ which comprises all the observations retrieved from the environment with length N .
- **A**, a finite action set $\{a_1, \dots, a_K\}$ which consists of all the actions feasible to be performed by the agent in a time step t with length K .
- **T**, a transition function that defines the probability of transition to a state $s + 1$ by applying action a in a state s . Where $a \in A$ and $s, s + 1 \in S$ and the transition function is defined as $T: S \times A \rightarrow P(S)$
- **R**, a reward function which specifies the reward of transitioning between a state s to state $s + 1$ by doing an action a in the form: $R: S \times A \times S \rightarrow R$.

The agent sequentially pops an action a out of the set of finite actions A via either exploration or exploitation. Initially, the agent has no knowledge of the optimal action to choose. Hence, it has to randomly explore the available actions and receive a reward that evaluates the consequences of this action. As the agent learns, choosing random actions becomes problematic as it might end up in a sequence of undesirable actions. So it's also important for the agent to exploit by choosing an action with a higher expected reward rather than relying solely on random trial and error. Balancing these two approaches is what's defined as an exploration-exploitation problem.

In the context of an MDP $\langle S, A, T, R \rangle$, a policy, π , is a computable function that maps the current state $s \in S$ to a feasible action $a \in A(s)$. Application of a policy in an MDP happens in the following manner:

- A start state s_0 is drawn from the initial state distribution I
- An action is chosen using the policy π as $a_0 = \pi(s_0)$
- Using the transition function T , a transition is made to state s_1 , and a reward is assigned to this action using the reward function R as: $r_0 = R(s_0, a_0, s_1)$
- The same process of action picking and transitioning repeats generating the sequence:

$$s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, \dots, s_g \quad (2.1)$$

- An episode ends if the environment returns goal state s_g , and a new episode is started by generating a state from the initial state distribution I .

Another important definition is the value function $V(s)$. Value function, defined for a particular policy π , is the expected return when starting in a state s and following the policy π as expressed in the formula:

$$v^\pi(s) = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right\} \quad (2.2)$$

where γ is the discount factor discounting future rewards according to how far in time they are. Given the definition of the value function $V(s)$, we could define the Q function $Q(s, a)$ as the expected return starting from state s , taking action a , and following the policy π . Q function is expressed in the following formula:

$$Q^\pi(s) = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right\} \quad (2.3)$$

Q function is a very useful notion especially in model free approaches as it doesn't require prior knowledge of T and R . The optimal action selection policy, denoted as the greedy policy, hence becomes as easy as maximizing the Q function as in the following formula:

$$\pi^*(s) = \operatorname{argmax}(Q^*(s, a)) \quad (2.4)$$

2.2 Q Learning

Q-learning is an off-policy algorithm, which means that it tries to estimate the optimal policy π^* while following an exploration policy π . [11] Q-learning gives a greater leverage over other RL algorithms in its simplicity. Unlike other algorithms, Q-learning directly approximates the optimal policy with guaranteed convergence given that all the state-action pairs are visited with a large enough number of times. In its simplest form, one-step Q-learning, it incrementally estimates Q-values for actions based on reward and agent's Q-value function in the following form:

$$Q_{k+a}(s_t, a_t) = Q_k(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q_k(s_{t+1}, a) - Q_k(s_t, a_t) \right) \quad (2.5)$$

Where α is the algorithm's learning rate. For problems with a finite state space; Q-learning could be implemented in a tabular manner relating each state-action pair with its corresponding Q-value via the following algorithm. [12]

Algorithm 0: Q learning algorithm

Initialize $Q(s, a), \forall s \in S, a \in A(s)$, arbitrarily, and
 $Q(\text{terminal} - \text{state},) = 0$;
while *Not last episode* **do**
 Initialize S ;
 while S is not *Terminal* **do**
 Choose A from S using policy derived from Q (*e.g.*, $\epsilon - greedy$);
 Take action A , observe, $R, S^{(dash)}$;
 $S \leftarrow S^{(dash)}$
 end
end

Chapter 3

Literature Review

In this chapter, we define the main approaches to solve the problem with examples for each approach.

The problem of clearing traffic for an emergency vehicle was addressed in the literature using three approaches:

- **Vehicle rerouting**, which involves planning alternative routes to clear the congestion or avoid it.
- **Dynamic signal control**, which depends on a control center that directs the traffic flow in a way that enables the emergency vehicle to reach its destination with minimum delay through controlling traffic signals in its area.
- **Independent vehicle planning**, where vehicles independently search and plan for possible routes for the traffic clearance mission.

3.1 Vehicle Rerouting

[13] suggests informing emergency vehicles ahead of time which roads are the most congested so that they could avoid taking them to their destinations. In [14], vehicles within a certain distance from the incoming emergency vehicle were informed about it so that their drivers could take the action they see appropriate.. On another hand, in [8], each vehicle was not only warned of the approaching emergency vehicle but also instructed, through a mobile application, with the exact suitable action its driver ought to take in order to make way for the emergency vehicle. This warning message contained information about the emergency vehicle's current position, velocity, heading, destination, and route information. In [15], a very similar solution was presented. A simulation of one ambulance and two other vehicles was tested and the decision algorithm succeeded in instructing the vehicles with the truly appropriate maneuvers.

These approaches have the merit of solving the emergency traffic clearance problem, however, a major demerit that should be noted is the possibility of not finding a solution or that the found solution might be longer.

3.2 Dynamic Signal Control

Various publications attempted the use of traffic information to generate a sequence of green light signals prioritizing the emergency vehicle. [16] proposes establishing communication between traffic lights and emergency vehicles such that they are prioritized at intersections and always given green lights. [17] proposed using light emitters that notice problems such as blocked line-of-sight and excessive noise. Similarly, [18] used technologies such as IR, and GPS to estimate traffic density and emergency vehicles presence. Others like [19] combined measurements of the distance between the emergency vehicle and an intersection using visual sensing methods, vehicle counting and time-sensitive alert transmission within the sensor network to plan for traffic signals sequence. They concluded that using Euclidean distance outperforms other techniques such as Manhattan distance and Canberra distance.

These approaches show promising results. However, they have many drawbacks such as not considering non-emergency vehicles' individual goals as they might be stuck in a red light due to poor planning for the emergency vehicle. Moreover, they are useless if the congestion is due to an accident for instance, and suffer from varying planning strategies in different coverage areas with each having its own intelligent control center.

3.3 Independent Vehicle Planning

In [20], David L. et al discusses cooperation between autonomous vehicles to perform lane merging with no communication needed in their paper "Tactical Cooperative Planning for Autonomous Highway Driving using Monte-Carlo Tree Search". The paper starts with presenting different rule-based algorithms such as "finite state machine" that were previously adopted to solve the problem. Then it notes that these algorithms don't suit the uncertainties in the input. Hence, they proposed using "Monte-Carlo Tree" MCT search to parse a tree of possible actions taken by each agent to minimize a joint cost while minimizing its own cost. Each agent, named "ego" in the literature, observes an environment of other interacting agents marked in blue, along with vehicles that are influenced by its decision marked in white.

The authors utilized the powerful capabilities of MCT while adding tactical and cooperative motion planning, TCMP, to come up with TCMP-MCT algorithm. In TCMP-MCT, A simulation period between two consecutive actions yields a combined cost function to guide the enhancement of both, the agent's plan, and the collective plan of the whole system. The main problem with this algorithm is that actions are taken sequentially by the agents in order for them to be organized in a tree form. Such a model should result in a behavior where agents learn to follow the actions of their neighbors rather than executing a joint simultaneous action. The proposed methodology proved successful in experiments concerned with planning highway driving scenarios that have a partially observable MDP, POMDP, nature. However, it lacks the powerful merit of V2V communication. Vehicles act independently with no cooperation and hence in real action, their plans might contradict.

3.4 Reinforcement Learning as an Alternative

In light of the previously discussed literature review and the research gaps identified, we found that RL offers more promising approaches. In recent years, RL algorithms were capable of competing with the previously dominant supervised learning techniques as they need little to no intervention from humans. In [21] for instance, a deep RL approach plays the game of AlphaGo based only on trial and error as an RL learning methodology. Also, the work of [22] develops a variant of Q-Learning, deep Q-Learning (DQRL), to play Atari on the Arcade Learning Environment. The algorithm not only outperforms all other existing algorithms in six games played, but it actually outperforms human players in 3 of them. This trial and error nature of RL algorithms makes it less problem defined, and hence easily transferred to other tasks. In addition, RL gives space for multi-agent learning, MARL, where connected agents can perform even better. [23] suggests that independently training single agents might show efficiency in certain tasks, but it's ultimately optimal to make the agents coordinate together.

In a MARL setting, multiple vehicles can use the availability of V2V communication to cooperate on clearing the road for the vehicle. However, with cooperation comes many questions; such as centralization and decentralization for instance. Who should give the orders? A central unit, a leading agent, or it's a form of anarchy where agents independently make decisions affecting others. An answer to this question was found in "Reinforcement Learning State-of-the-Art" by M. V. Otterlo [11], where the problem is defined as an MDP game. The book proposes using value iteration based algorithms that could help arrive at nash equilibrium between players (agents) such as joint action learner (JAL). In JAL, a joint action is taken based on observing both the environment and a probabilistic estimation of the possible actions made by the agents learning from previous experience of such state-joint action encounters. A noteworthy work on MARL, [24], uses a DQRL approach with a value decomposition network. Thus, Q function estimation would depend on the addition of all agent's Q functions to relieve the problem of conflicting behaviors or lazy agents.

Chapter 4

Project Design

In this chapter, we discuss the overall project design of our system, the tasks, and the used tools for each task.

4.1 Technical Specifications

In this section, we explain the project’s technical specifications regarding autonomous functionalities, communication, and cooperation. Autonomous functionalities deals with the design of an autonomous car that is capable of decision making. Communication deals with how information about every AV is transferred to its neighbouring ones regarding the message structure. Cooperation deals with the solutions to make the AVs clear the traffic for the EV through reinforcement learning.

4.1.1 Environment and Agent Model

In this subsection, we introduce our simulation environments: Gazebo [5] and SUMO [4], each of which was chosen for a different specific purpose. In order to simulate the realistic dynamics of our vehicles and implement the different autonomous functionalities they should be capable of performing, we chose to use Gazebo on the Robotics Operating System (ROS)[6]. On the other hand, for the sake of training and testing our reinforcement learning model and measuring aggregate system metrics, we chose SUMO. All the same, upon fully training the model, we then deploy it on our fully functional autonomous vehicles in Gazebo.

For each of the two simulation environments, we provide a brief description as well as present the used tracks (assumptions/ specifications/dimensions). We also demonstrate our agent model (sensors and mechanics) in both Gazebo and SUMO.

4.1.1.1 Gazebo

- **Gazebo Environment**

Gazebo is an open-source 3D multi-robot simulation environment[5]. By far the most attractive characteristic of Gazebo is that it is compatible with the Robotics Operating System (ROS) which we are using. ROS is an open-source, meta-operating system used for multitudes of robotics applications[6]. Owing to the several useful services it provides, such as hardware abstraction and

inter-processes message-passing[6], ROS is used as our basic framework in developing our system of autonomous vehicles.

From another perspective, what makes Gazebo quite an enticing choice of a simulation environment is its remarkable degree of realism[5]; it greatly cuts down on the effort of transferring simulated systems into functional ones on real hardware. This can be inferred from its following features:

1. Gazebo allows the rendering of realistic 3D environments, allowing for the simulations of textures and lighting[5]. In order not to complicate our simulated scenarios, we used Gazebo to design our track to be a simple straight three-lane road as demonstrated in Figure 4.1. The lanes are numbered 0-2, where lane 0 is the left-most one. Since our adopted agent model is that of a vehicle scaled down ten times, as discussed in 4.1.1.1, our road width is set to 1.575 meters (each of the three lanes is 0.525 meters wide) and our road length is set to 100 meters.

In order to better define and simplify our simulated track, the road was designed with the following underlying assumptions:

- The ground is smooth; no bumps or pits exist.
 - The track is perfectly straight without any intersections or merges.
 - The road is black and the lane lines are white to facilitate lane detection.
 - The road is sufficiently lighted.
 - There are no pedestrians involved.
 - Other vehicles are the only dynamic obstacles on the road.
2. Gazebo provides a simple API for adding any new model to the environment. Models include anything from arbitrary stationary objects to whole dynamic robot models[5]. In our case, an example of added stationary models are the sidewalks and construction barrels, while autonomous vehicles are our only simulated dynamic objects.
 3. Gazebo is built on top of The Open Dynamics Engine, a high-performance physics engine which allows for simulation of the realistic dynamics and kinematics of rigid bodies[25]. This allows us to simulate our autonomous vehicles and their control with a relatively acceptable degree of accuracy.
 4. A very important aspect of Gazebo is that it not only allows for simulating numerous actuators and sensors[5], but it is also capable of obtaining noisy measurements from these sensors so that their performance better matches that of the real hardware sensors.
 5. Gazebo supports more than one choice of update rates of the simulation environment. The update rate could either be manually specified by the user, or the simulator could be prompted to attempt running in real time or as fast as possible. In our simulations, we use Gazebo's default setting; which is attempting to run in real time or as fast as the machine can handle.

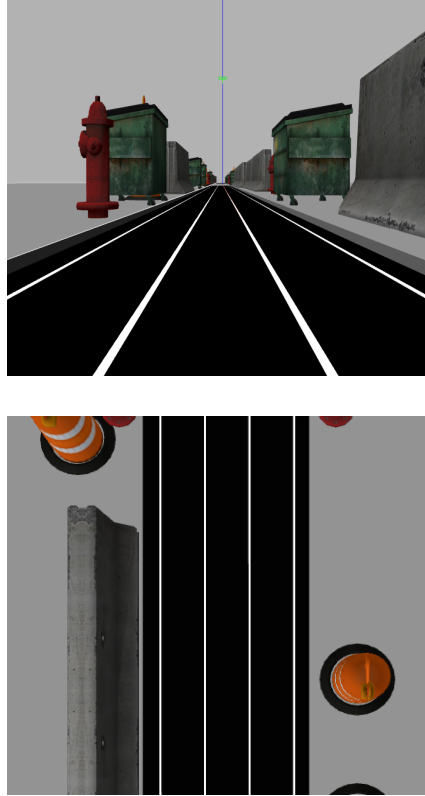


Figure 4.1: Straight three-lane road simulated in Gazebo.

- **Gazebo Agent Model**

Our Agent model is adapted from the F1/10 platform which introduces a fully autonomous Ackerman steering race robot inspired by the formula one race car. It is named F1/10 as it is 1/10 scale of an actual formula race car. This platform helps in making vehicle autonomy in a way that makes it easy to develop and implement different algorithms like perception, planning and control. F1/10 platform was first created in 2015 and is being continuously updated by its developers to break into autonomous racing. It is an open source project and is well documented on the official website f1tenth.org [9][26].

Our agent in Figure 4.2 is mimicking an actual 1/10 RC race car, traxxas xo-1, which is similar to the Tesla car Model S. Thus, we make sure that our model will have similar dynamics to that of a full scale car. Moreover, the sensors used in the model are similar to those used in full scale autonomous cars except for their limited specifications. The Agent model has the following sensors; Hokuyo Laser Scanner, Black and White camera, and Vesc: an open source electronic speed controller [9]. The sensors used in this model are for different purposes and are fused together to have a fully functional car:

1. **The Laser scanner**

The Laser scanner is used for obstacle detection and can be utilized in mapping, localization and navigation [27]. The one in our model is 270° wide detection angle with minimum range 0.1m and maximum range 10m

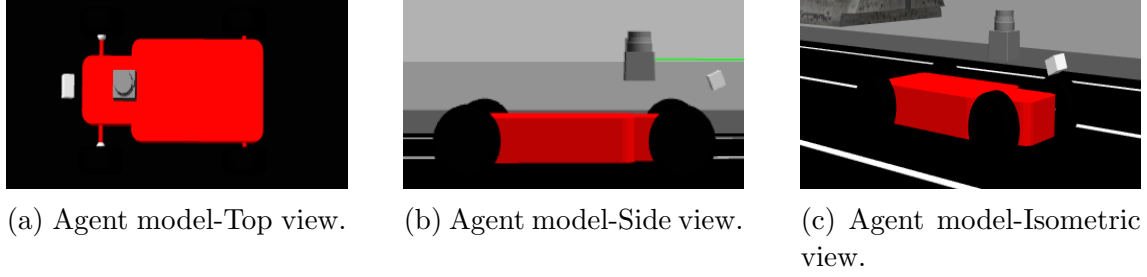


Figure 4.2: Agent model spawned in Gazebo environment

at a 0.01 resolution, see Figure 4.3. It is the grey object at the top of the AV in Figure 4.2.

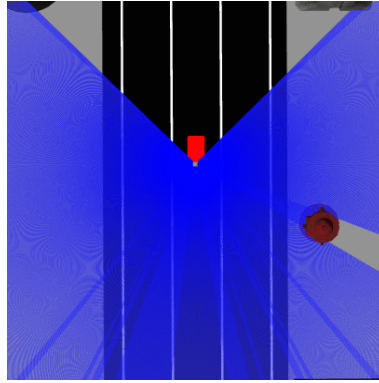


Figure 4.3: 270° wide detection angle of the laser scanner.

2. The Black and White camera

The camera is a black and white camera with image size 1280x1024 pixel. At the front of the vehicle, 0.08m above its center, with negative angle of 0.4 radians for it to have a good view of the lane lines. Also, its horizontal field of view is about 1.4 radians.

3. VESC

The vesc is used to drive the brushless dc motor and control the speed, current, voltage, PWM and different parameters [28]. It also controls the servo motor and the steering accordingly, and publishes the vehicle velocity and yaw.

Our Agent motion model is an Ackerman drive model [9]. In order to avoid problems with wheels slipping and the inaccuracies with the robot motion; ackerman model was introduced in [29][30], see Figure 4.4. This is done by maintaining a reference centre point about which all the tires are rotating, and the angle of each tire with respect to this reference point varies from one tire to another; $\phi_R > \phi_L$ in Figure 4.4. Also, each tire path has a different radius curvature to the reference point; each tire has different rotational speed

accordingly. Under this design, the vehicle can rotate smoothly on high speeds without facing slipping problems, also avoiding any rolling over[29].

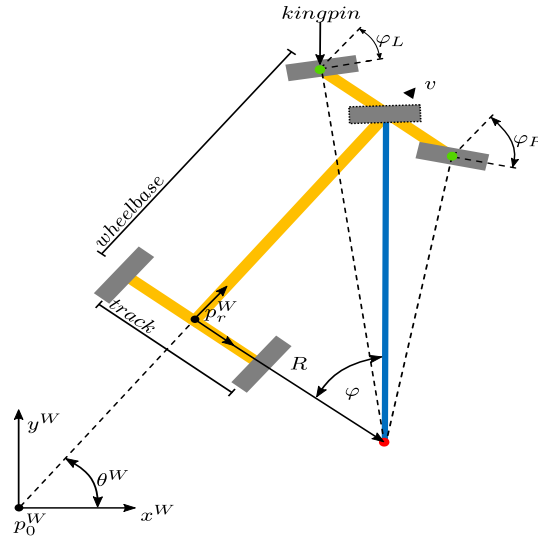


Figure 4.4: The geometry of the ackermann robot in the two dimensional space [29].

4.1.1.2 SUMO

• SUMO Environment

SUMO simulation is an open source microscopic and continuous simulation package. It is mainly built for traffic management and vehicular communications. SUMO[31] provides the functionality of building roads, controlling their traffic signals and controlling Vehicles along with facilitating communication between them. SUMO developers, mainly from the Institute of Transportation Systems at the German Aerospace Center, built a python API that is regularly updated through active discussions with the community. SUMO suite consists of four main applications:

1. Road Network Generation

SUMO road networks mimic real world roads by representing them as graphs where intersections are treated as nodes and roads are treated as edges. We used the SUMO application “netgenerate” to design a simple road network of three nodes and two edges, similar to that highlighted in Figure 4.1 in the case of Gazebo. The first two nodes are used for vehicle placing with the first used for initializing the EV while the second is used for initializing agents. A distance ranging from 40 to 250 meters separating the two nodes was enforced to allow for variant interaction scenarios between the agents and the EV. Finally, the third node was set to be the end of the track where vehicles exit the simulation.

2. Vehicles and Routes

Being a micro-simulation package, SUMO provides the functionality for deploying individual vehicles with a given set of parameters such as departure time, vehicles geometry, maximum allowed speed and vehicle types. For our purposes, we define two vehicle types:

- **Emergency vehicle:** To which the EV belongs.
- **Ego vehicle:** A type defined for agents in the simulation.

In addition to the parameters mentioned, each type is assigned a route to follow consisting of the start node, the destination node, and the edges connecting them.

3. Simulation

The application “sumo” launches a simulation of a previously configured network, our 3-lane network in this case. The simulation time is discretized, with a default simulation step size of 1 second. During simulation, vehicles navigate through the network according to the route specified by their type.

We used the application “sumo” to perform a series of simulation epochs in our RL algorithm to allow agents a near realistic experience with the environment. Hence, rewards and Q table updates are iteratively performed after every epoch.

4. On-Line Interaction

SUMO provides the ability to interact with an external application via a socket connection. This is done through the python interface TraCI: “Traffic Control Interface”, which facilitates controlling and retrieving vehicles states such as: speed, lane, position, as well as communicating this information between the vehicles. We also made use of TraCI in checking action feasibility. For instance: checking whether an agent is able to change lane (left/right) or accelerate.

• SUMO Agent Model

As discussed, SUMO is a microscopic simulator used to measure macroscopic properties of a traffic system; therefore, it provides control over vehicle actions, but in an accessible interface and -more importantly- simplified mechanical models. In the paragraphs below, we discuss the simplified vehicle-modelling approach allowing SUMO to achieve its goal in a computationally efficient manner.

1. First and foremost, SUMO is a **top-view 2-dimensional simulator**, so it completely ignores any vertical (z-axis) forces. The concept of road-bumps or suspension mechanisms, for example, is non-existent. This is justified by the fact that these forces are irrelevant to the goal of traffic simulation, they are only useful for mechanics simulation, which is addressed in our ROS implementation.

2. Since SUMO allows microscopic actions (eg: Lane change, Lane keeping, acceleration, and deceleration), it must adopt an agent-mechanics model; the vehicle bicycle model is chosen for this purpose.
3. SUMO is a discrete-time continuous-space simulation environment, meaning that it updates the vehicles' positions in time-steps Δt moving them by a positional increment $\Delta x(t) = \Delta t \cdot (v(t) + \Delta t \cdot a(t))$, where $a(t)$ is the acceleration chosen by the underlying car-following mode (see Point 4). This equation (called first order Euler scheme) simplifies the mechanics by assuming that the vehicle travels at a fixed speed within a time step and changes this speed instantaneously in between two steps. The instantaneity of actions is discussed more in Point 5.

The "discrete-time" property also means that any possible actions to be taken by the agent should take the same amount of time = 1 time step = Δt , which is a simplifying assumption allowing granular yet simulator-orchestrated user-control for each vehicle; where the user gains control over each vehicle after each time step, further facilitating its use for RL applications.

4. For longitudinal control, SUMO implements a set of car-following models[4], assigning Krauss model[32] by default, which is the one used in our autonomous agents; it is not used in the case of RL agents though- more on that in Point 7. The model, further discussed in 4.1.2.2, produces a safe desired "follow velocity" for the following (i.e. behind) vehicle to undertake, allowing it to maintain minimum-gap from the leading vehicle at all times. In the absence of a leading vehicle, the following vehicle uses its maximum allowable velocity.

For lateral control, SUMO's default lane-changing model[33] is used, with turning-off all automatic (strategic/ cooperative/ tactical/ regulatory) lane changes, specifically using lane change mode **512** from SUMO's options.

5. One simplifying and helpful feature of SUMO is instantaneity: At the moment the user retrieves control over each vehicle from SUMO (between each two time steps), the user can utilize SUMO's functionalities to set the speed of the vehicle for the next step instantaneously, as opposed to increasing the velocity gradually over the next time step. This reveals one simplification of SUMO: it does not simulate based on force/mass but rather on velocity/acceleration, ignoring some dynamical aspects such as inertia and force-reactions.

The instantaneous speed, however, is limited by the agent's maximum possible velocity, acceleration, and deceleration.

6. For each individual vehicle, SUMO provides a set of controllable parameters, the ones we used were:
 - vehicle's maximum speed: set to 10 cells/sec in the EV, and 5 cells/sec in the agent vehicle.
 - vehicle's maximum acceleration: set to 2 cells/sec² for the EV, and 1 cells/sec² for the agent vehicle.
 - vehicle's minimum acceleration (maximum deceleration): set to -2 cells/sec² for the emergency vehicle, and -1 cells/sec² for the agent

vehicle. It is important to note here that: vehicles in SUMO can not move backwards, so that the minimum possible velocity of any vehicle is zero.

- minimum gap for following vehicle to keep: 2.5 cells for both emergency and agent vehicle.
- vehicle length: set to 2 cells for both EV and agent vehicle.

The larger speed for the EV compared to our agent is assigned in similarity to typical real environments, where emergency vehicles are allowed larger speeds. These parameters are maintained in all our experiments.

7. On associating each of our agents with a control model, in our experiments we use the models defined below:
 - **SUMO_KRAUSS** : name for the SUMO-controlled autonomous vehicles. These vehicles have automatic lane changes turned off, and try to reach their maximum safe speed using Krauss model mentioned in Point 4. These vehicles are also **not** aware of the presence of the EV. They also avoid making any accidents.
 - **Q_LEARNING_SINGLE_AGENT** : Our RL-controlled vehicles. These vehicles have automatic lane changes turned off, and leave the option for the RL algorithm (Q_learning, in our case) to choose between the list of its possible actions given the observed state; more on this later. These vehicles are aware of the presence of the EV. They also avoid making any accidents.

4.1.2 Autonomous Functionalities

In this subsection we explain how the autonomous functionalities are designed. The explained functionalities are: mapping, localization, longitudinal control, lateral control, lane keeping, lane changing, path planning, navigation, external commands following, and the high-level control master.

4.1.2.1 Mapping and Localization

The AV has to be capable of mapping its environment as well as localize itself in it. The following points discuss the main approaches used for those purposes, which we also adopt in our implementation.

1. Mapping

Gmapping is probably most used in SLAM algorithm, its problem formulation has the observations and odometry measurements as the givens and the posterior and a grid map as the result. The main key idea of Gmapping is that it depends on an advanced particle filter algorithm called Rao-Blackwellized particle filter [34][35]. Each particle is considered to be a sample of history of robot poses and posterior over maps given the sample pose history; approximate posterior over maps by distribution with all probability mass on the most likely map whenever posterior is needed. Each particle in the particle filter carries an individual map of the environment and hence contributes to generating a collective output map [36].

2. Localization

In order for the robot to navigate in a provided map, it must first be able to localize itself in it. This can be done using the AMCL algorithm. The AMCL algorithm, or the adaptive Monte Carlo localization algorithm, is a probabilistic localization algorithm which uses a particle filter to determine the robot position and orientation in a given map [37]. The AMCL algorithm takes as input the map, the robot's laser scans, the mean and covariance of the robot's initial position and the transform messages. The algorithm outputs the mean and covariance of the robot's estimated position, the estimated positions of the particles in the filter, and the transformation [37][38]. Note that the AMCL algorithm has a lot of parameters, like the minimum and maximum number of particles that we set to be their default values.

4.1.2.2 Longitudinal Control

One primary functionality that our autonomous vehicle should be capable of performing is regulating its driving speed as necessary. It has to be able to determine the appropriate velocity that it needs to move with at all times and then carry out the suitable throttle/brake commands accordingly [39]. This is achieved by the vehicle's longitudinal controller.

For the velocity generation task, we chose to implement the car-following model named Krauss model [40]. It calculates the "follow velocity" with which the vehicle needs to drive at the time in order to guarantee a sufficient gap between the leading vehicle and itself. It ensures that even if the leading vehicle used the maximum deceleration at any time instant, the vehicle would manage to accommodate, maintain the minimum gap and avoid any unsafe maneuvers. To realize this, the model needs to know the current velocity of the leading vehicle and its maximum deceleration. Thus, it makes use of the vehicle-to-vehicle communication established between vehicles, discussed in 4.1.3, to access such information. In case the vehicle is not preceded by any others, it is prompted to use the maximum allowable velocity at the time; based on its current velocity and its maximum possible velocity and acceleration.

After determining the desired velocity, a simple lower-level proportional (P) controller is used to determine the required throttle in order for the vehicle to maintain this velocity.

4.1.2.3 Lateral Control

In order for our autonomous vehicle to be able to follow a certain trajectory, pre-defined by any of its higher-level modules, it needs to be able to control its lateral motion by issuing the appropriate steering commands at every instant. These steering commands ought to track the changing curvature of the followed trajectory as well as account for any possible accumulation of errors [41].

For this purpose, we chose to implement the Stanley controller; one type of the larger category of geometric lateral controllers which take into consideration the

geometry of the reference trajectory and the kinematic models associated with the vehicle [42]. The Stanley controller accounts for the error in the vehicle's heading alignment, as well as the cross-track error between the center of the vehicle's front axle and the nearest trajectory way-point [42]. It then limits the output steering command such that it does not exceed the maximum allowable steering angle [41].

After our Stanley controller module determines the required steering angle, we pass it, together with the vehicle's current one, through a lower-level proportional-derivative (PD) controller in order to apply the necessary correction to properly direct the vehicle.

4.1.2.4 Lane Keeping

One of the main higher-level control tasks that the autonomous vehicle ought to be able to perform is lane keeping. Unless it is given a reasonable cause to do otherwise, the vehicle should be concerned with driving at the center of its current lane, with the appropriate velocity. The lane keeping task has two main components, the first is the perception and planning, the second is the lateral control discussed in the lateral control subsection. In this subsection, we discuss the perception and planning part.

As a first step to implementing this autonomous functionality, the vehicle should be able to identify the lane lines. For this purpose, we chose to implement a vision-based approach in which the vehicle uses images from the camera placed on its front-most end to detect the bordering lines of the lane and consequently estimate the lane center-line.

In order for the vehicle to keep this detected lane, way-points are generated along the estimated center-line. These way-points are then passed to the vehicle's lateral controller, discussed in 4.1.2.3, so that the vehicle takes the appropriate steering commands to follow them. Lastly, the lane keeping velocity is determined by the vehicle's longitudinal controller in 4.1.2.2.

4.1.2.5 Lane Changing

Lane change in many real life applications in autonomous driving is considered to be a separate controller not a module in the main controller system. It takes into consideration many aspects more complex than we simulate them in our problem formulation [43]. Accordingly, we designed our own algorithm to fit different scenarios for the lane change action.

We make use of v2v communications in checking lane change feasibility, we assume the worst case scenario; other vehicles decide to take extreme actions in the coming time steps. The algorithm takes into consideration the ego vehicle position and velocity and other surrounding vehicles position and velocity.

4.1.2.6 Path Planning

Given an approachable goal, the autonomous vehicle should be able to plan a global path that avoids static obstacles and a local path that avoids dynamic ones so that it

can then accurately follow it using its navigation module. To achieve these goals, we based our global and local path planning module on the system published in [44], in addition to the applied modifications to suit car-like robots presented in [45]. Next, we introduce our own two main adjustments to the system. The aim of the first adjustment is to enhance the system performance making use of the established vehicle-to-vehicle communication. The second adjustment tackles the fact that the vehicle has to stick to lanes and cannot move diagonally through them; hence it aims to translate the planner's generated plan into lane keeping and lane changing commands.

In order to be able to generate an obstacle-free path to follow, the chosen system makes use of an occupancy grid which stores information about the vehicle's surrounding environment in the form of costs for each of the grid cells [44][46]. These pieces of information include the static obstacles present in the environment as well as the dynamic obstacles identified by the vehicle's raw sensor data.

The global and the local planners make use of a global and a local version of the occupancy grid to generate the needed paths. For the global planner, we chose Dijkstra's algorithm; it works by finding the path with the minimal cost between the starting point and the ending point [47]. The global planner does not take into consideration the dynamics or the kinematics of the robot [44], but the local planner does as it follows the Timed-Elastic-Bands algorithm (TEB) [45]. TEB algorithm optimizes robot's path through minimizing the trajectory execution time and distance from obstacles in addition to following kinodynamic constraints which make it fits car-like robots [45].

Building upon this chosen system architecture, we added the following two major adjustments:

1. Adding a Communication Costmap Layer

The occupancy grid encoding information about the surrounding environment is in the form of a master grid-based costmap. This master costmap is an accumulation of costs provided by an ordered list of costmap layers; each tracking information of a certain aspect or constraint of the environment [46]. The three main costmap layers incorporated by our chosen system are the static layer, the obstacle layer and the inflation layer.

The static layer accommodates information about the static obstacles in the environment; this is in practice the environment map previously discussed in 4.1.2.1. For the obstacle layer, the system uses a sensor processing pipeline which applies filters to convert the raw sensor data into obstacles information and store these pieces of information in a voxel grid (an efficient three-dimensional occupancy grid). The 3D voxel grid columns are then projected into the 2D obstacle costmap layer [44]. Lastly, the inflation layer adds a buffer-like zone of medium cost around grid-cells previously marked as occupied by the other two costmap layers [46].

In order to enhance the system performance, we propose to make use of the fact that vehicles are capable of communicating their positions through the

communication system we are establishing between all vehicles in the environment, as further discussed in 4.1.3. Consequently, our adjustment is to add a new costmap layer, the communication layer, which carries this piece of information received through the communication channel. Upon integrating this layer with the rest of the costmap layers, the autonomous vehicle's path planning module is able to generate a path which takes into consideration the position of all preceding vehicles within the communication range.

2. Adding a Custom Planner for Lanes

Reaching this point, the AV could navigate freely in any cluttered environment, taking into consideration other AVs places but does not abide by lanes as the local planner outputs velocity and orientation commands [44]. Accordingly, we translated the AV's position and a point belongs to the local planner trajectory into lanes, where any point can fall in only one of three lanes, and compared both points to know whether the car should lane keep or lane change.

Because the navigation system gets feedback from the robot position every time step, the local planner adjusts itself to keep following the global plan trajectory as much as possible.

4.1.2.7 Navigation

The robotic systems are MIMO (multiple-input multiple-output systems), they should handle parallel tasks, preempt when necessary, and have a solid base of communication between different nodes. ROS provides multiples of system architectures that depend on types of communication between nodes [6]. We chose ROS actions to build the navigation system upon. ROS actions represent an interaction between a server and a client transferring three types of messages: goal, feedback, and result [48]. ROS actions are preemptive in their nature and can handle complex systems. Actually, the path planning module uses ROS actions [6].

The designed architecture is composed of a master handling three clients which are communicating with three servers to send goals such as lane keeping with a certain velocity or lane changing. The chosen action in any time stamp is generated by our customized planner introduced in 4.1.2.6. The three action servers client systems handle:

1. Lane keeping
2. Velocity controlling for lane keeping
3. Lane changing

The navigation system has its own logic so that the AV has to be executing an action in any time stamp.

4.1.2.8 External Commands Following

Beside navigation, the AV may need to follow external commands for three reasons:

- Testing a single action

- Ensuring AV's safety, e.g, if a certain action has to be enforced to avoid crashing
- Following models, e.g, RL model introduced in 4.1.4

The external commands are:

1. Left Lane Change
2. Right Lane Change
3. Accelerate
4. Constant Velocity (No Acceleration)
5. Decelerate

The external commands following module has the same architecture introduced in 4.1.2.7; it deals with the same servers (lane keeping server, velocity controlling server, lane changing server) but it has three different clients. The chosen action is generated by an external system that wants to take control over the AV. If the external commands following module is activated, the AV can no longer take any decisions on its own. Meaning, if an action is infeasible, it waits until it has been commanded to do another action that is feasible. There is no other logic the AV follows except following an external system.

4.1.2.9 High-Level Control Master

The high-level control master module manages two sub-modules: the navigation module 4.1.2.7, and the external commands following module 4.1.2.8. Therefore, the high-level control master module should contain both architectures presented in 4.1.2.7 and 4.1.2.8 making it include the six clients. Every server has two clients: one to communicate with navigation module and the other to communicate with the external commands following module. Also, it prioritizes the external commands following module over the navigation module.

It receives two types of messages from external systems:

1. A stream of messages coming from the path planning module 4.1.2.6.
2. A request message coming from an external system such as cooperation module 4.1.4.

4.1.3 V2V Communication

For many of the autonomous functionalities we are implementing in our vehicles, we chose to depend on exploiting pieces of information shared among vehicles through communication channels. In the past several years, vehicle-to-vehicle (V2V) communication has been implemented using the dedicated short-range communications (DSRC) technology [7][49], which is based on the IEEE 802.11p standard: an enhancement of the IEEE 802.11 Wi-Fi standard meant to support Intelligent Transportation Systems (ITS) applications [49]. Although the adaptation of Wi-Fi technologies to fulfill the requirements of vehicular communications is a quite interesting

and challenging area of research [7][50], it is beyond the scope of our project. Our main interest is simulating this exchange of messages in order to invest in the employment of the shared information.

The design of our simulated communication system enables each vehicle to broadcast, with a certain frequency, a message of pre-defined structure containing useful information relating to its current status. It also receives messages broadcasted by other vehicles which are within a certain radius representing the vehicle's communication range. An assumption in our design is that vehicles do not re-transmit received signals; thus each vehicle's access to messages is strictly restricted to those sent by vehicles within its communication range.

We designed the structure of communicated messages such that it includes key details about each vehicle's standing, such as its velocity, position and lane number, as well as other general specifications, such as its maximum velocity and acceleration. As discussed in 4.1.2, those pieces of information are then used by the other vehicles either to calculate their safe lane keeping velocity and the feasibility of lane changing, or to perform more accurate path planning.

4.1.4 Cooperation Technique

As stated earlier, to clear the road for emergency vehicles, we allow cooperation between autonomous vehicles via communication such that an emergency vehicle can broadcast its location, lane number and velocity ahead of time. Consequently, preceding vehicles perform appropriate maneuvers to immediately clear the emergency vehicle's path. Preceding vehicles will choose their actions based on their trained reinforcement learning model.

As highlighted in Chapter 2, reinforcement learning is the sub-field of machine learning that enables an agent to learn a decision policy from a reward, enabling RL agents to achieve goals in complex and uncertain environments. Training the agent is a step only possible after a carefully studied problem formulation. We leave the details of our problem formulation to be discussed later in Chapter 5 (Methodology), and dedicate this section to highlight the main components of our used problems, environments, and agents.

To achieve cooperation between agents and the EV, we have divided our work into the following main milestones :

- Training single agent with Q-learning algorithm in a single agent environment
- Testing single agent with Q-learning algorithm in a multi-agent environment

4.1.4.1 Training Single Agent with Q-learning in a Single-Agent Environment

The goal of this stage is to make our single RL agent learn how to cooperate with the EV by taking all the necessary actions to evacuate the road for it .To train our agent, we have used Q-learning algorithm which is an off-policy RL algorithm that

selects the best action to take given the current state . For more information about Q-learning , refer to Chapter2 . In preparation for training our agent using a value based algorithm like Q-learning , We need to formulate our RL problem in terms of : states ,actions , and reward. Here is a subtle hint about them :

- States describe our system condition .They include both the agent state and EV state in terms of their lanes , speed , and relative distance between each other .
- Actions encompass agent's possible driving actions . Agent's possible actions are acceleration , deceleration , change lane to the left , change lane to the right , and continue as it is (no speed nor lane change is involved) . At any time step , feasibility of actions are checked so that the agent only chooses from a set of feasible actions .
- Reward is what learns the agent how to behave. In our case , it is a linear function of EV acceleration . The less the EV acceleration , the more negative reward the agent receives . This makes sense as if the EV could not apply its maximum acceleration, then the agent has stopped it from achieving it ; because the environment only contains both of them.

Training single agent was very insightful and helpful for testing environments and for learning process and was the first step towards training multi-agent system. We have just mentioned our problem formulation in an extremely brief and crude summary just to focus on the big picture . For more information about it , refer to Section 5.3

4.1.4.2 Testing Single Agent with Q-learning in a Multi-agent Environment

To test the Single Agent trained model in a multi-agent environment, the same Q-Table approach is maintained from the previous subsection, but in a different environment: one with many other vehicles and one EV, to evaluate the factors that could be improved about with our model.

Same as before, the agent can observe its own state as well as other vehicle's states; but now has access to action feasibility states from information about all other vehicles. Thus, cooperation here is only implemented through feasibility communication between different agents.

This is our second step towards building a Multi-Agent solution. Maintaining the decentralized execution approach. This approach is argued to be most suitable for partially observable environments [51], similar to the agent's case with limited communication.

4.1.5 System Integration

Reaching this point, we explained two important, yet unrelated, parts of our solution which are:

- Developing **cooperation techniques** on Sumo. The aim is to abstract the solution as much as possible to focus on the main details in the traffic clearance

details and training methods apart from the autonomous functionalities. For instance, the dynamics of the car and the latency in the system are ignored such as the fact that all actions take the same time. Therefore, this part gives a general, more abstract, solution that could be realized later on any simulation environment after adding the dynamics of the AV.

- Developing the **autonomous functionalities** on ROS and Gazebo. The aim is to add the car dynamics and real world complexities to the problem formulation of our solution in order to be applied on a real hardware. In addition, it takes care of how AVs' simulators may tend to crash more often than Sumo.

In this section, we present a framework for the single agent problem, refer to section 4.1.4, on ROS and Gazebo that do the same functions as RL in two separate sub-modules which are:

- **RL Algorithm:** We designed a module on ROS that can initialize Q-table, update Q-table, and send the high-level control master, explained in section 4.1.2.9, an action to be executed and take control over the AV.
- **Environment:** We designed a module on ROS that can interact with Gazebo environment and handle its more often crashes. It also gets agent's state, calculate the reward and handles when the agent gets inside the defined window around the EV.

After developing the frame work for the single agent RL problem on ROS and Gazebo that handles AV's dynamics. We designed a module that masters the training of the AV. This module uses RL algorithm sub-module and environment sub-module. The main function of the master are:

1. Begins an episode.
2. Starts or restarts the simulation environment (Gazebo).
3. Gets the needed states.
4. Take actions, calculate rewards, and update Q-table.
5. Save any required results.
6. Handles crashes or any corner case that will stop the simulation.
7. Synchronizes timing for all the previous

4.2 Design Alternatives and Justification

This section illustrates the motivation behind the selected design and the other design alternatives.

4.2.1 Environment and Agent Model

4.2.1.1 Gazebo

The primary characteristic we sought while choosing a simulation environment on which to develop our vehicle’s autonomous functionalities was that it ought to be as realistic as possible. This is absolutely essential since it allows us to test our developed algorithms in real complex scenarios in which various uncertainties exist. Consequently, the deployment of our algorithms on real hardware would be much straightforward. While Gazebo perfectly demonstrated this feature, there were a couple of available alternatives. These include:

- **CARLA:**

CARLA is an open-source simulation platform which supports the advancement and validation of autonomous driving applications [52]. It is one of the most realistic platforms which allows for not only rendering complex environmental specifics, but also fully specifying sensors and actuators [52]. The major drawback of CARLA was that its stable releases do not support the control of more than one agent; only development ones like CARLA 0.9.0 [53]. Since having more than one active vehicle in the environment was inevitable in our system, we discarded the option of using CARLA.

- **LGSVL:**

LGSVL is an open-source multi-agent simulator based on Unity’s game engine [54]. It allows for the creation and manipulation of photo-realistic 3D environments, supports customizable vehicle sensors and provides a basic dynamic model for vehicles [54]. Although LGSVL seems promising, there was no real incentive to favor it over Gazebo. In fact, owing to its extensive documentation and very wide supporting community, Gazebo seemed to be the best practical option.

As for our adopted agent model, there were many alternatives to choose from other than the F1/10 packages. Most of these alternatives also provide a hardware kit besides the software work[55][56][57][58]. One of the most relevant kits and packages to our work is the Multi-agent System for non-Holonomic Racing (MuSHR)[59]. This platform was designed for multi-agent autonomous racing. It was developed by the Personal Robotics Lab at University of Washington. Like the F1/10 platform, it is also inspired by the MIT racecar. It has the same sensors like F1/10; LiDAR, camera and VESC. most of the sensors used are the same except that the LiDAR is YdLidar; lower resolution one. The edge of this kit is that its cost is significantly smaller than F1/10. However, it lacks continuous and good documentation in contrast with F1/10 [9][59]. Thus, it is a good compromise if the target was a hardware prototype because of its low cost. All the same, F1/10 beats it as it is more robust, well documented and has continuous development.

4.2.1.2 SUMO

Before settling on Sumo to be our simulation environment for training and testing our RL models, we have considered other choices. These include CARLA, Simulane,

Unity, Open AI gym, and creating our environment. We will illustrate why they were not suitable for our project design.

1. **CARLA:**

CARLA is an open-source simulator for autonomous driving research [52]. The main important feature of in CARLA is that it provides a realistic graphical representation of buildings, streets, and urban areas. However, this realistic representation comes with a significant drawback: it needs high hardware capabilities. Since our project is not mainly dependent on image processing techniques, then CARLA's high graphical capabilities are just overhead for us.

2. **Simulane:**

Simulane is a highway traffic simulator which has been introduced in "Deep Multi Agent Reinforcement Learning in Open Homogeneous Population" paper [60]. It was intriguing that Simulane has been designed to simulate an environment similar to ours where multi-agents drive autonomously in the vicinity of each other. What stepped us back from using it is that it is not open source. After communicating with its developers, we have figured out that it is written in Python 2, and currently there is neither support nor update associated with it. Consequently, the idea of using Simulane has been dismissed.

3. **Unity:**

Unity is one of the most common cross-platform game engines developed by Unity Technologies. We have tried using it as our environment while implementing multi- agents that start from different initial points to reach various goal positions using A* algorithm. From this trial, we can see that Unity has two primary programming language to use Java and C-Sharp. Both are more demanding than other programming languages like Python, so they make the development progress slower. It is worth noting that Unity Machine Learning Agents toolkit has been recently released which is the first machine learning product offerings by Unity that trains agents with reinforcement learning via a simple Python API [61]. However, this toolkit is still in beta version and under development. Unity realistic physics engine is overhead and would make the training process very hardware demanding.

4. **Open AI Gym:**

Open AI Gym is a great toolkit for developing and comparing reinforcement learning (RL) algorithms [62]. It has done a great favour for the RL field by providing environments to benchmark RL algorithms. Unfortunately, we did not manage to use it as our environment design is not similar to existing environments. However, we have inspired the way of structuring code for our RL problem and how to make it independent from specific RL algorithm from their work.

After exploring the environments mentioned above, we have formed a clear vision about how we really want our simulation environment to be. First, it should give us

the freedom to organize our code the way we want without forcing a specific structure. This is important to make our code independent from specific RL algorithms in order to be able to test many different ones if we want to with minimal efforts. Second, the simulation environment should give us the capabilities to insert many agents, EVs and have roads that are divided into lanes. Third, offering an easy programming language to use like Python is a great plus.

After developing a comprehensive vision about we actually want, we thought of creating our own simulation environment. In parallel with this idea, we were investigating SUMO. We have seen that SUMO is not only consistent with our vision, but also has top-notch features. These features include autonomous features, so we are not concerned about teaching agents how to drive but rather teaching agents how to cooperate in evacuating road for the EV. Choosing SUMO to be our simulation environment has saved us effort to implement autonomous features and design graphical user interface.

4.2.2 Autonomous Functionalities

In this subsection, we discuss the design alternatives of the autonomous functionalities, and justifications of the selected design (perhaps how we simplified the autonomous problem).

4.2.2.1 Mapping and Localization

Some alternatives for GMapping are HectorSlam and KartoSlam. Gmapping and Hector slam are mainly laser-based algorithms, but kartoSlam is a graph-based algorithm [63].

The key concept of the gmapping is particle filter (PF), through which each PF is holding some information about the positions, while the key concept of hector slam is the endpoint beams optimization with the last produced map, in contrast to the kartoSlam, a graph-based algorithm where each node in the graph corresponds to different points through the timespan, and the graph edges represent the constraints between the consecutive changes [34][64].

We excluded the use of the kartSlam, as it is not well documented, and its package on ROS is fairly used among developers when comparing with the gmapping and hectorSlam [65].

Gmapping makes use of laser scan and the odometry data from the inertial sensors. Instead, hectorSlam depends only on laser scan. Although in our case we do not use real odometry data, we make use of laser_scan_match package, an incremental laser scan registration tool that can serve as a standalone odometry estimator[66][67]. We can consider it as a fake odometry data; thus, gmapping can make use of this fake data accordingly [68]. Moreover, hectorSlam is used mainly in indoor applications, as it depends on only laser scan data; however, if there is a long road or hallway without enough descriptive features, the results tend to be not accurate. In this case, we should increase sensor accuracy and decrease readings uncertainties by fusing the odometry data with the laser scans ones to get better results. This approach is done using gmapping. So, hectorSlam is good with indoor applications, in contrast with gmapping that is good with outdoor ones. Also, our problem scenario

is more relevant to outdoor applications, so we prefer using gmapping, too [69][70]. Furthermore, gmapping is well-documented with a lot of tutorials and used a lot by the ROS developers community.

For the localization part, there are not many other alternatives to AMCL. It is the most used localization package, very well documented and robust.

4.2.2.2 Longitudinal Control

For the task of appropriate safe velocity generation, which comes before the actual issuing of longitudinal control commands to follow this generated velocity, there were several other alternatives to the Krauss car-following model we chose to adopt. One of those alternatives is the Intelligent Driver Model (IDM) [71].

IDM is one of the simplest car-following models which belongs to the bigger category of safety distance models, to which also the Krauss model belongs [72][73]. However, instead of generating a follow velocity as is the case with Krauss model, it calculates the appropriate follow acceleration that allows for the safe following of the leading vehicle [73]. This is done through taking into consideration the follow vehicle's current velocity as well as the relative position and velocity between the lead vehicle and itself [73].

Although according to the tests in [73], the Krauss model was better than the IDM in strictly keeping the safe distance between vehicles, it was also shown that the IDM performed better in matching real driver behaviors, as opposed to the Krauss model which seemed to react faster than real drivers did. The proposed reason was that it was easier for drivers to estimate the distance to their leading vehicle than to estimate the leading vehicle's velocity and acceleration. Hence, their behavior better coincided with the IDM model since it more directly depends on the relative distance than the Krauss model [73]. However, this problem is irrelevant in our case owing to the fact that the established V2V communication will allow such pieces of information to be readily shared among vehicles.

It is worth mentioning, though, that the original Krauss model defined in [40] assumed that the deceleration profiles of all vehicles are the equivalent [32]. However, since this is not the general case, and we will need to simulate vehicles with different deceleration profiles, we were inclined to resort to an adjustment which embraces this reasonable diversity. This modification on the original Krauss model was also introduced by SUMO in their implementation of the model [74].

4.2.2.3 Lateral Control

As an alternative to the chosen Stanley controller, several other lateral controllers exist; two main of which are:

1. Pure Pursuit Controller:

The pure pursuit controller is another type of geometric controllers which chooses a reference point on the followed trajectory at a certain look-ahead

distance from the centre of the vehicle's rear axle [42]. Next, it calculates the required steering angle from the curvature of the arc connecting this reference point to the vehicle's rear axle center [42].

This controller shares many of the advantages and disadvantages of the Stanley controller, specifically with regard to our problem at hand. Consequently there was no tangible incentive to favour it over the Stanley controller. Additionally, the choice of the look-ahead distance is not quite straightforward and would require, for instance, to be a function of the path curvature or cross-track error in order to provide an enhanced performance [42]. Thus, Stanley controller was the better choice.

2. Model Predictive Controller (MPC):

The model predictive controller (MPC) belongs to the other main category of lateral controllers: the dynamic controllers [42]. It uses an objective function and the imposed constraints to numerically solve a horizon optimization problem at every time-step and arrive at the optimal control inputs to be applied at the time [75]. Since optimization takes time and the state of the vehicle is needed in the process, the controller should also be able to predict the vehicle's state at the time of applying the optimization result so that it can use it instead in its calculations [75].

The main reason for not adopting an MPC is that optimization solvers need to be capable of coping with the high update rates and narrow windows of time that the problem suggests in order to arrive at convenient results [75]. Obviously, apart from the increased complexity, this requires much more computational resources than can be provided in our case.

Although the Stanley controller is a relatively simple and effective controller that is not affected by the exact shape of the path, it has some disadvantages. These include [42]:

1. The output control action could get aggressive at relatively higher speeds.
2. The controller is not as robust when the followed trajectories are very rough.

All the same, in our case at hand, those drawbacks are not quite relevant since the curvature of the used track and the ranges of vehicles' speeds are all within the controller's limits of satisfactory performance.

4.2.2.4 Lane Keeping

Lane keeping alternatives could have been:

- Using a deep learning (CNN) model, with the camera image as its input and the desired steering angle as its output [76]. This approach was avoided because:
 1. It is more time consuming (due to the time that would be taken to collect the data)
 2. It is not general enough (will need to retrain and re-collect data if we change the track)

3. And most importantly, it would add computational complexity without any extra benefit to our simple case.
- Introducing Time Coherence to the lane detection methodology by maintaining a memory of the past detected lanes and integrating this information into the subsequent decisions, instead of detecting the lane in each frame. That is: use detection with tracking, instead of just detection.
- This was not implemented, however, as our vision for this module and the whole system was to provide a simple baseline solution, not a full system that can tackle all the complex situations; and the current solution could tackle the straight track efficiently. We therefore leave this part to future work and recommend that it gets implemented.

4.2.2.5 Lane Changing

There are many other alternatives algorithms for lane change but they are way too complex for our problem formulation or not that complex from their approach point of view but they are still complex to implement and integrate them with our environment and simulation settings. One of these algorithms is the virtual curvature method[43].

The main idea of this algorithm and its main edge is that it uses the same controller for both lane change and lane keeping actions. Accordingly, it avoids the headache of switching and synchronization between different controllers. The key concept of this algorithm is that it assumes there is a virtual road curvature in between the current lane and the target lane and it can be used as a lane transition curve. The vehicle follows this virtual road by the lane keeping controller, the only difference is the input being fed to the algorithm. This new input, the virtual road curvature, clips the original data from the hardware sensors, as this data is trying to apply the real lane keeping.

The virtual curvature algorithm assumes that the roads are two types, physical road curvature and a virtual one. For the physical one, it is used in the calculations when the vehicle is performing a lane keeping action; about to go to the transition lane or has just reached the end of the transition lane. The virtual road curvature is being used when the vehicle is in the transition lane itself.

The algorithm is mainly divided into three stages. The first one in which the vehicle is performing normal lane keeping in the original lane. In this stage the physical road curvature is zero so the vehicle is performing normal lane keeping. After then, it is prompted to apply lane change so go to the second stage of the algorithm. In the second stage, the algorithm uses the virtual road curvature and generates a virtual transition lane trajectory using a polynomial function that takes the longitudinal distance, lateral acceleration, and the lane width as its arguments. From this polynomial and the steering angle we can calculate the lateral position of the vehicle, and will use it to perform the lane change. According to this new lateral position, the vehicle will perform lane keeping as if it is a normal real lane keeping but the difference is that it will perform it following another reference other than the one that the hardware sensors are publishing. In stage three, the vehicle lateral

position will be reset to the default one as the hardware sensors take over and by this means there is no need to switch or synchronize between several controllers.

This algorithm may seem to be simple however it has some defects; as it does not take into consideration if there are any obstacles or not. Also, it relies on lateral sensors when it comes to transitions between different stages[43]. Thus, we implement our own lane change algorithm that best fits our problem formulation, make use of v2v communication and can be integrated easily with other modules.

4.2.2.6 Path Planning

- **Alternative to Path Planning System:** The navigation system published in [44] is the chosen system for the path planning module. It is based on probabilistic methods, occupancy grids (2D costmap layers) and laser range sensors. However, an alternative to this system, based on neural processes, is presented in [77]. The brain based navigation system can operate with low resolution monocular image data; moreover, instead of multi-layered two-dimensional costmaps, it uses pose cells, an experience map, and local view cells which are inherited from the cells responsible for navigation in rodents' brains.

Weighing the pros and cons for each method, the brain based navigation system was favorable for the choice of sensors as it can operate with low quality camera. However, the occupancy based navigation system was well-documented, robust and had a wide community support. These qualities counterbalanced the hardware reliability advantage found in the brain based navigation system. Furthermore, there is a well-established ROS meta-package for this navigation system.

- **Alternative to Global Path Planner:**
 - **Anytime Repairing A* Algorithm (ARA*):** ARA* is based on the heuristic algorithm, A* [78]. ARA* is an anytime algorithm which tunes its performance based on available search time. Henceforward, it is faster and works best for the cluttered environments, which is not the case for our environment. Therefore, using it adds unnecessary complication to the problem. On the contrary, Dijkstra's algorithm complexity was reasonable in addition to its well documentation and high speed.
- **Alternative to Local Path Planner:**
 - **Dynamic Window Approach (DWA):** DWA selects among possible commands based on a cost function that combines distance to obstacles, distance to the global path, and the speed of the robot [44]. However, this technique does not support car-like robots unlike TEB algorithm.

- **Alternatives to Our Adjustments:**

1. **Communication Costmap Layer**

An alternative to adding our new communication costmap layer, was

to depend on the vehicle's LiDAR sensor in detecting the surrounding vehicles and incorporating their positions in the obstacle layer. However, as will be further discussed in 4.2.3, the data received from the vehicle's LiDAR sensor is not only susceptible to distortion through noisy measurements, but also limited to the sensor's physical range. Consequently, using a much reliable source of data, like the communicated information, besides the sensor data greatly boosts the accuracy of our system.

From another perspective, a possibly useful feature of our communication layer is that it holds information about vehicles that are relatively far ahead of our autonomous vehicle, which gives a hint about the level of congestion in each of the different lanes. This feature can surely make it attractive to other likely future applications.

2. Custom Planner for Lanes

There are two approaches to process the navigation system outputs and translate it into lanes; either using the global planner or the local planner. The output of both planners in ROS package is an array of poses (position and orientation) [6]. The last pose of the global path is the goal while the last pose of the local path is in a certain window surrounding the AV. The AV could check one pose every defined number of poses and translate it into lanes. The main problem in this approach is that the global planner path does not get updated as fast as the local planner path. Thus, the local planner path is more flexible with the system's dynamic obstacles, cluttered environments, curves and intersections, and AV's kindodynamics.

4.2.2.7 Navigation

An alternative to ROS actions is ROS services which is also a way of communication between a server and a client but it depends on a request and a response [6].

There are two main differences between ROS actions and ROS services that favoured ROS actions:

1. **Preemption Possibility:** ROS actions allows preemption of the server; on the contrary, ROS services' servers must run till the end. Therefore, ROS actions design was better as the navigation system has real-time constraints such as a new dynamic obstacle appears in the way of the AV that it must lane change or decelerate to avoid crashing.
2. **Communicated Messages:** ROS actions has an additional message than ROS services to send feedback, e.g, the lane change server can send whether the lane change is feasible or not to the master in order to continue lane keeping or cancel.

4.2.2.8 External Commands Following

Regarding the system's structure, the external commands following module alternative was to use a separate system with new three servers and new three clients. Nevertheless, it would have been a waste of time and space and it will add complexity and latency to communicate between two large systems.

Also, the navigation module and the external commands following module can never run at the same time. Henceforward, they can use the same servers. On the contrary, the external commands following module doesn't use the same clients to improve the modularity of the code to separate external commands following and navigation modules.

4.2.2.9 High-Level Control Master

The high-level control master listens to two messages. Their types with justifications are:

1. Regarding navigation, the high-level control master has to receive a stream of messages coming from the custom planner for lanes mentioned in 4.1.2.6. The messages has to be continuous as the local planner gets updated at a high rate in order to be flexible with the dynamic environment.
2. Regarding external commands following, the the high-level control master has to receive the message as a request in the form of ROS service server. It cannot be a stream of messages because the external system that wants to take over the AV's control, such as 4.2.4, has its own logic of choosing control actions. Henceforward, it needs to know the feasibility of any given action and the time taken to execute it.

4.2.3 V2V Communication

The reason for establishing a V2V communication system is to allow vehicles to share essential information which includes, but is not restricted to, their position, lane number and velocity. An alternative to attaining this set of data through communication can be to depend on the vehicle's perception system to determine other vehicles' position and on velocity estimation algorithms to predict their velocities [79]. Unfortunately, the limitations of this alternative are drastic. They include the inaccuracy of sensor data, the limited range of the sensors, the existence of blind-spots due to the physical constraints of the sensors, and the inaccuracy in the estimation algorithm itself.

On the other hand, the proposed ongoing communication between the vehicles allows them to share together all pieces of information about their states at regular time intervals. Consequently, each individual vehicle has continuous access to a flow of updated information about its environment that is much more reliable and immune to not only the blind-spots and range of action of the perception system but also the constraints of estimation algorithms.

Another crucial significance of this established communication is that it allows for

sharing some other pieces of information that cannot be attained by any other means. This includes information like the other vehicle's maximum velocity and maximum acceleration; which are very beneficial to the used control algorithms.

4.2.4 Cooperation Technique

Before choosing reinforcement learning to be our approach for handling cooperation between vehicles, we have investigated algorithmic approach which is multi-agent path finding prioritization. Regarding path finding, we have chosen A* algorithm. A* is one of the best and most popular techniques for path finding. It is an advanced best first search algorithm that finds optimal path from starting point to end point. It searches for shorter paths first rather than longer ones [80]. Then for prioritization, we were planning to use the information about other vehicles' states (such as position and velocity) to prioritize the agents which can plan their way using A* algorithm, with the EV always having the highest priority. Then, the higher priority agents transmit their paths to the lower priority agents as constraints for their future plans. At this stage, we were using Unity as our simulation environment. After investigating this approach, we have formed the following vision. There are five main steps to achieve cooperation between vehicles:

- Adding multi-agents in unity who path plan using A*.
- establishing a communicator between them.
- applying an algorithm that prioritizes agents.
- transmitting path plans from higher priorities as restrictions.
- applying the algorithm in ROS on autonomous vehicles.

We have successfully implemented the first two steps. Consequently, we managed to make multiple vehicles drive from different initial positions to final positions while communicating their position and velocity. After implementing these two steps, we have decided not to proceed with this approach for the following reasons:

- A* algorithm is slow for a dynamic environment like ours where repeated planning is involved and real time driving decisions are made .
- We had to establish prioritization criteria. We figured out how it is very hard to choose a criteria that is optimal in all cases. For instance, if we agreed on the following criteria: The nearer the agent to the EV, the higher priority it has. Then, the nearer agents would transmit their paths to the further agents as constraints for their future plans. We can see that it is not always the case as a far agent could make action help evacuate road better. If its plan has a low priority, it will not be executed and hence the road will not be evacuated.
- Agents could be inserted near to the EV in case of road with intersections and hence have high priority. Consequently, agents with lower priority need to recalculate their plans and this is computationally exhaustive.

Due to the mentioned reasons, we have aborted this approach and migrated to Reinforcement Learning approach.

Chapter 5

Project Implementation

This chapter contains a detailed description of each of the system functionalities; including the algorithms and implementations.

5.1 Autonomous Functionalities

This section highlights the implementation of our AV's autonomous functionalities. Each subsection presents the algorithmic details of the corresponding section previously discussed in 4.1.2.

5.1.1 Mapping and Localization

As highlighted in 4.1.2.1, we adopt gmapping and AMCL as our algorithms for implementing the the mapping and localization functionalities of our AVs. This sub-section describes our implementation in more detail.

1. Mapping

Our map was created manually by moving the vehicle throughout the world using the keyboard, it was find that spawning the vehicle near the origin of the world will result in a better map during the mapping process. Usually creating maps with normal gmapping package can generate a map that has some defaults and needs to be improved to achieve better results further. One way to edit the map is to manually edit it using GIMP software [81]; a photo editor software that supports .pgm extensions.

2. Localization

We use AMCL package in localization, it takes the generated map as an input and tries to localize the the vehicle in it. One of the most important parameters is the number of particles where we chose to use to default configuration for this parameter; `min_particles = 500` and `max_particles = 2000`. Also, Default configuration were used in all other parameters.

5.1.2 Longitudinal Control

In this subsection, we explain our implementation of the Krauss model, previously discussed in 4.1.2.2, whose intended output is the velocity for the vehicle to undertake at each time step. This output velocity is then mapped to the required throttle output as in [82], and then passed through a simple proportional (P) controller to issue the appropriate final control action.

The inputs and needed parameters of our Krauss model are as follows:

- **Inputs:**

1. **Vehicle Position:** This is obtained from the vehicle's LiDAR sensor.
2. **Vehicle Velocity:** This is obtained from the vehicle's electronic speed controller (ESC).
3. **List of Other Vehicles' IDs:** This is the list of IDs received, through V2V communication (see 5.2), from all other vehicles within the vehicle's communication range. This is needed in order for our vehicle to be able to determine which vehicle, if any, is its leading vehicle and to save its data for use in calculations.

- **Parameters:**

1. **Vehicle Maximum Velocity**
2. **Vehicle Maximum & Minimum Acceleration**
3. **Vehicle Length:** In case a lead vehicle is found, this is needed in order to properly calculate the current gap between the follow and the lead vehicles. This parameter is assumed to be a standard set for all vehicles; i.e. all vehicles have the same length.
4. **Safety Inflation Factor:** This parameter is given as a percentage of the vehicles' length. It defines a value with which to inflate the actual length of the vehicles to be used in calculations; for the sake of increased safety.
5. **Lane Numbering Convention:** This is the convention of the lane numbering in the current map. It is needed in order for the vehicle to map its position coordinates to a lane number.
6. **Average Reaction Time:** In case of the presence of a lead vehicle, this is an estimated value for the time our vehicle takes to respond to a sudden change in the lead vehicle's behavior. It reflects on the gap our vehicle seeks to keep between the lead vehicle and itself.
7. **Delta Time:** This parameter specifies the time interval between iterations of the Krauss model in which it calculates and outputs the required velocity.

Using these inputs and specified parameters, the Krauss model calculates the desired velocity every delta time using the following steps:

1. With the lane numbering convention and the vehicle's coordinates in the map being known to the vehicle, the vehicle's lane number is calculated.

2. The model checks which of the other vehicles, if any, is a lead vehicle with respect to our vehicle. This is done by traversing through the communicated list of ID messages belonging to vehicles in the communication range, and checking which of them is ahead of our vehicle in the same lane. Of vehicles satisfying this condition, the closest one to our vehicle is considered our lead vehicle at the time, and its relevant data is stored to be used in calculations.
3. If a lead vehicle is found, the current gap between the two vehicles is calculated [40]. This is the actual physical distance from the front edge of our vehicle to the rear edge of the lead vehicle after adding the specified inflation to their lengths. Since all the provided vehicle coordinates are of the vehicles' centers, the gap (g) is calculated using equation 5.1, where x_l and x_f are the x-coordinates of the lead and follow vehicles respectively, f_{si} is the safety inflation factor and l is the length of the vehicle.

$$g = x_l - x_f - (1 + f_{si}) * l \quad (5.1)$$

4. Next, the maximum safe velocity with respect to the lead vehicle is to be calculated. This safe velocity is the one which guarantees that, under all circumstances and despite any extreme behavior that might be adopted by the lead vehicle, our follow vehicle can be brought to a complete stop without colliding with the lead vehicle [40]. In other words, this means that the total of the distance to be travelled by our follow vehicle while still reacting to the lead vehicle's actions and the distance to be taken by our vehicle to completely stop should be less than or, at the least, equal to the total of the current gap between vehicles and the distance to be taken by the lead vehicle to halt. This is demonstrated in equation 5.2 [40], where v_l and v_f are the velocities of the lead and follow vehicles respectively, $d(v)$ is the needed braking distance to stop a vehicle travelling with velocity v , g is the gap calculated in 3 and τ is our follow vehicle's average reaction time.

$$d(v_f) + v_f\tau \leq d(v_l) + g \quad (5.2)$$

Assuming both vehicles use their maximum deceleration, b_{max} , in order to halt, equation 5.2 can be re-written as equation 5.3, and solved for the maximum v_f which resembles the maximum possible safe velocity with respect to the lead vehicle.

$$\frac{v_f^2}{2b_{fmax}} + v_f\tau \leq \frac{v_l^2}{2b_{lmax}} + g \quad (5.3)$$

5. Another consideration that might limit the value of the velocity outputted by the model is that neither of our vehicle's velocity or acceleration should be allowed to exceed its maximum value. Consequently, the final output velocity in case of the presence of a lead vehicle is as calculated in equation 5.4 [40], where v_k is the velocity outputted by the Krauss model, v is our vehicle's current velocity, v_{max} and a_{max} are its maximum velocity and acceleration

respectively, Δt is the specified delta time parameter, and v_{safe} is the maximum safe velocity calculated in 4.

$$v_k(t) = \min(v_{max}, (v(t) + a_{max} * \Delta t), v_{safe}(t)) \quad (5.4)$$

In absence of a lead vehicle, the safe velocity consideration is irrelevant, hence dropped, as shown in equation 5.5.

$$v_k(t) = \min(v_{max}, (v(t) + a_{max} * \Delta t)) \quad (5.5)$$

A summary of the explained is provided in algorithm 1.

Algorithm 1: Krauss Model Algorithm (every Δt)

```

Determine the vehicle's current lane
Check for the presence of a lead vehicle
if a lead vehicle is found then
    | Calculate the gap between vehicles using equation 5.1
    | Solve equation 5.3 for the maximum safe velocity
    | Calculate the output model velocity using equation 5.4
else
    | Calculate the output model velocity using equation 5.5
end
    
```

5.1.3 Lateral Control

In order to implement the Stanley controller previously discussed in 4.1.2.3, the following inputs and parameters are needed by the controller in order to provide its highlighted output:

- **Inputs:**

1. **Trajectory way-points:** The way-points are sent to the controller from any higher-level module. In our case, this is either the lane keeping module providing the necessary way-points for the vehicle to follow in order to keep its lane, or the lane changing module providing the lane change trajectory way-points. These way-points could either be in the map coordinates or the vehicle-centric coordinates; the algorithm handles both cases.
2. **Vehicle pose:** The controller needs to know the vehicle's orientation in order to calculate the required steering angle. This orientation is sent to the controller from the vehicle's LiDAR sensor. If the coordinates of the trajectory way-points are given in the map coordinates, then the controller also needs to know the current coordinates of our vehicle in the same frame. Hence, they are sent to it from the vehicle's LiDAR sensor as well.
3. **Vehicle velocity:** The vehicle's current velocity affects the magnitude of control action needed to drive it to course. Thus, the Stanley controller receives the vehicle's velocity from its electronic speed controller (ESC).

- **Output:** The output of the controller is the steering angle required to follow the given trajectory at this instant. This output steering angle is then sent as an input to a lower-level proportional-derivative (PD) controller.
- **Parameters:**
 1. **Length between vehicle's center and the center of its front axle:** The Stanley controller takes the center of the vehicle's front axle as the vehicle's reference point in calculations. Consequently, since all other coordinates of the vehicle are those of its center, the length between those two points has to be known to the controller.
 2. **Steering angle limits:** The maximum and minimum steering angles allowed by the vehicle's hardware have to be known to the controller so that it can limit its output steering angle accordingly.

Using these inputs and specified parameters, we arrive at the desired output steering angle through the following steps:

1. We calculate which of the given trajectory way-points is the closest to the center of the vehicle's front axle. This point, without any look-ahead distance, is then the reference point of our trajectory, denoted (c_x, c_y) in figure 5.1.
2. In order to correct the vehicle's heading error relative to the trajectory, we calculate the angle θ_e between the vehicle's heading and the tangent to the trajectory at the reference point [42], as shown in figure 5.1.
3. In order to correct the vehicle's cross-track error, we first calculate the error distance between the trajectory reference point and the center of the vehicle's front axle, denoted e_{fa} in figure 5.1. Next, we add a proportional gain scaled by the vehicle's current velocity v [41][42]. An inverse $\tan$ function is then used to scale the control output to the range of $-\pi$ to π . This results in a correcting term equivalent to:

$$\tan^{-1}\left(\frac{ke_{fa}}{v}\right) \quad (5.6)$$

where k is a tunable gain parameter. When e_{fa} is zero, this term diminishes. Otherwise, it pushes towards $\pm\pi$.

4. In the event of receiving noisy near-zero velocity measurements, the term which accounts for the cross-track error becomes significantly amplified resulting in an aggressive output steering angle [41]. In order to overcome such behavior, a tunable softening constant k_s is added to the velocity term yielding:

$$\tan^{-1}\left(\frac{ke_{fa}}{k_s + v}\right) \quad (5.7)$$

5. The steering angle is then the summation of the terms correcting each of the heading and the cross-track errors [41], as shown by the following equation:

$$\delta(t) = \theta_e(t) + \tan^{-1}\left(\frac{ke_{fa}(t)}{k_s + v(t)}\right) \quad (5.8)$$

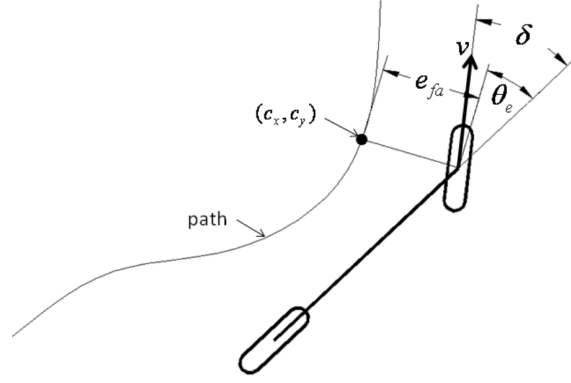


Figure 5.1: Geometry associated with the Stanley controller [42]

6. Lastly, we clip the output steering angle such that it is within the maximum and minimum allowable steering angles $[\delta_{min}, \delta_{max}]$. The final equation then becomes:

The gains k and k_s are then tuned in the simulated track in order to yield the desired controller performance.

5.1.4 Lane Keeping

In this section, we elaborate on the technical specifications and mathematics behind the perception and planning steps, highlighted earlier in 4.1.2.4.

In this perception step, our goal is to find a linear transform (i.e. matrix) that maps between the real-world coordinates (what the vehicle camera sees) and the planning coordinates (our map). Hence, a perspective transform is first calculated based on the image of a simple calibration object whose dimensions and simulation world coordinates¹ are known in advance. For our case, a simple rectangular white plane was used. The only constraint on the used calibration object is for it to have at least four distinctive points known in simulation-world coordinates (retrieved from simulation) and their corresponding map coordinates (generated by the user, keeping the original geometry). The constraint of having a minimum of four points² is present since each point provides us with two equations, and a total of 8-equations (given a scale factor) are needed to be solved for the perspective transform (also known as homography). The homography equation can be expressed as $P_{sim} * H = P_{map}$:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}_{3 \times 3} \times \begin{bmatrix} x_{sim} \\ y_{sim} \\ 1 \end{bmatrix}_{3 \times 1} = \begin{bmatrix} x_{map} \\ y_{map} \\ w \end{bmatrix}_{3 \times 1} \quad (5.9)$$

Where the sim subscript denotes a simulation coordinate, map subscript denotes a map coordinate, and w denotes a scaling factor between the two coordinates. After a linear transformation H from the simulation to the map coordinates is retrieved, an inverse transformation H^{-1} calculated from it and kept for later use.

¹an arbitrary 2-D system could be assigned, given one condition: knowing how to transform from this system to agent-centric view

²Another condition is that no 3 of the 4 points should be co-linear

Now that we have a map-view (bird's eye view) by applying the homography (shown in Fig 5.2), we can detect the lane's left and right edges, and generate the way-points our agent needs to follow. The detection is performed first by thresholding a grayscale version of the transformed image to keep only the two bright (white) lane edges. The lane edges are then detected by using a series of rectangular windows for each lane (shown in Fig 5.3 for the right lane) that are generated sequentially, where each rectangle has its center (and thus the the search-starting position of the next rectangle) positioned based on the weighted sum of the white pixels inside it. The generated rectangles' centers are then used as points on which a quadratic equation is fit, giving an equation for each of the two lanes' edges.

Now that we have an equation describing the lane edge, we can generate the way-points by inputting a y-axis (longitudinal) desired distance to each of the left and right equations, and averaging the x-axis output to have the output point in front of the agent, at a specific distance (y-axis), in the middle of the lane (averaged x-axis). This process three times per frame, generating three points for the agent on each frame (demonstrated in Fig 5.3).

Now that the points we have represent our desired way-points in the bird's eye view, we apply the inverse homography to retrieve them in the simulation view (demonstrated in Fig 5.4).

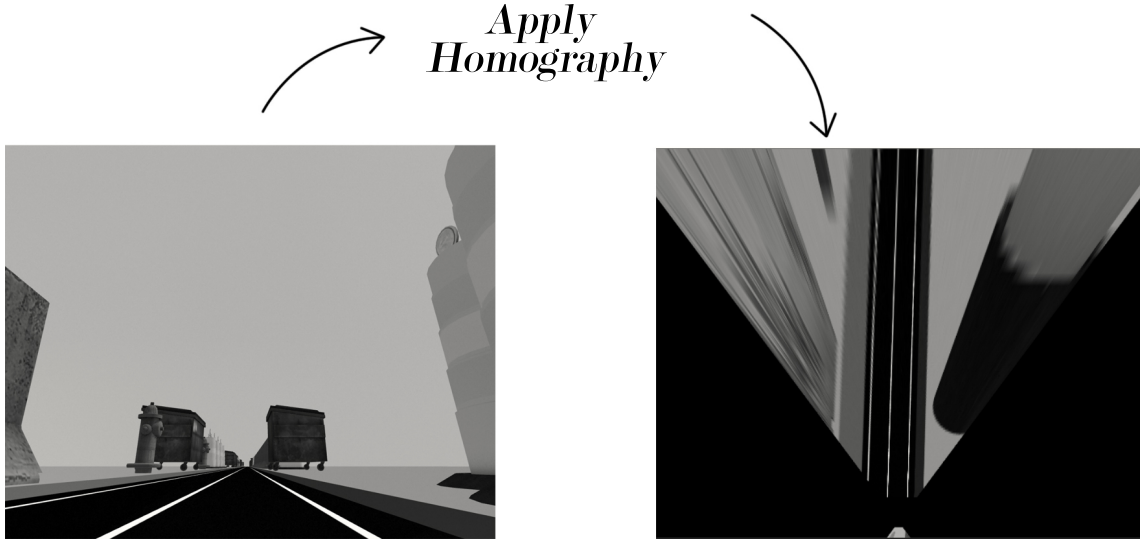


Figure 5.2: Applying homography on a raw grayscale image from the simulation, giving us our map (bird's eye view) view.

5.1.5 Lane Changing

In this sub-section, we provide the implementation details of our lane changing module previously discussed in 4.1.2.5. We make use of the V2V communication and assume that the surrounding vehicles to the ego one will take the worst case scenario actions over the coming time-steps. We assume that the ego vehicle is in a certain

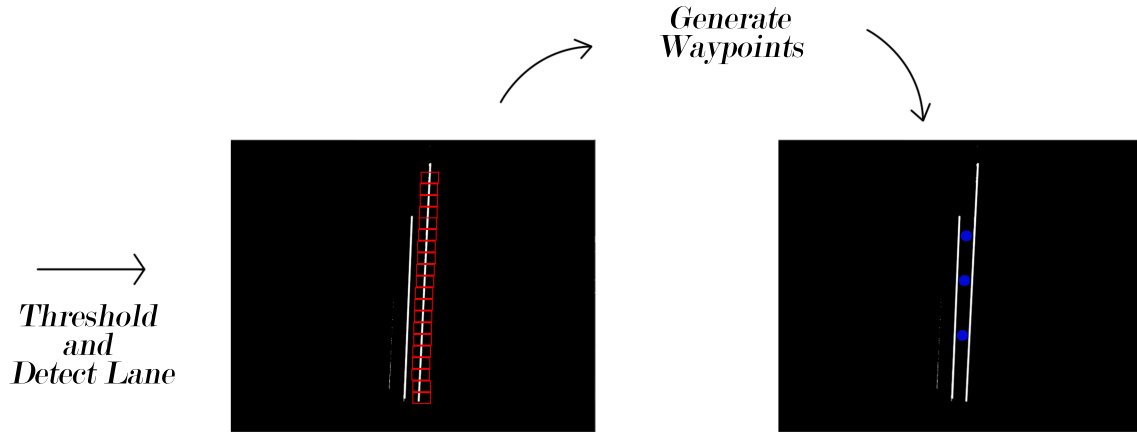


Figure 5.3: Detecting the lane on a thresholded bird's eye view image using sequential rectangles (shown in red), followed by generating way-points (blue dots) from the calculated quadratic equations of both the left and right lanes.



Figure 5.4: Applying inverse homography on the way-points generated in the bird's eye view to retrieve them in the simulation coordinate system.

lane and there are other four hypothetical vehicles in hypothetical positions A, B, C, and D as described in figure 5.5.

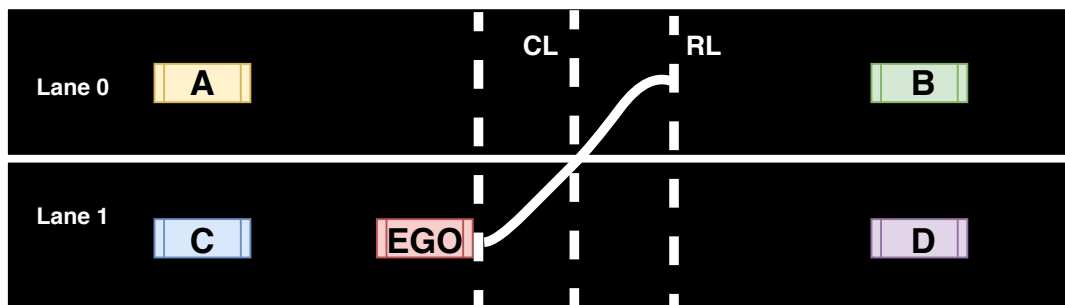


Figure 5.5: Lane Change schematic diagram.

Where:

<i>Lane0</i>	= to go lone; where the ego vehicle is going to change lane to it
<i>Lane1</i>	= current lane; where the ego vehicle exists
<i>Centerline</i>	= the line corresponding to half the distance taken to make the lane change action
<i>Rightline</i>	= the line corresponding to the point where lane change action will be finished
<i>A</i>	= vehicle position in the to go lane ; where it is located before the center-line
<i>B</i>	= vehicle position in the to go lane ; where it is located after the center-line
<i>C</i>	= vehicle position in the current lane ; where it is located before the center-line
<i>D</i>	= vehicle position in the current lane ; where it is located after the center-line

As mentioned before, the velocity ranges from 0 to 0.5 m/s with 0.1 m/s step. i.e:[0 0.1 0.2 0.3 0.4 0.5]. The lane change action is executed in a certain time and distance for each velocity, these results were calculated empirically in Table 5.1.

Velocity (m/s)	Total time taken to finish lane change action (sec)	Total distance taken to finish lane change action (m)	Half the distance taken to finish lane change action (m)	Time taken to reach the center-line , reach half the total distance (sec)
0.1	15	1.367	0.681	7
0.2	9	1.631	0.819	4
0.3	6	1.754	0.984	3
0.4	4	1.855	1.108	2
0.5	4	1.976	1.23	2

Table 5.1: Lane change time and distance at certain velocities.

Assumptions:

1. Lane change occurs at the instant velocity of the ego vehicle.
2. Lane change action is executed in N time-steps where the time-step is lane change execution action time divided by the delta time.
3. The descriptive parameter is not the whole time-steps, it is half the time-steps; number of time-steps taken by ego vehicle to reach center-line. For simplicity we will conveniently say that half the time-steps is “time-steps” as we don’t use the whole time-steps as a parameter in any calculations.

Algorithm Core:

The algorithm starts with finding the existing vehicles in the environment, vehicles may be A, B, C or D. We are not concerned with vehicle C as it will not affect

the lane change action as we will explain later on.

To know whether there is a vehicle D or not, we simply make use of the V2V communication and check whether there is a vehicle in the same lane and ahead of the ego vehicle. We keep looping over all the vehicles that are in the same lane and ahead of the ego vehicle till we find the nearest one to the ego vehicle and this is considered our vehicle D.

The case for vehicles A and B is different because if any of the two vehicles accelerate or decelerate too much this may change it from position A to B or vice versa.

To make sure that a vehicle is B we assume that the vehicle applies maximum deceleration during all time-steps the ego vehicle takes to reach center-line, if its position is ahead of the centre-line then it is B. Otherwise, we assume the vehicle applies maximum acceleration during all time-steps the ego vehicle takes to reach center-line, if its position is behind the centre-line then its A. Also, there is a case where we can't decide if the vehicle is for sure A or B so we return that lane change is infeasible. This part of the algorithm being handled in the code is `isSafeWRTD()` and `isAorB()` functions.

After getting all the information related to vehicles in the environment surrounding the ego vehicle, we check for the feasibility of executing the lane change action and whether it is safe or not. This is done by assuming that every vehicle will make the worst case scenario affecting the lane change action and then determine the feasibility of executing the lane change action.

In case of vehicle C, it will not affect the ego vehicle by any means as it will follow it by Krauss model. It will adapt its velocity with the ego vehicle and keep a minimum safe distance accordingly.

In case of vehicle D, the possible actions that vehicle D can do is to accelerate, decelerate or stop. In case of acceleration there will be no problem because it will be out of the ego's vehicle window and will not affect its lane change action.

In case of deceleration, we assume the worst case vehicle D will make. The worst case here is to perform maximum deceleration during the time-steps, number of time-steps taken by ego vehicle to reach center-line. If the hypothetical position of vehicle D becomes before the center-line, therefore lane change action is not feasible. In case of the stop action, it is implicitly handled in the deceleration action case because if the vehicle decelerates several times till it stops this will end up as if we are handling the stop action from the very beginning. So if the vehicle stops and is found to be before the center-line this will make the lane change also infeasible. This part of the algorithm being handled in the code is `isSafeWRTD()` function.

In case of vehicle B, vehicle B is in the target lane for the ego vehicle so there are two cases to handle considering the hypothetical velocity and position, these cases are calculations before reaching the center-line and calculations after reaching the center-line. In the first case, before reaching the center-line, we assume that vehicle B will perform maximum deceleration during the time-steps and make sure

it is still considered to be at hypothetical position B. In the second case, we calculate the number of time-steps taken by the ego vehicle to reach the right-line (starting from center-line) and assume max deceleration during these time-steps. If the final hypothetical position of vehicle B is before the right-line then lane change action is infeasible. If it is after the right-line then we check the feasibility using Krauss model by calculating the maximum possible safe velocity between the ego vehicle and vehicle B and check if the velocity of the ego vehicle is greater than the maximum safe velocity then lane change action is not feasible, otherwise it is feasible. This part of the algorithm being handled in the code is in `isSafeWRTB()` function.

In case of vehicle A, since vehicle A is also in the target lane for the ego vehicle then it has two cases to handle; before and after reaching the center-line. In the first case, before reaching the center-line, we assume that vehicle A will perform maximum acceleration during the time-steps and make sure it is still considered to be at hypothetical position A. In the second case, after the ego vehicle reaches the center-line, we check whether the relative velocity between ego vehicle and vehicle A is safe (using Krauss model) by calculating the maximum possible safe velocity between the ego vehicle and vehicle A and check if the velocity of vehicle A is greater than the maximum possible safe velocity then the lane change is not feasible, otherwise it is feasible. This part of the algorithm being handled in the code is in `isSafeWRTA()` function.

5.1.6 Path Planning

In this sub-section, we provide the implementation details of our AV's path planning module introduced in 4.1.2.6. Our work was based on ROS navigation stack [83]. However, we need to tune its used parameters such as max acceleration and max velocity in both angular and linear directions. Moreover, we use the default parameters for the TEB local planner [84] except for the goal tolerance parameter. We increase the goal tolerance parameter; to make the AV start to decrease its velocity before reaching the goal with safe distance, to ensure it stops exactly at the goal position.

As for our two adjustments to the module previously highlighted in 1 and 2, the following provide a summary of the details of their implementation.

1. Adding a Communication Costmap Layer

Before getting into the details of the functionalities that our newly added costmap layer has to be able to perform, it helps to understand the mechanism by which each of the individual layers in the ordered list of costmap layers contributes to the final generated master costmap which is eventually used in the path planning.

Each of the costmap layers, except the inflation layer, holds its own copy of grid cells which are each marked as either LETHAL, if the cell is known to have an obstacle, NON LETHAL, if the cell is known to be free, or NO INFORMATION, if the state of the cell is unknown to this layer. For a single

update of the master costmap, the two following passes through the ordered list of layers are taken [46]:

- (a) **Update Bounds Pass:** The aim of this first pass is to determine how much of the master costmap layer has to be updated in this round. Consequently, the pass starts with an empty bounding box and iterates, in order, over the costmap layers [46]. Each layer expands this box as appropriate, resulting in a final bounding box representing the whole area of the master layer that has to be updated.
- (b) **Update Costs Pass:** The aim of this second pass is to perform the actual update of the costs of all the grid cells of the master layer that are within the bounding box specified in the first pass [46]. Again, the layers are iterated over in order, and each of them uses the costs of its copy of the grid cells to update the cost of all master grid cells within the bounding box [46]. There are different ways in which this update step could be executed. As one example, layers which come later in the ordered list can be allowed to overwrite the costs specified by the previous layers or not. Accordingly, the order of the layers in the list and the update method adopted affect the final generated master layer and how it can be interpreted.

Based on the explained update mechanism, it is important that we choose the order of our communication layer in the list of costmap layers, as well as the update method to be used in the update costs step. The mostly used order of the other three layers is: 1. Static Layer 2. Obstacle Layer 3. Inflation Layer [46]. Since we are confident about the communication data encoded in our communication layer, we might want it to overwrite cost values specified by either the static layer or the obstacle layer. However, we want the inflation layer to add a buffer zone around the cells marked as LETHAL by our new layer as well. Hence, we choose that the order of the costmap layers is as follows: 1. Static Layer 2. Obstacle Layer 3. Communication Layer 4. Inflation Layer. As for the method of updating costs adopted by our communication layer, we choose that it overwrites the cost of the master cells with ONLY the valid costs in its copy of grid cells; where only the LETHAL and NON LETHAL labels are considered valid. This means that the cost of master cells are not overwritten by any cells marked as NO INFORMATION in our layer's copy of the grid cells.

Therefrom, there are two main functionalities that our communication layer should be capable of performing apart from the actual extension of the bounding box (during the update bounds pass) and the actual update of costs of the master layer grid cells (during the update costs pass). These functionalities are:

- (a) Setting the costs of its own copy of grid cells based on the communicated positions of other vehicles within its communicated range.
- (b) Determining which region of its copy of grid cells has changed since the last update pass of the master layer. This is necessary so that it can,

during the update bounds pass, increase the size of the bounding box accordingly.

In order to perform these functionalities, the communication layer needs the following inputs and parameters:

- **Inputs:**

- (a) **List of Vehicles' Footprints:** This is a list of all the footprints of vehicles that are currently within the vehicle's communication range, where the footprint is the corner coordinates of the rectangular region occupied by the vehicle in the map. Since the vehicle's communication module already allows it to keep a list of all the data of vehicles in its communication range, as thoroughly discussed in 5.2.3, this list of footprints is already available to the vehicle.
- (b) **Last Marked Region:** This is in the form of coordinates of a rectangular region resembling the area within which exist all the cells in the layer's copy of grid cells which were marked as LETHAL in the last update pass. The communication layer already keeps track of these coordinates during each pass; therefore they are always known to it.
- (c) **Vehicle Position:** The communication layer could be chosen to be part of a global layered costmap, used by the global planner in generating the global path, or to part of a local layered costmap, used by the local planner in generating the local path. While the global layered costmap and its origin are static, the local layered costmap is **rolling**. This means that the origin of the local layered costmap and all its layers should always be at the center of the vehicle. Therefrom, in case the communication layer is added to a local layered costmap structure, the current vehicle position needs to be known to the layer so that its origin gets updated accordingly. This is obtained from the vehicle's LiDAR sensor.

- **Parameters:**

- (a) **Is Rolling:** This is a boolean parameter indicating whether the origin of the layered costmap should be at the vehicle's center at all times. This is the case with the local layered costmap but not the global one.
- (b) **Safety Clearing:** When the layer clears the cost of cells previously marked as LETHAL, it could be prompted to clear some extra meters around these cells to enhance robustness. This parameter specifies those extra meters.

When first created, the communication layer is initialized with a copy of the grid cells of the master layer. The costs of all the cells are initially NO INFORMATION. Next, using the above inputs and specified parameters, the communication layer performs the following steps during every update pass of the master layer:

- (a) If the “is rolling” parameter is true, this means the communication layer is part of a layered costmap that is rolling; whose origin has to be the vehicle’s center at all times. In this case, the origin of the communication layer is updated to coincide with the current coordinates of the vehicle. This is necessary so that any coordinates specified in the map frame could be directly mapped to coordinates in the costmap layer.
- (b) Given the corner coordinates of the last marked region, the communication layer clears this region in its copy of the grid cells by setting these cells’ cost to NO INFORMATION. This is to clear any costs from the previous update pass. For robustness purposes, the rectangular region to be cleared is inflated by the amount specified by the safety clearing parameter.
- (c) Upon receiving the updated list of vehicle footprints, the layer marks all those cells as LETHAL in its copy of the grid cells.
- (d) The layer then stores the coordinates of the rectangular region within which all those cells marked as LETHAL exist.
- (e) Now that the marking and clearing have been completed, and upon receiving the coordinates of the bounding box during the update bounds pass, the communication layer increases the box’s bounds as appropriate. It includes the marked region whose coordinates are stored in 1d, and the cleared region in 1b, whose coordinates are those of the marked region from the previous path plus the safety clearing buffer.
- (f) During the update costs pass, the communication layer simply overwrites the costs of the master grid cells within the final bounding box with those stored in its copy.

For further clarification, steps 1b - 1e are summarized in figure 5.6. Lightly shaded cells are cells marked as LETHAL in the previous update pass, and resemble vehicles’ footprints at that time. Darkly shaded cells are cells marked as LETHAL in the current update pass, and resemble the current vehicles’ footprints. The safety clearing parameter is set to 0 meters in this example for simplification.

2. Adding a Custom Planner for Lanes:

- **Inputs:**

- (a) The AV’s current position
- (b) The AV’s local planner path (An array of poses)
- (c) If the AV has already started lane changing
- (d) If the AV has finished lane changing

- **Outputs:** The control actions:

- (a) Lane Keeping
- (b) Left Lane Changing
- (c) Right Lane Changing

- **Parameters:** Refer to 4.1.1

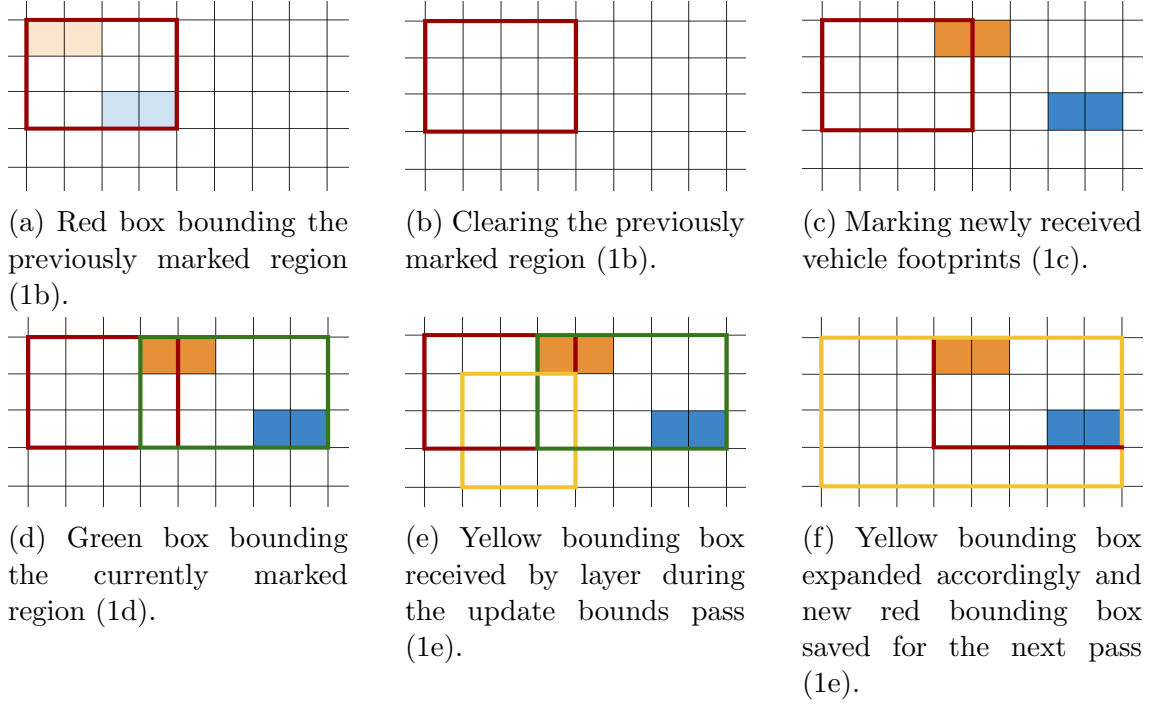


Figure 5.6: Communication Layer Update Algorithm

- (a) The y-coordinates of the separating lines between lanes
- (b) The x-coordinates of the lanes indicating their lengths
- (c) Lanes numbers

• **Algorithm:**

Algorithm 2: Adding a Custom Planner for Lanes

Determine which lane the AV is positioned at: by getting the AV's position through localization module then getting its location relative to lanes using their coordinates.;

Determine which lane the AV is heading to: by getting the last pose in the local planner trajectory and getting its location relative to lanes as well.;

Communicating with the navigation module to know if the AV has started lane changing if it was ordered to do so.;

if *It has already started lane changing* **then**

do not send other lane changing goals in order for the AV to change its lane only once then wait for the next action. Send lane keeping goals instead for the AV to separate two lane changes with lane keeping for one time stamp;

end

if *It has not already started lane changing* **then**

if *The heading lane is greater than the current lane* **then**

Output left lane changing.;

else if *The heading lane is smaller than the current lane* **then**

Output right lane changing.;

else if *Both lanes are the same* **then**

Output Lane Keeping.;

5.1.7 Navigation

In this sub-section, we go through our designed system for the navigation module previously discussed in 4.1.2.7.

The default action in the navigation module is lane keeping, the AV has to keep moving until it reaches the goal. It only stops if it is stuck.

- **Inputs:**
 1. The chosen action in any time stamp by the path planning module (4.1.2.6).
 2. A Boolean indicating if the AV has already reached the goal.
- **Output:** The AV performs one action in any given time stamp.
- **System Components:** A master that handles three clients:
 1. Lane Keeping Server Client
 2. Velocity Control Server Client
 3. Lane Changing Server Client
- **Server-Client Communication Type**
 1. **Goal:** It contains information about:
 - (a) **Control Source:** The control source, whether it came from navigation module or an external command. (4.1.2.8).
 - (b) **Control Action:** Lane Keeping or Lane Change.
 - (c) **Direction:** Changing lane left or right in case of lane changing control action.
 - (d) **Acceleration:** The acceleration in case of lane keeping in case of lane keeping control action.
 2. **Feedback:** If the control action is feasible or not.
 3. **Result:** If the control action is completed or not due to infeasibility.
- **System Algorithms:**
 - **Servers:** After receiving goals:

Algorithm 3: Lane Keeping Server Algorithm

```

while While preemption from the client is not requested do
  Listen to the velocity requested from velocity controlling server in a
  stream of messages that changes every time stamp;
  Start lane keeping using 5.1.4, and 5.1.2 modules;
end
Stop lane keeping;
Send the result to the navigation master;
  
```

Algorithm 4: Velocity Controlling Server Algorithm

while *While preemption from the client is not requested* **do**
 Get the requested velocity from the local planner in 4.1.2.6;
 Execute 5.1.2 module to check its feasibility and adjust it to be feasible
 if it wasn't; Send the velocity to lane keeping server;
end
Stop generating velocities;
Send the result to the navigation master;

Algorithm 5: Lane Changing Server Algorithm

Check whether the lane change in the requested direction is feasible;
if *The lane change is feasible* **then**
 Execute lane change using 5.1.5 module;
 Send result to the navigation master indicating that the lane changing is
 finished;
else
 Send feedback and result to the navigation master indicating lane
 changing was not feasible;
end

- **Navigation Master:** The high-level control master module (5.1.9) listens to the path planning module that generates a control action every time stamp, and pass the chosen control action to the navigation module master.

Algorithm 6: Navigation Master Algorithm

For every time stamp, a thread is generated by the high-level control master in section (5.1.9) [6] and executes this algorithm;

```

if The navigation master is deactivated  $\vee$ 
    The AV has already reached the goal  $\vee$ 
    The AV is currently executing lane change  $\vee$ 
    The control action hasn't changed then
    | Terminate this thread;
else
    | Check the control action;
    | if The control action is lane keeping then
    | | Send goals to lane keeping and velocity controlling servers;
    | | Terminate this thread;
    | else if The control action is lane changing then
    | | Send goal to lane changing server;
    | | Block the thread till the lane changing server sends feedback;
    | | if The feedback indicated that lane changing is infeasible then
    | | | if The AV hasn't been lane keeping already then
    | | | | Send goals to lane keeping and velocity controlling servers for
    | | | | the AV to start lane keeping;
    | | | | Terminate the thread;
    | | else
    | | | Cancel all goals sent to lane keeping and velocity controlling
    | | | servers;
    | | | Send to the path planning module;
    | | | (5.1.6) that lane change is starting;
    | | | Terminate this thread;
    | end
end

```

5.1.8 External Commands Following

In this sub-section, we explain the details of our designed system for the external commands following module previously discussed in 4.1.2.8.

When the external commands following module is activated, there is no default actions unlike the navigation module 5.1.7. An external system decides the action in any time stamp. The AV stops and wait until a suitable action is chosen.

- **Inputs:**

1. Control action (Left lane change, Right lane change, Accelerate, Don't change velocity, Decelerate).
2. Acceleration value.
3. A Boolean to indicate if the external commands following module should end.
4. Time needed to execute the control action if it was lane keeping.

- **Outputs:**
 1. A trial for command execution.
 2. Its feasibility.
 3. Time Taken if the command was executed.
- **System Components:** The same servers in the navigation module (5.1.7) but with three different clients.
- **Server-Client Communication Type** The same communication type mentioned in the navigation module (5.1.7).
- **System Algorithms:**
 - **Servers:**
 - * **Lane Keeping Server:** Same algorithm presented in the navigation module (5.1.7)

Algorithm 7: Velocity Controlling Server Algorithm

```

Get the acceleration from the external commands following master;
while preemption from the client is not requested do
    Check the feasibility for the new velocity given the acceleration value
    using 5.1.2 module;
    if it was feasible then
        | Send the velocity to the lane keeping server for this time stamp;
    else
        | if it was the first time stamp then
            | Send the result to the external commands following master
            | indicating the action was not feasible at the first time stamp;
        | else
            | Break;
        | end
    end
end
Stop generating velocities;
Send the result to the external commands following master indicating the
action was successful for at least one time stamp;

```

- * **Lane Changing Server:** Same algorithm presented in the navigation module (5.1.7)
- **External Commands Following Master:**

The high-level control initializes a thread in the external commands following master with a given control action. Therefore, the external commands following master executes the following algorithm.

Algorithm 8: External Commands Following Algorithm

```
if the action is lane keeping then
    Send a goal to the lane keeping server;
    Send a goal to the velocity control server and block the thread for the
        required time;
    Get the result from the velocity controlling server;
    Calculate the execution time between sending the goal and getting the
        result;
    Cancel the lane keeping goal;
    Send the result and the execution time to the high-level control master;
    Terminate;
else if the action is lane changing then
    Send a goal to the lane changing server;
    Block the thread till the feasibility is executed;
    if lane changing was infeasible then
        Send the result with zero execution time to the high level control
            master;
    else
        Block the thread till the lane change get executed;
        Send the result with the execution time to the high level control
            master;
    end
```

5.1.9 High-Level Control Master

This sub-section discusses our implementation of our AV's high-level control master previously highlighted in 4.1.2.9. The following is a summary of the module's inputs, outputs and associated algorithm.

- **Inputs:**

1. **First Communication Channel:** A Stream of messages sent by the path planning module (5.1.6).
2. **Second Communication Channel:**
 - A request sent by a server client connection with the external system that, at some point, will need to take control over the AV. The external system is 5.3.
 - A Boolean to indicate whether external commands following module is activated or deactivated.

- **Outputs:** The AV should be controlled by only one module (navigation module or external commands following module) in any given time stamp.

- **Algorithm:**

Algorithm 9: High-Level Control Master Algorithm

```
while the system is on do
  Block the main thread until a message is received in any communication
  channel;
  if a message is received in the first channel then
    Generate a new thread and run the following;
    Let the navigation module take control (5.1.7);
    Send the control action to the navigation module 5.1.7;
    Wait till the navigation module terminate the thread;
  else if a message is received in the second channel then
    Generate a new thread and run the following;
    if the external system asks to deactivate the external commands
    following module then
      Activate the navigation module again;
    else if the external system asks to execute a control action then
      Deactivate the navigation module so that its concurrent threads
      will be terminated, refer to 5.1.7;
      Hand the control action to the external commands following
      module to execute it;
      Wait for the external commands following module to send the
      control action's feasibility and time taken to be executed if it
      was feasible, refer to 5.1.8;
      Send the result to the external system client, such as 5.3;
      Wait for the external commands following module to terminate the
      thread;
  end
```

5.2 V2V Communication

In order to simulate our simple V2V communication system previously discussed in 4.1.3, each vehicle should be capable of both broadcasting its own identification data (ID) and receiving the ID of other vehicles within its communication range.

5.2.1 ID Message Structure

The chosen contents of the communicated ID messages are as follows:

- **Vehicle Type:** “racecar”/“EV”, according to our problem
- **Vehicle Position:** the x and y coordinates of the vehicle center in the map coordinates
- **Vehicle Lane:** an integer indicating the number of the lane in which the vehicle is, according to a preset convention
- **Vehicle Footprint:** the corner coordinates of the rectangular region occupied by the vehicle in the map
- **Vehicle Velocity:** the current velocity of the vehicle

- **Vehicle Maximum Velocity:** the maximum possible velocity of the vehicle
- **Vehicle Maximum Acceleration:** the maximum possible acceleration of the vehicle

5.2.2 Broadcasting ID Messages

In order for each vehicle to broadcast its ID message, it needs the following inputs and predefined parameters:

- **Inputs:**
 1. **Vehicle Position:** This is obtained from the vehicle's LiDAR sensor.
 2. **Vehicle Velocity:** This is obtained from the vehicle's electronic speed controller (ESC).
- **Parameters:**
 1. **Vehicle Maximum Velocity**
 2. **Vehicle Maximum Acceleration**
 3. **Vehicle Type**
 4. **Vehicle Dimensions:** The vehicle dimensions are needed so that the vehicle is able to calculate its footprint.
 5. **Lane Numbering Convention:** The convention of the lane numbering in the current map has to be known to the vehicle so that it can map its position coordinates to a lane number.
 6. **Vehicle Number:** This is a unique number assigned to each vehicle so that its broadcasted ID could be associated with it.

Using these inputs and specified parameters, the vehicle broadcasts its ID through the following steps:

1. Using the vehicle's coordinates in the map frame as well as its known dimensions, its footprint in the map is calculated.
2. Knowing the lane numbering convention and the vehicle's coordinates in the map, the vehicle's lane number is calculated.
3. All the information needed to construct the vehicle's ID message are then ordered in a structured format and stamped by both the vehicle's number and the current time at which it is broadcasting its ID.
4. The vehicle then sends its ID message over a public communication channel accessible to the other vehicles. This communication channel, or topic, is one of the main communication styles supported by ROS. It is a unidirectional bus over which messages could be exchanged in an asynchronous manner [6].

5.2.3 Receiving ID Messages

Since the topic over which vehicles send their ID messages is public, each vehicle can have access to the ID of all other vehicles in the environment. However, this does not match the realistic assumption that vehicles can receive messages from others only within their communication range. Consequently, to simulate the realistic scenario, vehicles discard messages sent by other vehicles beyond their communication range. Henceforth, the needed inputs and predefined parameters are:

- **Inputs:**

1. **Vehicle Position**
2. **New Communicated ID Message:** This is an ID message read by the vehicle from the public topic. The vehicle reads the ID messages one at a time, and it receives them ordered by the time at which they were broadcasted.
3. **Last List of Other Vehicles' IDs:** The vehicle keeps a list of IDs received by all other vehicles within its range. The last version of this list is needed as an input so that it can update it upon receiving a new ID message.

- **Parameters:**

1. **Vehicle Number**
2. **Vehicle's Communication Range:** This is the radius of the circular zone resembling the vehicle's communication range. Broadcasted IDs of any vehicles beyond this zone cannot be known to the vehicle.

Using these inputs and specified parameters, the vehicle is able to update the list of IDs associated with all vehicles in its communication range. This is done through the following steps:

1. The vehicle reads an ID message from the public topic.
2. Since the vehicle itself has also sent its ID over the same topic, it has to first check that the ID message it just read does not belong to it. If it does, then the vehicle should ignore this message. This is done by comparing the vehicle number of the sender with the vehicle's own number.
3. With the knowledge of the vehicle's coordinates in the map and the coordinates incorporated in the received ID message, the vehicle calculates the distance between the sender vehicle and itself. If this distance exceeds the radius of the vehicle's communicated range, this means that the sender vehicle is currently outside the accessible range. Hence, if the last ID of this sender vehicle was previously stored in the list of IDs, it should be deleted. If it was not, then this newly received message is discarded.
4. If the sender vehicle is found to be within the vehicle's communication range, then this newly received ID is added to the list of IDs. If the sender vehicle's last ID was already stored in the list of IDs, then its ID should be updated instead.

This guarantees that a simple V2V communication system is simulated such that, at all times, each vehicle has an updated list of the IDs of only the vehicles that are within its specified communication range.

5.3 Cooperation Technique

In this section, we provide the implementation details of our adopted cooperation technique previously described in 4.1.4.

5.3.1 Single Agent Problem Formulation

In a typical reinforcement learning problem, the agent learns how to perform a certain task by interacting with the environment. Concept-wise, it observes the environment. Then it takes an action accordingly. As a result, it receives a reward from the environment. Code-wise, the agent observes the system state which is composed of agent state and the environment state. Then, it chooses an action from a pool of feasible actions according to action choice paradigm. The environment executes that action i.e. it applies changes that result from that action. The environment then returns: an observation which is the system's new state, a reward of a value determined according to whether the agent's picked action yields favorable results or not, a done flag to indicate whether the episode has been terminated or not, and other information that is not used in training but important such as goal state which is the state at which the episode terminates. If done is false, the episode has not been terminated, hence the loop will continue. In order to formulate any problem as an RL problem, we need to define an environment, system states, agent actions, and reward function.

5.3.1.1 Environment

Environment is an urban road which is divided into lanes. Each lane is discretized into cells. Two entities co-exist inside the environment: the Emergency Vehicle (EV), and the agent which is an autonomous vehicle. The EV is not a trainable agent. It drives using a car-following-model all the time and does not use our RL model to drive. Refer to Section 4.1.1.2 for more information about the used car-following-model. The justification is that the EV has the highest priority and should not change its predetermined plan for any other vehicle. According to its car-following-model, it picks a value of five possible acceleration values at every time step. Possible acceleration values are integers in the range of $[-2, 2]$. This range could be altered. However, we are just concerned about making it limited to 5 possible values for the sake of our Q-Table. Acceleration values affect velocity range which in return affects system states size. The overgrown states size makes the training process significantly more demanding and sometimes intractable, especially using non-deep learning RL models. The agent is a trainable/learning vehicle and was trained using the Q-learning algorithm to evacuate the road for the EV to help it reach its destination within optimal time.

5.3.1.2 States

The observed state is the one used for training: the one that is used to index the Q-Table. It is composed of EV velocity, EV lane, agent velocity, agent lane, and the EV position relative to agent if the EV is within agent's window. If it is outside the agent's window, then y position will be denoted by a cut-off value which represents whether it is in front of or behind EV. Agent window represents the perception and proximity area around the agent where it can detect the existence of the EV and receive rewards for its actions.

The following value ranges are scenario-specific and could be altered with wider or narrower ranges. Our goal was to make training affordable by limiting states size. EV velocity is an integer in the range of $[0, 10]$. So the maximum velocity for the EV is 10 cells per time step. Agent velocity is an integer in the range of $[0, 5]$. EV and agent lanes could be any lane of the three lanes, indexed as 0, 1, 2.

The hidden state is only known to the environment itself. It contains all information in observed state besides goal global coordinates, and EV global coordinates. It is used to check for episode termination. If the EV position is the same as the goal position, the episode will terminate. The episode terminates if any case of the following three cases happens: First, if EV has reached its goal position. Second, if the agent has finished its trip. Third, if maximum number of time-steps has reached to stop infinite episode occurrence.

5.3.1.3 Actions

Actions encompass agent's possible driving actions. Agent's possible actions are : acceleration, deceleration, change lane to the left, change lane to the right, and continue as it is (no speed nor lane change is involved). We check feasibility of all actions given a certain state. This is possible as number of possible states and actions is known, limited in size and more importantly feasibility check in SUMO is not a time consuming process. We could visualize this as state action space, that is reduced to feasible state action space which is determined at the beginning before model get trained.

5.3.1.4 Reward

Designing a reward function was the most challenging part of the problem formulation. We were willing to design reward function that rewards results rather than behaviors which is a common mistake as illustrated in [80]. These results should be always desirable. A well formulated reward function should reward/penalize activity, passivity and do nothing towards side effects. Activity means that agent actively takes an action to give way to EV. Passivity is when the agent does not take any action that blocks EV way. Side effects mean actions done by the agent that do not affect EV in a way or another. There are two types of reward : step reward and final reward.

- Step reward is the reward that agent takes every time-step. We have designed it to be a linear function of EV acceleration at every time-step as increasing speed of EV is always a desirable result. Then we have edited it to be a

weighted linear function of EV acceleration and agent's actions. The main reason for this edit was to make sure that our agent drives safely and stops its oscillating behavior. Refer to Section 6.3.1 for more information about step reward phases and associated results with every phase until we have managed to make agent behavior an optimal one.

- Final reward is the reward given to the agent when the episode terminates. We have designed it to be a linear function of the ratio between actual EV travel time taken until episode has terminated and optimal travel time. Optimal time is the time taken by EV if it has always managed to drive with its max speed during the episode. At first, we believed it will be useful to introduce final reward to our problem formulation design. However, we have discovered later on that it is not suitable for our problem design. The reason was the lack of a method to divide this reward over pairs of states and actions along the episode fairly in a way that represents their contribution to the success of failure of the agent. Especially if some of agent actions over certain episode were wise enough to efficiently evacuate road for the EV while other actions are not. There was no way to give every action its corresponding portion of final reward. For this concern, we have disabled the final reward and made reward design only limited to step reward.

5.3.1.5 General Problem Formulation Design Considerations

Besides designing states, actions, and reward, there are other vital design considerations:

- **Interaction Guarantee**
- **Agent window**
- **Dynamics Handling**

Agent training process is totally dependent on its interaction with environment and corresponding rewards it receives during this interaction. In case of training single agent, interaction is represented only in agent interaction with the EV. We have generated random initialisation of both agent and EV lanes, positions, and relative distances. However, this random initialization ensures that there is sufficient interaction time and distance before EV reaches its goal. So how is this interaction guaranteed?

- Vehicles' initial positions in their lanes are ordered according to their maximum velocity. For instance, if the maximum velocity of agent is less than EV maximum velocity, then agent will be inserted in the lane ahead of EV to ensure interaction.
- Given that the maximum velocity of every vehicle is known and goal position is determined, we calculate upper bound on time at which EV and agent must meet before EV reaches its goal using the following inequality :

$$maximum\ velocity_{faster\ vehicle} * time_{meeting} < position_{goal} \quad (5.10)$$

- According to the upper bound of their meeting time, we calculate the upper bound of meeting position using the following inequality :

$$position_{meeting} + maximum\ velocity_{slowervehicle} * time_{meeting} < position_{goal} \quad (5.11)$$

Agent window represents the perception and proximity area around the agent where it can detect the existence of the EV and receive rewards based on how its actions have affected EV acceleration. We have utilized the concept of agent window for three reasons:

1. To keep state space fixed even if road or lanes are infinite as part of observed state is the relative position between agent and EV with the agent being the origin. If EV is inside agent's window, its position will be represented as a relative position to agent. If it is outside window, it will take one of two states which represents whether EV is in front or behind agent. In other words, if the EV is outside the agent's window and behind it by hundred cells, it is exactly the same as being behind it by a thousand cells.
2. To reward agent for activity, passivity, and do nothing towards side effects. If EV is inside the agent window, the agent will receive reward based on how its active and passive actions affect the EV reward. If EV is outside agent window, it means that agent and EV are too far to affect each other. Hence, the reward will update the value function of this metaphorical state-action pair that represents any EV outside window.
3. Defining a window around agent where the agent is the origin makes agent perspective a centric one. Agent-centric perspective is better in learning policies that generalize better [85].

We have simplified dynamics by assuming that the road is divided into lanes where every lane is discretized into cells so there is no meaning for lateral movement within the lane itself, but rather there is only longitudinal movement within the lane. This is aided by the assumption that lane keeping and lane change functionalities are to be handled by the AV capabilities. Consequently, we have assumed that once agent has decided to change lane, it will be in that lane in the next time-step. Our focus at this stage was to train the agent on how to cooperate with the EV not how to drive. This was our plan to simplify dynamics. It was impressive to figure out that SUMO holds the same principles about handling dynamics, refer to Section 4.1.1.2 for information about how Sumo handles dynamics. Also, we have considered dynamics along with cooperation in our ROS implementation thoroughly.

5.3.2 Moving the Single Agent to Multi-Agent Environment

Placing the Single-Agent model in a Multi-Agent environment was done to test it. We wanted to evaluate our model's performance when placed with other vehicles. Several clarifications are in order here:

1. Several minimal code changes were made. Mainly, the episode did not terminate upon the agent exiting the road; as there were many agents now and our

goal was to observe the EV behaviour not train the agent. Also, instead of the RL Algorithm being a global variable to the system, each vehicle now had its own control Algorithm (one of those mentioned in 4.1.1.2).

2. The agent vehicle is still not able to integrate any position/velocity related parameters with its RL decision making process. It still consults the Q-Table using the earlier clarified observed state of itself and the EV only. The agent, however, has knowledge about which actions are feasible and which are not, based on the states of the other agents.
3. Control over the number of vehicles on the road and the percentage of AVs using each model was introduced, allowing to conduct controlled experiments and observe the effect of different parameters on the travel time. More details on this in Section 6.3.

5.4 System Integration

After developing the autonomous functionalities of our vehicles, we aim at deploying the single-agent RL model we designed in SUMO (5.3.1) on one of our agent vehicles in the presence of an EV in Gazebo. In general, we adopt the same conventions relating to the agent-environment interactions. However, owing to the much more realistic nature of the Gazebo simulation environment, several changes had to be introduced. These changes deal with real-time constraints, dynamics of the AV, and system latency. These changes are summarized in the following points:

1. In the original model, the action to be taken at each time step is chosen, either using the exploit or the explore policy, from a set of **only the current feasible actions**. This was possible then because SUMO handled the feasibility calculations in a reasonable amount of time. On the contrary, with the complexity in dynamics and the system as a whole in Gazebo, calculating the feasibility of all the different actions at every time step is an extremely time-consuming process that would introduce an undesirable lag to the simulation and adds latency. This is besides the fact that it is not realistic to perform the checking of all actions' feasibility in such a regular manner. As a result, we resorted to choosing the action to be taken from the set of "all" actions and checking this chosen action's feasibility. If it is found to be feasible, then it gets executed and if it is not, another one is chosen. This is repeated till a feasible action is found and taken.
2. Since our system in Gazebo is meant to mimic a real one, the agent vehicle's window should be within the limits of its communication range at all times. This is essential in order for the vehicle to be able to receive the data broadcasted by the EV so that it can use it in its model.
3. The velocity ranges assigned to the agent and the EV were changed to $[0, 0.5]$ and $[0, 1]$ respectively. This is because our vehicle's controllers were tuned to those ranges.
4. One of the major assumptions in the implementation on SUMO is that each of the different actions of the agent get executed in a single time-step. This

means that the vehicle takes one time-step to complete its action regardless of whether it is a simple “accelerate” action or a complex “lane change” one. This is certainly unrealistic; a lane change action, for instance, will definitely require more time to get executed than other actions. Moreover, the time taken to change the lane depends on the vehicle’s velocity during this maneuver.

As a result of such difference, actions taking longer to get executed seem to buy the emergency vehicle extra time in which it could accelerate, if possible. Accordingly, since the agent’s reward mainly depends on the change in the emergency vehicle’s velocity as a result of the agent’s action execution, higher reward values might end up being associated with longer actions. This means that the agent might falsely learn to prefer certain longer actions. To resolve this issue, we modified the agent’s reward function such that the reward is normalized by the time taken to execute the corresponding action.

5. Adding to 4, in order to reduce the huge difference in the actions’ execution times, we associated the simple “accelerate”/”decelerate”/”no acceleration” actions with a certain time period in which to be taken. In other words, the new action is “accelerate for x seconds” instead of merely “accelerate”. All the same, if any action was found to be infeasible any time during the specified time period, the action is simply aborted and the reward calculated based on the actual execution time it took.
6. As a result of 5, we modified the acceleration ranges of the agent, and accordingly of the emergency vehicle, to be of lower values. The reason for this modification is that, now that the agent is asked to accelerate/decelerate for longer time periods, we do not want it to reach its maximum/minimum velocities during the execution of a single action. Consequently, based on our velocity ranges in 3, our chosen acceleration ranges for the agent and the EV are $[0, 0.0167]$ and $[0, 0.0333]$ respectively.
7. Based on the decreased values of acceleration/deceleration in 6, for both the agent and the EV, the number of possible velocity values within the allowed range increased. This led to a considerable increase in the size of our Q-Table.
8. Owing to the fact that our simulation on Gazebo is much more realistic than that on SUMO, the system is more susceptible to crashing as a result of the different existing uncertainties. Thus, we added another reason for which the episode can be forced to terminate: “simulation’s death”. If this happens, all the Q-Table updates which occurred in the last episode are discarded.
9. The track simulated in Gazebo is shorter than the one in SUMO in order to avoid having very long episodes. However, the interaction between the agent and the emergency vehicle is still guaranteed in the same way discussed in 5.3.1.5.

Apart from the above mentioned adjustments, all other assumptions in the problem formulation were kept the same.

According to, 4.1.5, our system can be divided into three main entities: the RL algorithm, the environment, and a master managing the system interactions. The responsibilities and provided services associated with each of the entities are summarized in the following subsections:

5.4.1 RL Algorithm

Since our adopted algorithm is Q-Learning, the master can request the following from the RL algorithm module:

- **Create/Load/Save/Update Q-Table**
- **Take Action:**

Algorithm 10: Taking Action Algorithm

```
Generate an array that have the all actions;
while While a feasible action is not found in the array do
    Pick an action;
    Send it to the high-level control master to execute it, refer to 5.1.9;
    Get its feasibility and time taken;
    if the picked action was feasible then
        | Send the action and time taken to master
    else
        | Remove the infeasible action from the array
    end
end
```

- **Disengage:** This is requested when the emergency vehicle exits the agent's window, in which case, the RL algorithm is prompted to notify the high-level control master (5.1.9) to deactivate the the external commands following in (5.1.8).

5.4.2 Environment

Several services can be requested by the master from the environment. Those services include:

- **Start/Reset/Close Simulation:** The track in Gazebo and the vehicle models together constitute our simulation. The master requests that the simulation is started at the beginning of either training or testing the RL model, reset between episodes and closed at the end of session. For optimization purposes during the resetting of the simulation, the vehicle models are deleted and re-generated in the track but the Gazebo simulator itself is not restarted unless it has crashed. This is detected by a secondary thread in the environment module which frequently pings the simulation and indicates when it has crashed. This thread is also the one indicating that the episode needs to end for the reason "simulation death".
- **Get Agent's State:** The full states of all the vehicles, agent and emergency vehicle in our case, are known to the environment. Upon request, the environment returns the agent's observed state.

- **Calculate Agent's Reward:** With the action execution time as an input, the environment calculates and returns the agent's normalized reward.

Aside from these services, the environment also continuously sends information to the master through a communication channel (topic). This information includes whether the emergency vehicle is inside or outside the agent's window; accordingly whether the master should request the disengagement of the RL model. The environment also publishes to the master the status of the current episode: whether or not it has finished and for what reason.

5.4.3 Master

As previously mentioned, the master is the maestro connecting the whole system together. Algorithms 11 and 12 summarize its role in training the RL model.

Algorithm 11: Master Training Algorithm

```

Request "Start Simulation" from Environment and block till it is ready
Run first episode using algorithm 11 if episode ended because simulation died
then
  | Prompt the RL Algorithm to discard Q-Table updates of this episode
12
while maximum number of episodes is not exceeded do
  | Request "Reset Simulation" from Environment and block till it is
  | ready
  | Run another episode using algorithm 12
  | if episode ended because simulation died then
  |   | Prompt the RL algorithm to discard Q-Table updates of this
  |   | episode
end
Request "Close Simulation" from Environment
Prompt RL Algorithm to save Q-Table

```

Algorithm 12: Master Episode Algorithm

```

while Environment did not signal the end of episode do
  | if Environment signaled that emergency vehicle is outside agent's
  |   window then
  |   | Prompt the RL Algorithm to declare disengagement
  |   | Block till Environment signals that the emergency vehicle is inside
  |   | agent's window  $\vee$  episode has ended
  |   | if Environment signaled that episode has ended then
  |   |   | Break
  |   | Request agent's initial state from Environment
  |   | Let RL Algorithm prompt taking action
  |   | Request agent's final state from Environment
  |   | Request calculation of agent's reward from Environment
  |   | Prompt RL Algorithm to update Q-Table
  | end
end

```

Chapter 6

Project Testing and Discussion

In this chapter we illustrate how we tested each of our modules and their integration. We also show the results of our experiments and follow them with a discussion. This chapter deals with the main three modules presented previously throughout the thesis: autonomous functionalities, communication and collaboration techniques.

In testing autonomous functionalities and system integration on ROS, we verified that the sub-modules are working properly by comparing the expected outcomes from every module discussed in the methodology, to the results generated from actual testing on the simulation environments: Gazebo and RViz. For every module, we designed scenarios and test cases that verifies a list of checks and discussed them briefly. Then we analyze whether the results are consistent with the expected outcomes or not. If they are consistent, then the module is verified and is discussed whether it needs some modification to improve the results. If they were not consistent, the reasons were explained.

In testing our RL models, we quantify their effect on the system's main metric: EV travel time, in the SUMO simulation environment.

6.1 Autonomous Functionalities

6.1.1 Mapping and Localization

1. Mapping

As mentioned in section 5.1.1, we used GIMP [81] to refine the generated map. Below are the results for the map before and after refining it.



(a) Resulted map before editing with GIMP.



(b) Resulted map after editing with GIMP.

Figure 6.1: Resulted map generated from Gmapping.

2. Localization

As for testing the vehicle's localization discussed in 5.1.1, it is addressed in other modules like the navigation in section 6.1.7 as the vehicle can localize itself. Also, with a given goal it can reach its destination.

6.1.2 Longitudinal Control

In order to test our longitudinal controller whose implementation was discussed in 5.1.2, we need to verify that our autonomous vehicle is able, under different circumstances, to accurately calculate its maximum safe velocity and carry out the appropriate throttle commands. For this purpose, we designed three testing scenarios. The aim of the first test is to verify that, in absence of a leading vehicle, our vehicle chooses to drive at the maximum allowable velocity. In the second test, our vehicle is preceded by another one and is expected to maintain the minimum gap at all times. In the third test, we verify that if, at some point, the leading vehicle performs maximum deceleration, our vehicle will have the time to adjust its velocity accordingly and still manage to keep the minimum gap. It is worth mentioning that in both tests two and three, the vehicles are within each other's communication ranges; thus information sharing is guaranteed between them.

1. Test 1:

- **Scenario:**

- Vehicle's starting position: in the middle lane at an x-coordinate of 10 m
- Vehicle's starting velocity: 0 m/s
- Vehicle's maximum velocity: 0.5 m/s
- Vehicle's maximum/minimum acceleration: $\pm 0.1 \text{ m/s}^2$

- **Results and discussion:**

As shown in table 6.1, the vehicle started out with a velocity of 0 m/s and kept performing maximum acceleration until it reached its maximum velocity and continued travelling with it. This coincides with the expected results since our vehicle has no other ones in front of it. Consequently, it is always safe to drive at the maximum velocity, and if it was not for the fact that the vehicle cannot accelerate above the maximum value, it would have chosen to travel at the maximum velocity from the start.

2. Test 2:

- **Scenario:**

- Follow vehicle's starting position: in the middle lane at an x-coordinate of 18 m (shown in red in figure 6.2)
- Follow vehicle's starting velocity: 0 m/s
- Follow vehicle's maximum velocity: 0.5 m/s
- Follow vehicle's maximum/minimum acceleration: $\pm 0.1 \text{ m/s}^2$

- Lead vehicle's starting position: in the middle lane at an x-coordinate of 20 m (shown in yellow in figure 6.2)

Time (sec)	Vehicle Velocity (m/s)
0	0
1	0.100000028834
2	0.199998297978
3	0.299982902831
4	0.399979595418
5	0.497777979084
6	0.499994486991
7	0.500000000102
8	0.499899889994

Table 6.1: Longitudinal Control - Test 1 Results

- Lead vehicle's starting velocity: 0 m/s
- Lead vehicle's maximum velocity: 0.2 m/s
- Lead vehicle's maximum/minimum acceleration: $\pm 0.2 \text{ m/s}^2$

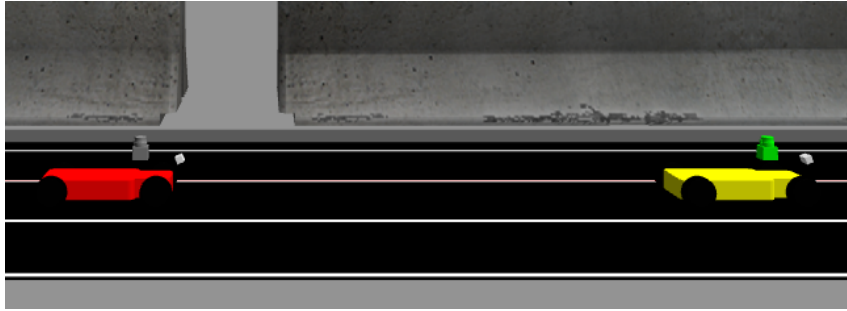


Figure 6.2: Longitudinal Control - Test 2: Initial relative vehicle positions.

• Results and discussion:

The results of the test are demonstrated in table 6.2. We can notice that the leading vehicle behaved in the same way as our vehicle in test 1. This is because it, too, is not preceded by any vehicles; thus it uses its maximum acceleration 0.2 m/s^2 until it reaches its maximum velocity 0.2 m/s and continues with it.

As for our follow vehicle, we can notice that it used its maximum acceleration only for the first 4 seconds. In the 5th second, it accelerated with a value less than the maximum. After that, it kept decelerating until its velocity became almost the same as that of its leading vehicle at the 12th second. Figure 6.3 shows the relative position between the vehicles at that time. Our follow vehicle then continued travelling at this same velocity; following the lead vehicle.

This agrees with the expected behavior. The distance between our vehicles was large enough at the beginning to allow for our follow vehicle to accelerate without breaking the minimum gap. When the follow vehicle accelerated and the distance between the vehicles decreased, the vehicle had to reduce its acceleration to avoid getting dangerously close to the

Time (sec)	Lead Vehicle Velocity (m/s)	Follow Vehicle Velocity (m/s)
0	0	0
1	0.199999888247	0.100000618849
2	0.199998447978	0.199995768353
3	0.200002002556	0.299702592357
4	0.199985552145	0.399726186689
5	0.199888755233	0.458158939312
6	0.199999874456	0.421969491718
7	0.198789998989	0.370105151177
8	0.199897122102	0.319735642604
9	0.199658972204	0.275992904082
10	0.199889552656	0.243248995639
11	0.199988592546	0.220834216365
12	0.198995523545	0.203398102118
13	0.199852563625	0.196112537823
14	0.199879855822	0.194719865449
15	0.199987522211	0.201073116384
16	0.199999875233	0.194627167177

Table 6.2: Longitudinal Control - Test 2 Results

leading vehicle. This reduction in acceleration continued until the velocities of both vehicles matched; at which point and forward, the minimum gap was perfectly maintained as evident in figure 6.3.

3. Test 3:

- **Scenario:**

After the two vehicles were now travelling together at the same velocity in test 2 and our follow vehicle was maintaining the required minimum gap, the lead vehicle was prompted to suddenly perform maximum deceleration.

- **Results and discussion:**

As shown in table 6.3, since the lead vehicle's speed was 0.2 m/s , it dropped down to 0 m/s upon performing its maximum deceleration of 0.2 m/s^2 . On the other hand, the follow vehicle's velocity started decreasing until it stopped as well at the 19^{th} second.

It can be seen in figure 6.4, depicting the relative position of the vehicles when they both stopped, that the follow vehicle succeeded in reacting to the lead vehicle's sudden maximum deceleration. It was prompted to decelerate and reduce its velocity to 0 m/s while still keeping the minimum gap; thus avoiding any collisions.

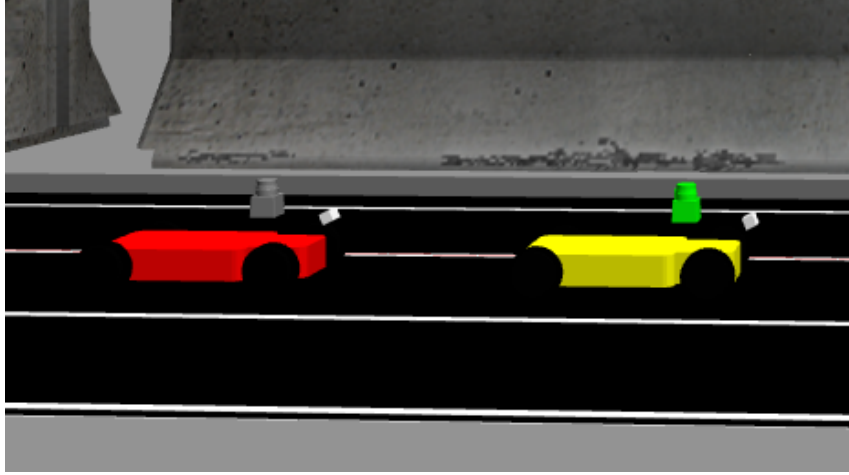


Figure 6.3: Longitudinal Control - Test 2: Final relative vehicle positions.

Time (sec)	Lead Vehicle Velocity (m/s)	Follow Vehicle Velocity (m/s)
16	0.199999875233	0.194627167177
17	0	0.131511928593
18	0	0.015579601206
19	0	0
20	0	0

Table 6.3: Longitudinal Control - Test 3 Results

6.1.3 Lateral Control

In order to test the performance of our Stanley controller whose implementation was explained in 5.1.3, we designed two different tests. The first test is to have our vehicle starting off with a heading similar to that of the trajectory to follow but with a significant cross-track error. The second test is to start with a zero cross-track error between the vehicle and the trajectory but with a different heading. The vehicle is expected to manage to follow the trajectory in both cases with minimum oscillations.

1. Test 1:

- **Scenario:**

- Trajectory way-points: along the center-line of the right-most lane
- Vehicle's starting position: at the start of the middle lane
- Vehicle's starting orientation: has a yaw of 0 radians
- Vehicle's velocity: 0.1 m/s

- **Results and discussion:**

The vehicle managed to successfully follow the given trajectory with almost no oscillations. The steering angle started out large enough to account for the large cross-track error which was dominating over the heading error as shown in figure 6.5a. As the cross-track error decreased, the steering angle started decreasing as well until the heading error was

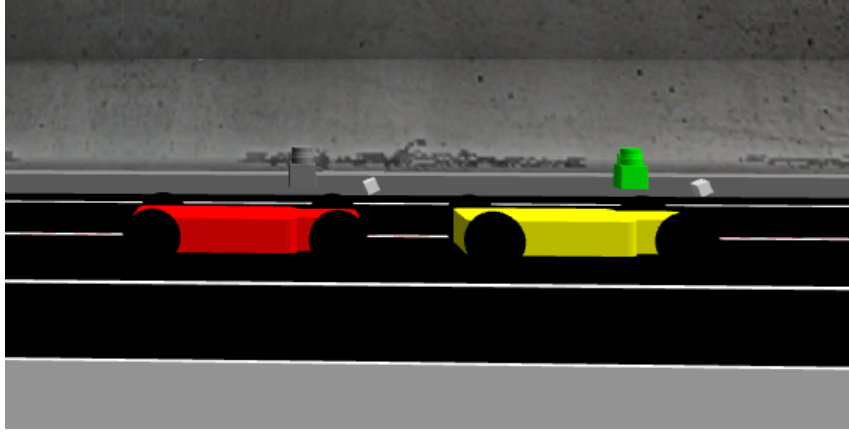
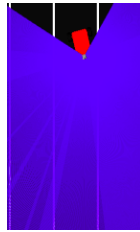
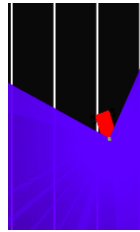


Figure 6.4: Longitudinal Control - Test 3: Final relative vehicle positions.

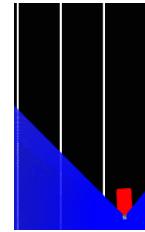
now dominating, 6.5b. Finally, the vehicle converged to the trajectory without oscillating further, 6.5c.



(a) Cross-track error dominating.



(b) Heading error dominating.



(c) Vehicle converging to path.

Figure 6.5: Stanley Controller - Test 1: Large initial cross-track error

2. Test 2:

- **Scenario:**

- Trajectory way-points: along the center-line of the right-most lane
- Vehicle's starting position: at the start of the right-most lane
- Vehicle's starting orientation: has a yaw of -0.25 radians
- Vehicle's velocity: 0.1 m/s

- **Results and discussion:**

The vehicle managed to successfully follow the given trajectory without any oscillations. The steering angle started out large enough to account for the heading error which was clearly dominating the cross-track error as shown in figure 6.6a. Consequently, the heading error decreased and the cross-track error slightly increased, 6.6b. The steering angle then accounted for this cross-track error and the vehicle converged to the trajectory without further oscillations, 6.6c.

Both tests were then repeated but with the vehicle travelling at 0.5 m/s instead of 0.1 m/s. In both tests, the high-speed vehicle was able to successfully follow the trajectory without oscillations. However, it is worth mentioning that the vehicle's behavior was slightly more stable at the lower speed; the damping slightly diminished

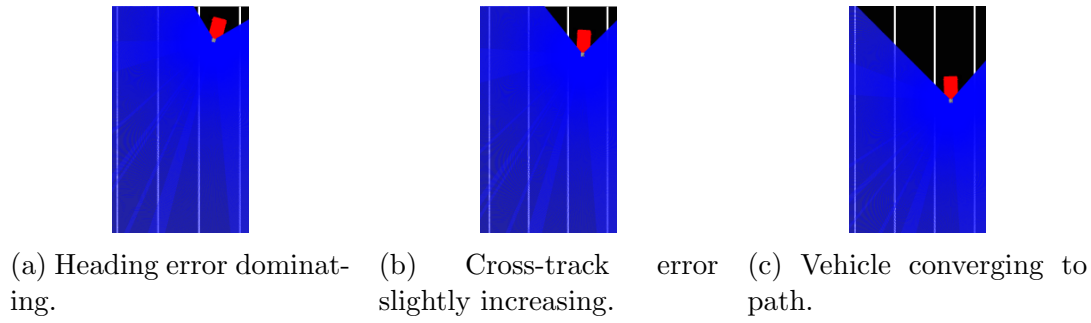


Figure 6.6: Stanley Controller - Test 2: Large initial heading error

by increasing the velocity. Although the controller still worked fine within these velocity ranges, active damping might be required at velocities that are considerably higher.

6.1.4 Lane Keeping

Our lane keeping module, explained in 5.1.4, begins with the lane detection and way-point generation step and ends with laterally and longitudinally controlling our vehicle in order to follow those way-points with the appropriate safe velocity. Since our longitudinal and lateral control modules were tested in sections 6.1.2 and 6.1.3, this section is dedicated to testing the lane detection and way-point generation algorithms. For this purpose, we designed two main tests. The aim of the first test is to verify that our algorithm is able to detect the lane lines in images taken by the vehicle's black and white camera. For the second test, we check whether, based on the detected lines, reasonable way-points are generated to be later followed by the vehicle.

- **Test:**

- **Scenario:**

A video was recorded from the vehicle's camera while driving forward in the track (manually controlled). The transform to the bird's eye view was then applied and the lane lines were detected. Our way-point generation algorithm was then used and another video with the way-points shown in the bird's eye view was outputted.

- **Results and Discussion:**

Figure 6.7 shows the snapshots from both the video taken by the vehicle's camera and that with the final generated way-points in the bird's eye view. As is clear in the figure, the output way-points, shown in blue on the figure, appear to be aligned with the lane's center-line at all times; which coincides with the expected output.

6.1.5 Lane Changing

In order to test our lane changing module explained in 5.1.5, test cases of many different scenarios were designed in order to show that lane changing is sometimes

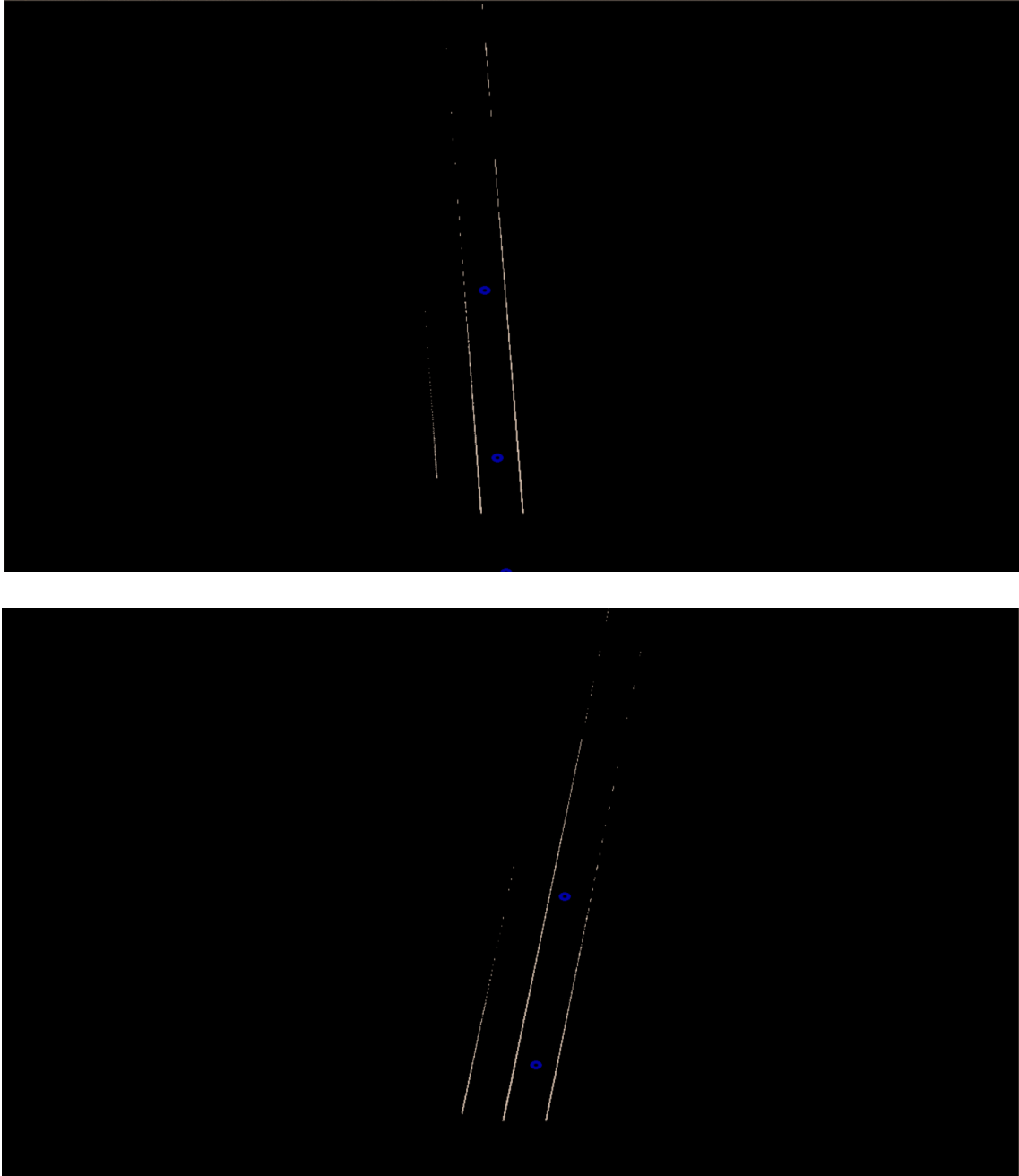


Figure 6.7: Way-points generated (in blue) appear at the center of the thresholded lane (in white) at all times in our tests.

a feasible action while other times it is not, or even that its feasibility cannot be predicted. As mentioned in section 5.1.5, we assume that the environment will have the ego vehicle and N vehicles surrounding it.

In this section, we assume that there are only 2 vehicles in the environment; the ego vehicle and another one. The ego vehicle is the red vehicle and the yellow one is the other vehicle taking position A, B, C or D. We perform 6 test cases scenario. If the lane change actions tested to be infeasible; the ego vehicle doesn't move and this behaviour only appears in the lane changing module test unit. If the action was infeasible at running the whole system there is a different behaviour as it will be mentioned later.

1. Test 1:

• Scenario:

- Surrounding vehicle's position case : Case A
- Distance between vehicles: 1.5 m
- Ego vehicle's starting velocity: 0.3 m/s
- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -41.5 m
- Surrounding vehicle's lane: Lane 0

```
[INFO] [1592466691.664186, 4753.285000]: Desired direction :0
[INFO] [1592466691.793261, 4753.318000]: Desired direction :1
[INFO] [1592466691.794126, 4753.318000]: Found vehicle A at index: 1
[INFO] [1592466691.794487, 4753.318000]: Found vehicle B at index: -1
[INFO] [1592466691.794932, 4753.318000]: Found vehicle D at index: -1
[INFO] [1592466691.795320, 4753.318000]: Lane change is: Feasible
[INFO] [1592466691.954758, 4753.368000]: Desired direction :1
[INFO] [1592466692.082749, 4753.419000]: Desired direction :1
```

Figure 6.8: Lane Changing - Test 1: Case A - Terminal log - Feasible.

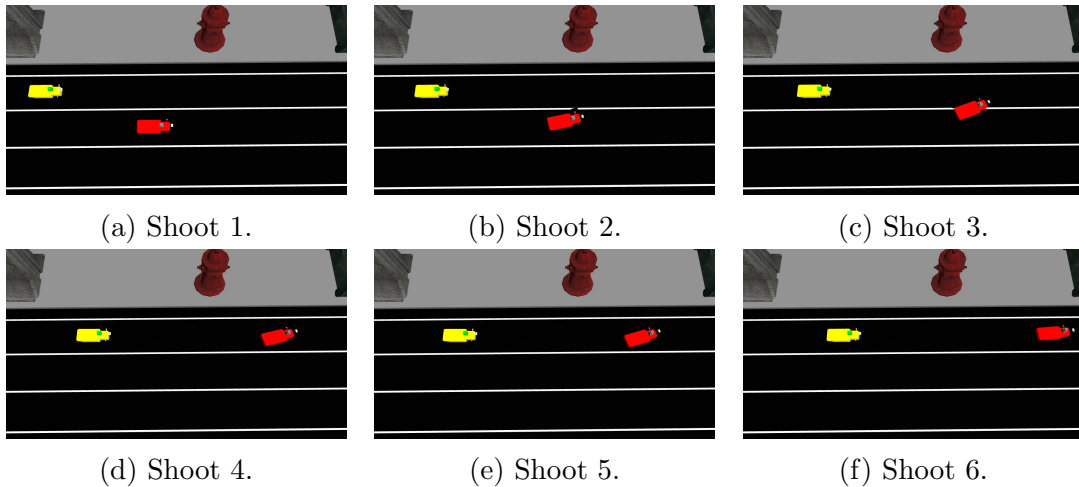


Figure 6.9: Lane Changing - Test 1: Case A - Feasible.

- **Results and discussion:**

As shown in table 6.4, figure 6.9 and 6.8, Vehicle at position A, the yellow vehicle, was found in the environment and lane change action checked to be feasible and will be executed. The red vehicle started from its lane and then was prompted to make a lane change action. It starts lane changing with its current velocity 0.3 m/s and the steering angle is changing throughout the time till it finishes executing the action and stops. On the other side the yellow vehicle is executing lane keeping action following Krauss model; following the red vehicle and keeping out the maximum safe velocity and distance. The results in table 6.4 are approximate to the stated above values.

The results are logical because there is a vehicle in the ego vehicle's target lane. When the ego vehicle hypothetically reached the center-line, the yellow AV was still behind it and before the centre-line so it is in position A as expected. Also, the distance between the two vehicles was enough for the yellow vehicle to adapt its velocity following the red one.

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.300000011921	0.0	0.10000000149	0.0
0.300936579704	0.142148435116	0.10000000149	6.74655893818e-05
0.300936579704	-0.0928664803505	0.10000000149	0.000105529383291
0.300936579704	-0.0659889876842	0.10000000149	3.91086323361e-05

Table 6.4: Lane Changing velocity and steering angle throughout time - Test 1: Case A - Feasible

2. Test 2:

- **Scenario:**

- Surrounding vehicle's position case : Case A
- Distance between vehicles: 1 m
- Ego vehicle's starting velocity: 0.3 m/s
- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -41 m
- Surrounding vehicle's lane: Lane 0

- **Results and discussion:**

As shown in table 6.5, figure 6.10 and 6.11, vehicle at position A was found in the environment and lane change action checked to be infeasible and will not be executed. The red vehicle's initial velocity was 0.3 m/s

```

[INFO] [1592466406.569852, 4754.452000]: Desired direction :0
[INFO] [1592466406.818236, 4754.502000]: Desired direction :1
[INFO] [1592466406.821333, 4754.502000]: Found vehicle A at index: 1
[INFO] [1592466406.823786, 4754.502000]: Found vehicle B at index: -1
[INFO] [1592466406.825370, 4754.502000]: Found vehicle D at index: -1
[INFO] [1592466406.826054, 4754.502000]: Lane change is: Infeasible
[INFO] [1592466407.032899, 4754.552000]: Desired direction :0
[INFO] [1592466407.292405, 4754.603000]: Desired direction :0
[INFO] [1592466407.519055, 4754.653000]: Desired direction :0

```

Figure 6.10: Lane Changing - Test 2: Case A - Terminal log - Infeasible.

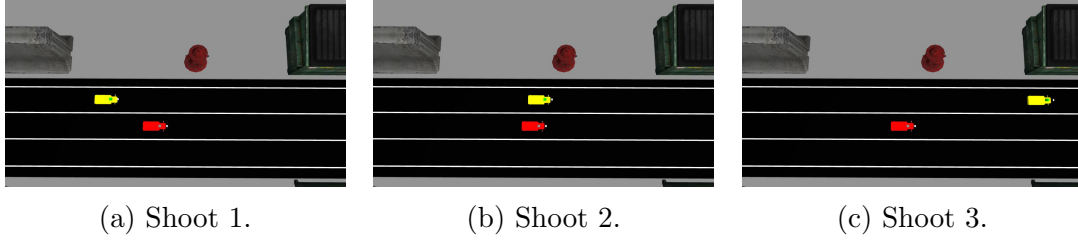


Figure 6.11: Lane Changing - Test 2: Case A - Infeasible

but it stopped as it can't perform lane change action and the yellow vehicle continues in its path.

The results are logical because there is a vehicle in the ego vehicle's target lane. When the ego vehicle hypothetically reached the center-line, the yellow AV was still behind it and before the centre-line so it is in position A as expected. However, it may be in the lane change region when the red vehicle starts to change its lane. So it returned that the lane change action is not feasible because the distance and relative velocities of the two vehicles are not enough..

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.3	0.0	0.10000000149	0.0
0.0	0.0	0.10000000149	-9.10808303161e-055
0.0	0.0	0.10000000149	-7.47770172893e-05
0.0	0.0	0.10000000149	- 0.000134485177114

Table 6.5: Lane Changing velocity and steering angle throughout time - Test 2: Case A - Infeasible

3. Test 3:

- **Scenario:**

- Surrounding vehicle's position case : Case Indeterminate A or B
- Distance between vehicles: $0.5\ m$
- Ego vehicle's starting velocity: $0.3\ m/s$

- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -39.5 m
- Surrounding vehicle's lane: Lane 0

```
[INFO] [1592468534.241030, 4753.686000]: Desired direction :0
[INFO] [1592468534.422420, 4753.741000]: Desired direction :1
[INFO] [1592468534.423498, 4753.741000]: Lane change is: Infeasible
[INFO] [1592468534.562532, 4753.784000]: Desired direction :0
[INFO] [1592468534.821196, 4753.845000]: Desired direction :0
[INFO] [1592468534.975951, 4753.888000]: Desired direction :0
```

Figure 6.12: Lane Changing - Test 3: Indeterminate Case A or B - Terminal log - Infeasible.

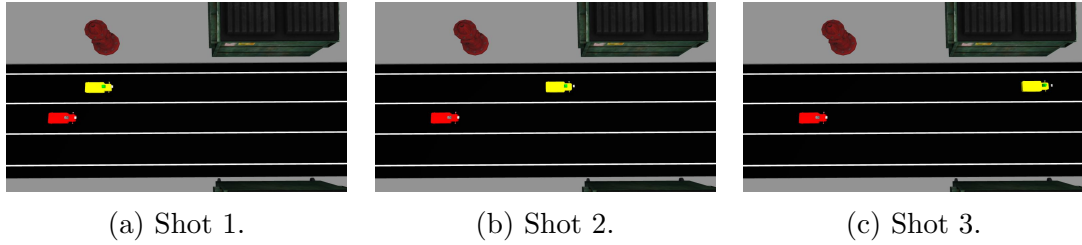


Figure 6.13: Lane Changing - Test 3: Indeterminate Case A or B - Infeasible

• Results and discussion:

As shown in table 6.6 and figures 6.14, 6.13, the yellow vehicle seems to be at position B but this is not the case. As it is not a determined case; can't determine whether it is at position A or B because of the small gap between the two vehicles. Consequently, the algorithm returns that lane changing is not feasible according to 5.1.5. The red vehicle's initial velocity was 0.3 m/s but it stopped as it can't perform lane change action and the yellow vehicle continues in its path.

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.3	0.0	0.10000000149	0.0
0.0	0.0	0.10000000149	6.74655893818e-05
0.0	0.0	0.10000000149	0.000105529383291
0.0	0.0	0.10000000149	3.91086323361e-05

Table 6.6: Lane Changing velocity and steering angle throughout time - Test 3: Case Indeterminate A or B - Terminal log - Infeasible

4. Test 4:

- **Scenario:**

- Surrounding vehicle's position case : Case B
- Distance between vehicles: 2 m
- Ego vehicle's starting velocity: 0.3 m/s
- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -38 m
- Surrounding vehicle's lane: Lane 0

```
[INFO] [1592480647.805912, 4752.932000]: Desired direction :0
[INFO] [1592480647.997087, 4752.983000]: Desired direction :0
[INFO] [1592480648.264502, 4753.035000]: Desired direction :0
[INFO] [1592480648.468339, 4753.082000]: Desired direction :1
[INFO] [1592480648.469245, 4753.082000]: Found vehicle A at index: -1
[INFO] [1592480648.469795, 4753.082000]: Found vehicle B at index: 1
[INFO] [1592480648.470267, 4753.082000]: Found vehicle D at index: -1
[INFO] [1592480648.470726, 4753.082000]: Lane change is: Infeasible
[INFO] [1592480648.685471, 4753.123000]: Desired direction :0
[INFO] [1592480648.864250, 4753.183000]: Desired direction :0
[INFO] [1592480649.057036, 4753.232000]: Desired direction :0
```

Figure 6.14: Lane Changing - Test 4: Case B - Terminal log - Infeasible.

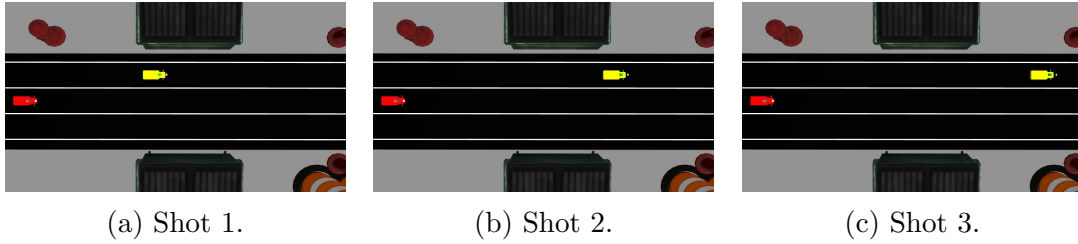


Figure 6.15: Lane Changing - Test 4: Case B - Infeasible

- **Results and discussion:**

As shown in table 6.7, figure 6.14 and 6.15, vehicle at position B was found in the environment and lane change action checked to be infeasible and will not be executed. The red vehicle's initial velocity was 0.3 m/s but it stopped as it can't perform lane change action and the yellow vehicle continues in its path.

The results are logical because there is a vehicle in the ego vehicle's target lane. When the ego vehicle hypothetically reached the center-line, the yellow AV was ahead of the red one so it is in position B as expected. However, it may be before the right-line when the red vehicle starts to change its lane or both the vehicles can't maintain the minimum safe velocity between each other. So it returned that the lane change action is not feasible because the distance and relative velocities of the two vehicles are not enough.

5. Test 5:

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.3	0.0	0.10000000149	0.0
0.0	0.0	0.10000000149	8.4286366473e-05
0.0	0.0	0.10000000149	-
0.0	0.0	0.10000000149	0.000128690691781
0.0	0.0	0.10000000149	-6.5345207986e-05

Table 6.7: Lane Changing velocity and steering angle throughout time - Test 4: Case B - Terminal log - Infeasible

- **Scenario:**

- Surrounding vehicle's position case : Case C
- Distance between vehicles: 1 m
- Ego vehicle's starting velocity: 0.3 m/s
- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -41 m
- Surrounding vehicle's lane: Lane 1

```
[INFO] [1592501503.909425, 4753.489000]: Desired direction :0
[INFO] [1592501504.281601, 4753.539000]: Desired direction :0
[INFO] [1592501504.817548, 4753.588000]: Desired direction :1
[INFO] [1592501504.818318, 4753.588000]: Found vehicle A at index: -1
[INFO] [1592501504.818764, 4753.588000]: Found vehicle B at index: -1
[INFO] [1592501504.819145, 4753.588000]: Found vehicle D at index: -1
[INFO] [1592501504.819541, 4753.588000]: Lane change is: Feasible
[INFO] [1592501505.244643, 4753.642000]: Desired direction :1
[INFO] [1592501505.471122, 4753.692000]: Desired direction :1
[INFO] [1592501505.773883, 4753.737000]: Desired direction :1
```

Figure 6.16: Lane Changing - Test 5: Case C - Terminal log - Feasible.

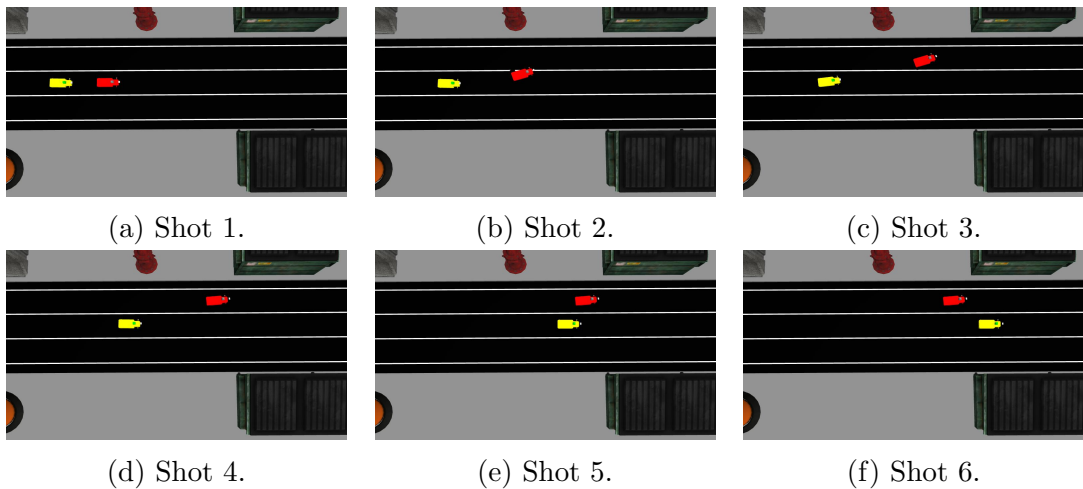


Figure 6.17: Lane Changing - Test 5: Case C - Feasible

- **Results and discussion:**

As shown in table 6.8, figure 6.17 and 6.16, vehicle at position C, the yellow vehicle, was found in the environment and lane change action checked to be feasible and will be executed. The red vehicle started from its lane and then was prompted to make a lane change action. It starts lane changing with its current velocity 0.3 m/s and the steering angle is changing throughout the time till it finishes executing the action and stops. On the other side the yellow vehicle is executing lane keeping action following krauss model. The results in table 6.8 are approximate to the stated above values.

The results are logical because there is a vehicle in the ego vehicle's lane. The yellow vehicle is behind the red one and in the same lane; so it is at position C as expected. The yellow vehicle will not affect the lane change action by any means because it will follow the red one using krauss model at any time. When the red vehicle is outside the yellow one window or in the other lane; the yellow vehicle will increase its velocity gradually.

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.300000011921	0.0	0.10000000149	-0.042406991124
0.300974905491	0.0597243718803	0.10000000149	0.00270679476671
0.300974905491	0.143652096391	0.10000000149	0.00182328152005
0.300974905491	-0.065448731184	0.10000000149	0.00165438361

Table 6.8: Lane Changing velocity and steering angle throughout time - Test 5: Case C - Feasible

6. Test 6:

- **Scenario:**

- Surrounding vehicle's position case : Case D
- Distance between vehicles: 1 m
- Ego vehicle's starting velocity: 0.3 m/s
- Ego vehicle's starting position: -40 m
- Ego vehicle's starting lane: Lane 1
- Ego vehicle's target lane: Lane 0
- Surrounding vehicle's starting velocity: 0.1 m/s
- Surrounding vehicle's starting position: -39 m
- Surrounding vehicle's lane: Lane 1

- **Results and discussion:**

As shown in table 6.9, figure 6.19 and 6.18, vehicle at position D, the yellow vehicle, was found in the environment and lane change action checked to be feasible and will be executed. The red vehicle started from its lane and then was prompted to make a lane change action. It starts lane changing with its current velocity 0.3 m/s and the steering angle

```
[INFO] [1592505864.753988, 4754.732000]: Desired direction :0
[INFO] [1592505865.204417, 4754.784000]: Desired direction :1
[INFO] [1592505865.205318, 4754.784000]: Found vehicle A at index: -1
[INFO] [1592505865.206670, 4754.785000]: Found vehicle B at index: -1
[INFO] [1592505865.207434, 4754.785000]: Found vehicle D at index: 1
[INFO] [1592505865.207926, 4754.785000]: Lane change is: Feasible
[INFO] [1592505865.523038, 4754.832000]: Desired direction :1
[INFO] [1592505865.831388, 4754.882000]: Desired direction :1
[INFO] [1592505866.269984, 4754.933000]: Desired direction :1
```

Figure 6.18: Lane Changing - Test 6: Case D - Terminal log - Feasible.



Figure 6.19: Lane Changing - Test 6: Case D - Feasible

is changing throughout the time till it finishes executing the action and stops. On the other side, the yellow vehicle is executing lane keeping action following krauss model. The results in table 6.9 are approximate to the stated above values.

The results are logical because there is a vehicle in the ego vehicle's lane. The yellow vehicle is in front of the red one and in the same lane; so it is at position D as expected. If the yellow vehicle stops or performs maximum deceleration all the time steps that the red vehicle takes to reach the centerline its position will be after the centerline; so it is a feasible lane change action. Moreover, the yellow vehicle will continue its path and will not be affected by the red vehicle at all and its velocity will increase gradually.

6.1.6 Path Planning

In this sub-section, we test our two main additions, explained in 5.1.6, to the path planning module.

- **Communication Layer Testing:**

In order to test the different functionalities that our communication costmap layer ought to be able to perform, we designed three tests. The aim of the first is to validate that our costmap layer is able to clear and mark cells as expected when being used as a layer of either the global costmap, used by the

Red vehicle velocity (m/s)	Red vehicle steering angle (rad)	Yellow vehicle velocity (m/s)	Yellow vehicle steering angle (rad)
0.300000011921	0.0	0.10000000149	-0.042406991124
0.272501945496	0.0689768791199	0.10000000149	6.0399153881e-05
0.272501945496	-0.111118324101	0.10000000149	-7.31826658011e-05
0.272501945496	-0.0656164810061	0.10000000149	-8.78940845723e-05

Table 6.9: Lane Changing velocity and steering angle throughout time - Test 6: Case D - Feasible

global planner to generate the global path, or the local layered costmap, used by the local planner to generate the local path. Hence, our ego vehicle and one another are placed such that they are both inside each other's communication range, and then the other vehicle is allowed to move. The expected result of this test is that our vehicle manages to mark the footprint of the other at all times, and clear the cells which were previously marked but are now free.

The aim of the second test is to make sure that the global planner does take into consideration the cells marked as LETHAL by our communication layer when added to the global costmap, but not the local one. Thus, our vehicle is given a goal to which it is allowed to navigate in the presence of the other vehicle. The global path generated by the global planner as well as our vehicle's motion are to be observed. It is expected that the global path does not pass through any of the marked cells within the footprint of the other vehicle.

For the third test, we verify that the local planner considers the cells marked as LETHAL by our costmap layer when it is added to the local costmap but not the global costmap.. Again, our vehicle is given the same task of navigating to goal in the presence of the other vehicle. The expected result is that, regardless of the global path generated by the global planner, the local planner forces our vehicle to avoid any marked cells within the other vehicle's footprint.

For all the tests, in order to be able to display the output of the communication layer, the vehicles' LiDAR sensors were raised a little above the vehicle bodies so the bodies are not detected by the sensors, and hence not displayed in the obstacle layer of the costmaps. To demonstrate the results of all three tests, figures of the vehicles' top-view in the different costmaps are shown in RViz, a 3D visualizer of environments, robots and sensor data [86].

1. Test 1:

– Scenario:

- * Global costmap layers in order:
 - (a) Static Layer
 - (b) Obstacle Layer

- (c) Communication Layer
- (d) Inflation Layer
- * Local costmap layers in order:
 - (a) Static Layer
 - (b) Obstacle Layer
 - (c) Communication Layer
- * Ego vehicle's starting position: in the right-most lane at an x-coordinate of -0.5 m (shown in red in figures 6.20 and 6.21)
- * Other vehicle's starting position: in the middle lane at an x-coordinate of 0.5 m (shown in yellow in figures 6.20 and 6.21)

– **Results and discussion:**

As shown in figures 6.20a and 6.21a of the global and local costmaps of our red ego vehicle, the footprint of the other vehicle was successfully marked as LETHAL in both costmaps. The yellow vehicle was then given a goal, indicated by RViz as an arrow, and thus allowed to move towards the goal. All while the vehicle was moving, the cells marked as LETHAL in both costmaps moved along with it, indicating successful clearing and marking of the free and occupied cells respectively as shown in figures 6.20b and 6.21b. Lastly, the yellow vehicle was given another goal which lies outside the bounds of the local costmap. As expected, the footprint was marked in the global costmap but not seen by the local costmap, as shown in figures 6.20c and 6.21c.

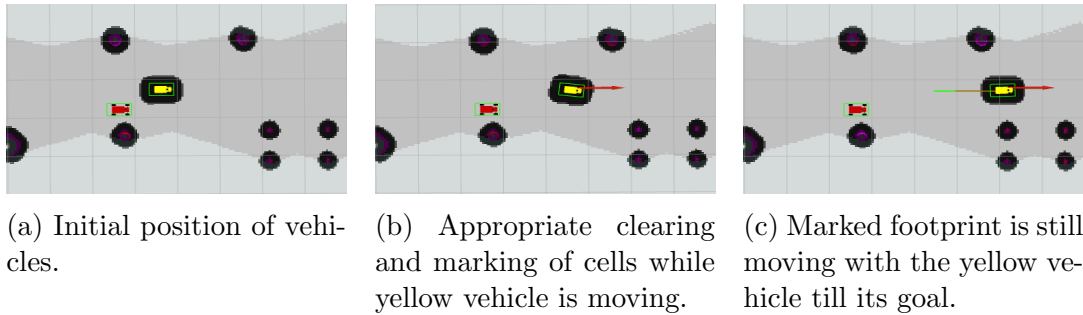


Figure 6.20: Communication Layer - Test 1: Ego (red) vehicle's global costmap (RViz).

2. **Test 2:**

– **Scenario:**

- * Global costmap layers in order:
 - (a) Static Layer
 - (b) Obstacle Layer
 - (c) Communication Layer
 - (d) Inflation Layer
- * Local costmap layers in order:
 - (a) Static Layer
 - (b) Obstacle Layer

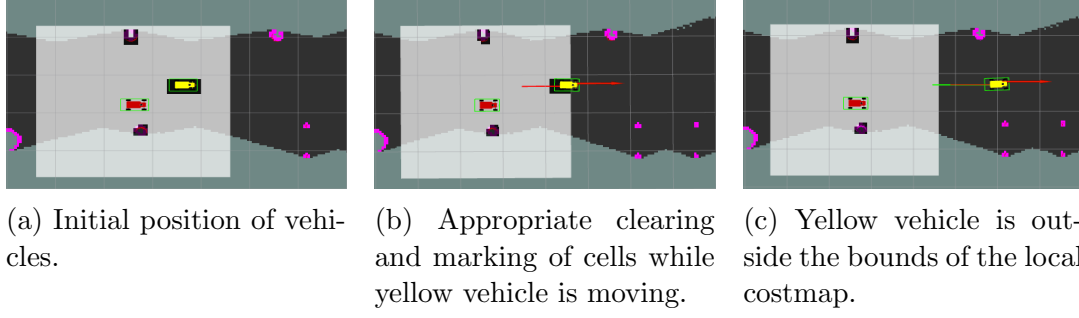


Figure 6.21: Communication Layer - Test 1: Ego (red) vehicle's local costmap (RViz).

* Both vehicles started at the same initial positions as those of test 1 and with the same color convention.

– **Results and discussion:**

Since the communication layer was only added to the ego vehicle's global costmap but not its local costmap, the other vehicle's footprint only appeared in the global costmap, as evident in figure 6.22. When our ego vehicle was given a goal in front of the yellow vehicle, the generated global path took the other vehicle's footprint into consideration and planned a path that is outside its bounds. This is shown in figure 6.23, where the arrow indicates the position of the goal and the drawn line indicates the generated global path. This coincides with the expected results since the global planner uses the global costmap in generating the path. Consequently, given that the global costmap marked the other vehicle's footprint as LETHAL, the global planner is aware of its existence and planned the path accordingly.

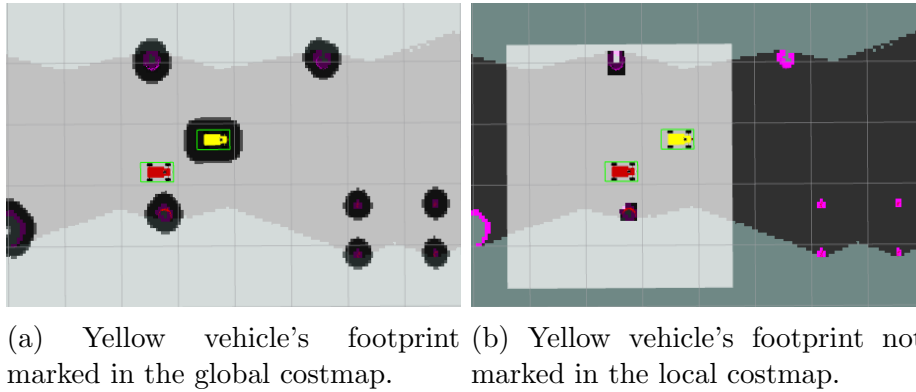


Figure 6.22: Communication Layer - Test 2: Communication layer added only to the ego (red) vehicle's global costmap (RViz).

3. **Test 3:**

– **Scenario:**

* Global costmap layers in order:

- (a) Static Layer
- (b) Obstacle Layer

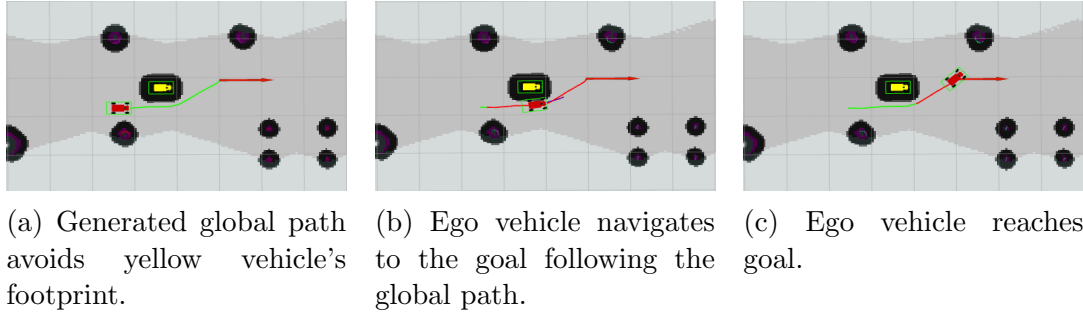


Figure 6.23: Communication Layer - Test 2: Ego (red) vehicle's global costmap while navigating to goal (RViz).

(c) Inflation Layer

* Local costmap layers in order:

(a) Static Layer

(b) Obstacle Layer

(c) Communication Layer

* Both vehicles started at the same initial positions as those of test 1 and with the same color convention.

– Results and discussion:

In this test, the communication layer was added to the ego vehicle's local costmap but not its global one. As a result, the other vehicle's footprint only appeared in the local costmap, as shown in figure 6.24. When the same goal as that in test 2 was given to our ego vehicle, the global planner generated a path that passed through the yellow vehicle's footprint. This makes sense since the communication layer is not a part of the global costmap, hence according to the global planner, these cells are free.

After the generation of the global path, the vehicle started to move towards the goal. However, the vehicle did not follow this generated global path and took a longer one which did not cross the yellow vehicle's footprint, as depicted in figure 6.25. This behavior matches the expected since the communication layer was a part of the local costmap. Therefrom, the local planner was aware of the presence of the yellow vehicle as its footprint was marked as LETHAL; hence it generated a local path for the ego vehicle to follow such that it did not collide with the yellow vehicle but managed to stick to the global path as much as possible, and eventually reached the goal.

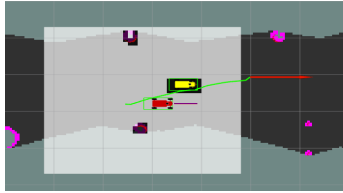


(a) Yellow vehicle's footprint not marked in the global costmap.

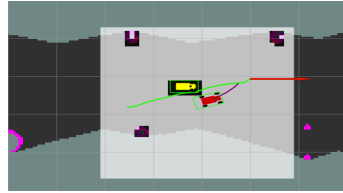


(b) Yellow vehicle's footprint marked in the local costmap.

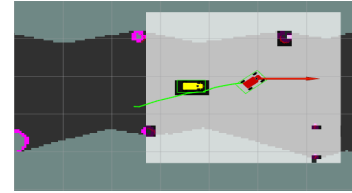
Figure 6.24: Communication Layer - Test 3: Communication layer added only to the ego (red) vehicle's local costmap (RViz).



(a) Generated global path does not avoid yellow vehicle's footprint.



(b) Ego vehicle deviates from the global path to avoid yellow vehicle.



(c) Ego vehicle reaches goal.

Figure 6.25: Communication Layer - Test 3: Ego (red) vehicle's local costmap while navigating to goal (RViz).

- **Custom Lane Planner Testing:** In order for the local planner to be verified, all control actions have to be tested in all lanes. We checked seven test cases in table 6.10. The table contains the cases and the chosen action in each one.

Number	Current (y-position)	Current Lane Number	Goal (y-position)	Future Lane Number	Chosen Action
1	0.2625	0	0.2914	0	Lane Keeping
2	0.2625	0	-0.3502	1	Right Lane Changing
3	-0.2625	1	-0.2641	1	Lane Keeping
4	-0.2625	1	0.4311	0	Left Lane Changing
5	-0.2625	1	-1.102	2	Right Lane Changing
6	-0.75	2	-0.7786	2	Lane Keeping
7	-0.75	2	0.7725	1	Left Lane Changing

Table 6.10: Path Planning - Test Cases: Scenarios and Results

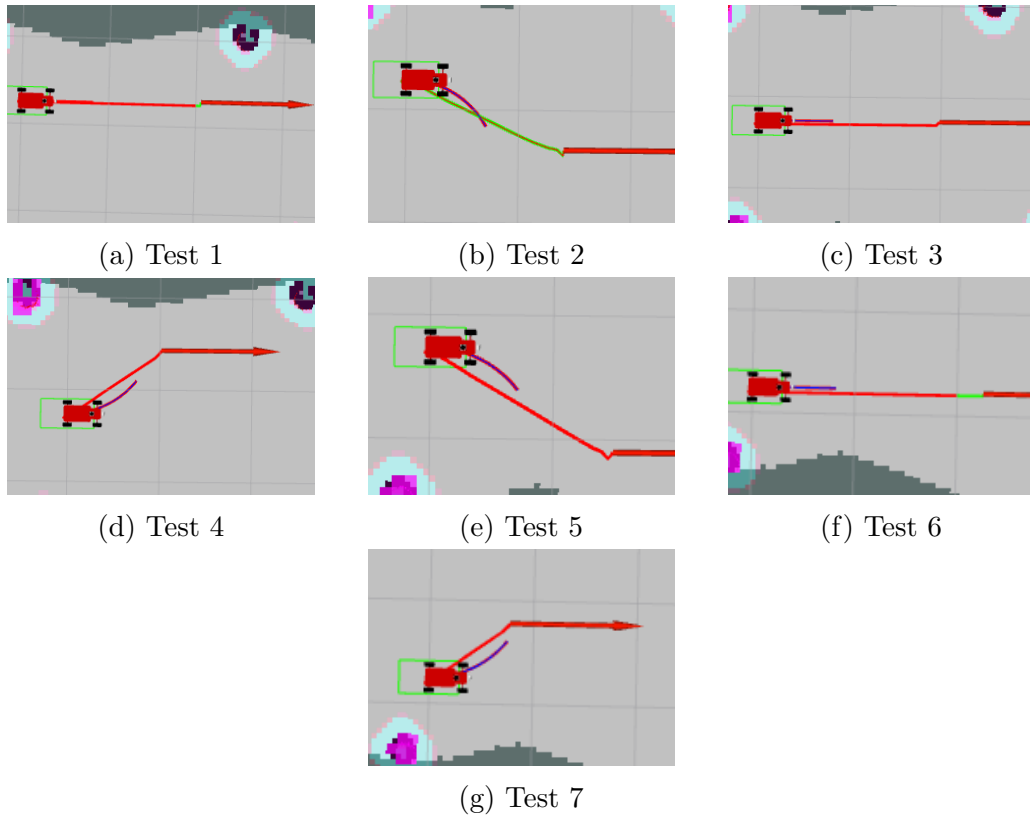


Figure 6.26: Test Cases and Results

6.1.7 Navigation

In order to test the navigation module discussed in 5.1.7, we to verify that all AVs in the same environment drive towards their goals, do the essential maneuvers to avoid

accidents and obstacles, either do lane keeping or lane changing, and stop when they reach their destinations. Therefore, we designed two test cases to measure how well the system is integrated in a smooth way through the following:

1. **Test 1:** It includes only one AV in a static environment, uses global and local costmaps. It checks if the AV:
 - (a) Make good path plans.
 - (b) Goes to multiple destinations in different lanes.
 - (c) Only does lane keeping and lane changing.
 - (d) Stops at the closest distance to the goal.
 - (e) Does not make redundant maneuvers.
 - (f) Does not hit any static obstacle.
 - (g) Does not stop unless it reaches the goal or get stuck.
2. **Test 2:** It includes two moving AVs, therefore, the environment becomes dynamic and uses global, local, and communication costmaps. It checks if the AVs can do the previous checks in test 1 beside:
 - (a) Don't make accidents with each other or with the surroundings (Do accurate feasibility checks)
 - (b) Use information communicated through v2v to make accurate lane keeping and lane changing.

1. Test 1:

- **Scenario:** The scenario includes four goals that requires unique control actions to reach them.

Type	x-position (m)	y-position (m)	Lane Number
Starting Position	-50.00	-0.2625	1
First Goal	-45.21	-0.2661	1
Second Goal	-39.13	-1.263	2
Third Goal	-30.54	0.2634	0
Fourth Goal	-24.26	-0.1573	1

Table 6.11: Navigation - Test 1: Scenario

• Results and discussion:

Goal Number	x-position (m)	y-position (m)	Lane Number
First Goal	-44.01	-0.2624	1
Second Goal	-37.35	-0.7556	2
Third Goal	-28.08	0.1662	0
Fourth Goal	-20.80	-0.2342	1

Table 6.12: Navigation - Test 1: Results

Goal Number	Error x-position (m)	Error in Lane Number
First Goal	-1.20	0
Second Goal	-1.78	0
Third Goal	-2.46	0
Fourth Goal	-3.46	0

Table 6.13: Navigation - Test 1: Error between planned goals and actual destinations

According to tables 6.12 and 6.13, the AV always reached the desired lane number; however, a relatively large error is observed in the goals' results (actual destinations). There are two reasons accounting for this error:

- (a) The frequency of the local planner in the path planning module (5.1.6) is much higher than the frequency of system controls, either longitudinal control (5.1.2) or lateral control (5.1.3).
To illustrate, consider the following example:

If the AV wants to use its maximum acceleration which is 0.02 m/s^2 , the longitudinal control calculates a new velocity every 1 sec . However, the local planner updates its velocity every 0.1 sec . Henceforward, the actual acceleration requested from the longitudinal control module maybe **10 times** the maximum possible acceleration, and that is definitely infeasible. This makes the local planner drop the requested velocity very fast when it comes closer to the goal that the longitudinal control cannot follow it anymore and uses its maximum eligible acceleration instead.

It should be noted that the values mentioned regarding the local planner in the last example are not accurate because its output depends on a cost function, refer to (4.1.2.6), which is beyond the scope of this thesis.

- (b) If a AV started lane change, it cannot do anything but complete it till the end. Therefore, if the custom local planner decided that the AV should lane change just before the goal, it will exceed the goal by a considerable length that depends to its lane changing velocity, and that was the case in the third and fourth goals in figure 6.30c and figure 6.31b.

Analysis of Goals to check the seven points (1):

- (a) **First Goal: Partially checked: 1a, 1b, 1c, 1d, 1g.**

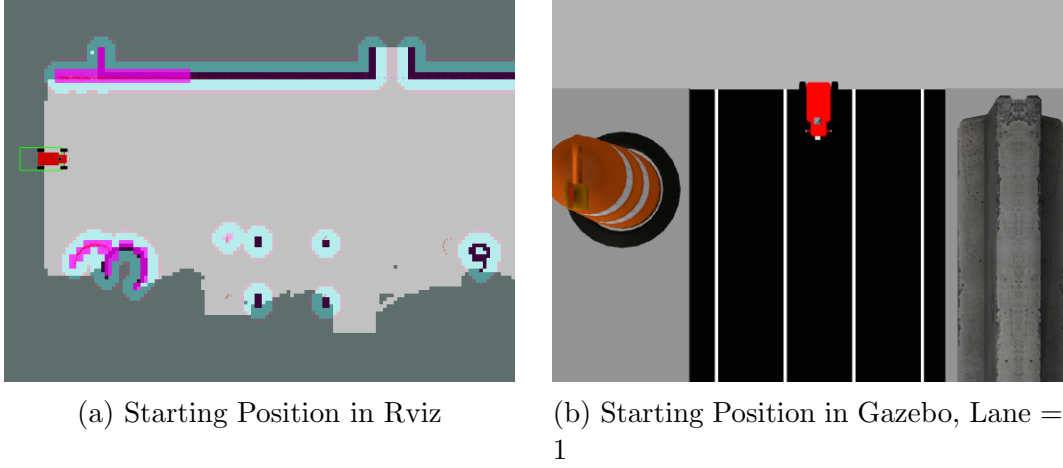


Figure 6.27: Test 1: Starting Position

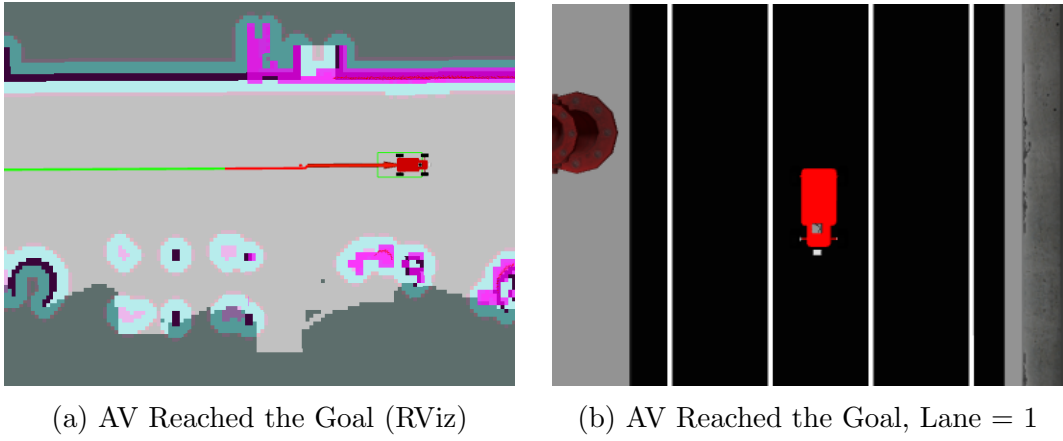


Figure 6.28: Test 1: First Goal

- (b) **Second Goal:** It is observed that the AV always stick to lanes. For example, this goal was outside lane number 2, its $y - position = -1.263m$; however, the lane's range in the y -direction is from $(-0.5, -1m)$. Therefore, we can observe a relatively larger error in the y -direction for this goal in figure 6.29b and table 6.12. Also, the AV will never hit static obstacles as it cannot go outside the lanes' boundaries. **Partially checked: 1a, 1b, 1c, 1d, 1f, 1g.**

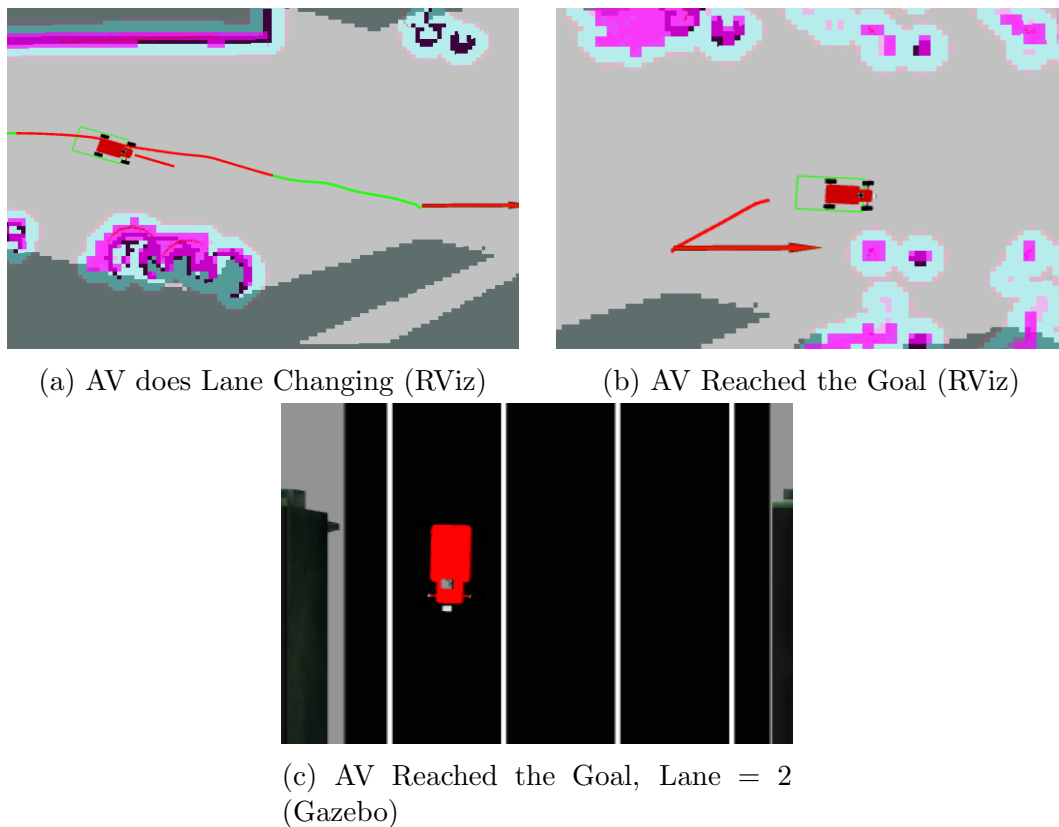


Figure 6.29: Test 1: Second Goal

- (c) **Third Goal:** According to the navigation algorithm, when the AV begins executing one lane change, the custom planner for lanes in the path planning module (5.1.6) will stop sending lane change actions until lane changing is completed. Therefore, the AV changed its lane in figure 6.30a, then it kept its lane for one time stamp 6.30b, and finally, it changed its lane again 6.30c.

Partially checked: 1a, 1b, 1c, 1d, 1e, 1f, 1g.

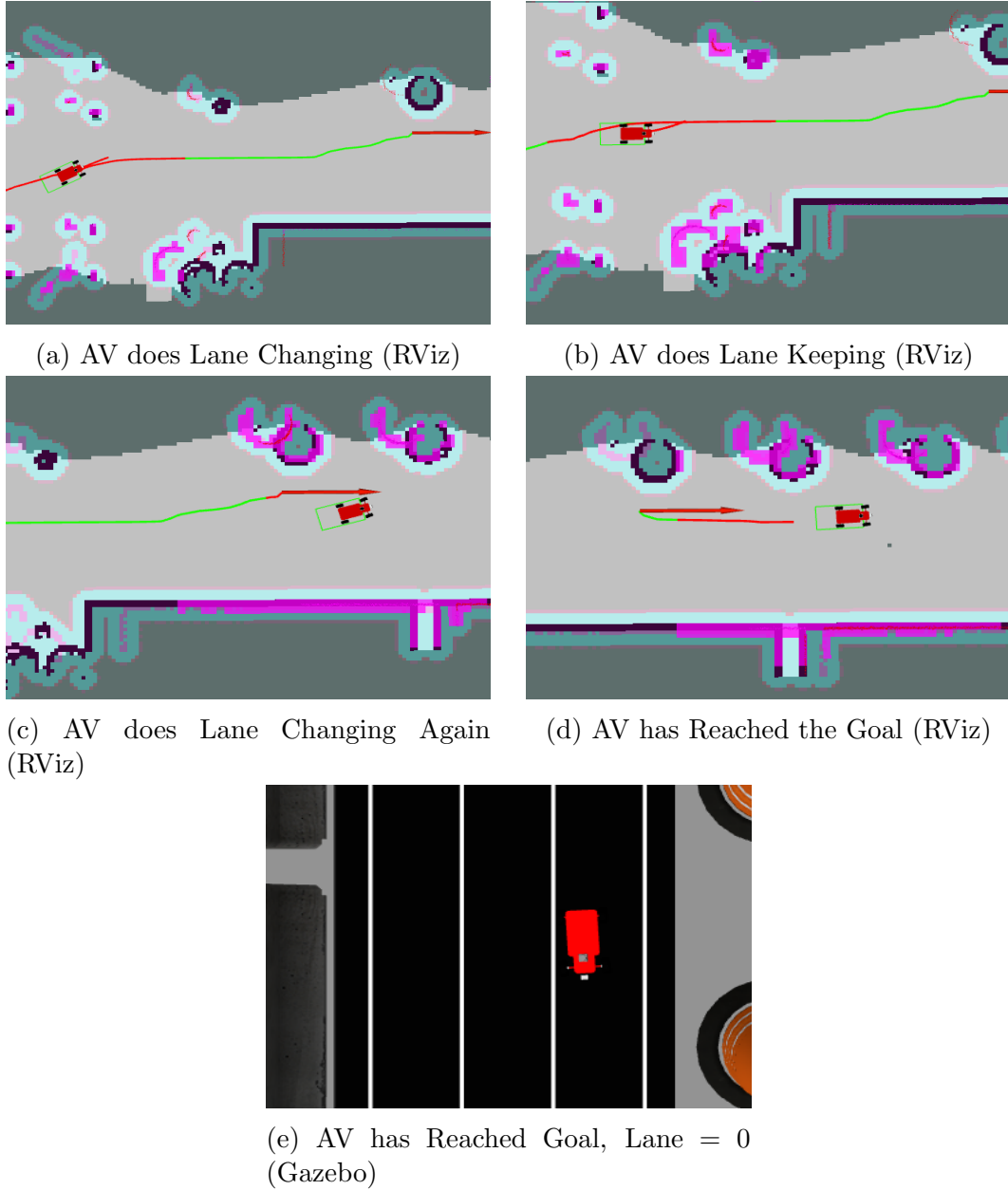


Figure 6.30: Test 1: Third Goal

- (d) **Fourth Goal:** It can be observed from fig 6.31a and table 6.11 that the desired goal was close to the AV's lane (Lane number 0). However, it lies in the beginning of lane number 1, along the y-direction. Hence, the path planning module took the decision to change lanes very near the goal.

Partially checked: 1a, 1b, 1c, 1d, 1e, 1f, 1g.

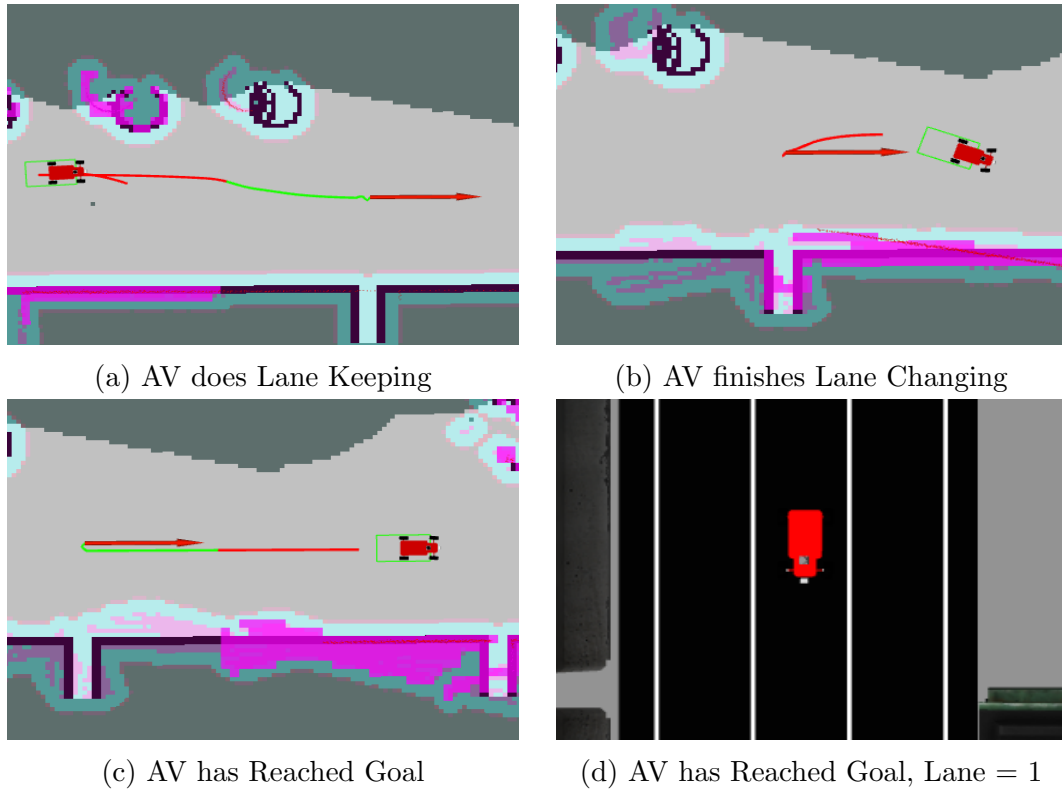


Figure 6.31: Test 1: Fourth Goal

2. Test 2:

- **Scenario:** The scenario includes two goals from two AVs.

Type	AV Color	x-position (m)	y-position (m)	Lane Number
Starting Position	red(1)	0	0.2625	0
Starting Position	yellow(2)	0	-0.75	2

Table 6.14: Navigation - Test 2: Scenario (Starting Positions)

Type	AV Color	x-position (m)	y-position (m)	Lane Number
First Goal	red(1)	16.36	-1.060	2
Second Goal	red(1)	25.02	0.4592	0
First Goal	yellow(2)	5.453	0.3638	0
Second Goal	yellow(2)	25.55	-0.7362	2

Table 6.15: Navigation - Test 2: Scenario (Desired Goals)

- **Results and discussions:** According to table 6.17, errors in the x-positions still occurred due to the previously mentioned reasons. In addition, one AV ended up in a wrong lane once. In this test, there is an additional reason that caused these errors:

- Having two AVs in the same environment will make both of them do feasibility checks before taking any action. Therefore, the AV will have extra restrictions preventing it from reaching the goal accurately or ending up in a correct lane number.

Type	AV Color	x-position	y-position	Lane Number
First Goal	red(1)	19.19	-0.6616	2
Second Goal	red(1)	26.15	0.1648	0
First Goal	yellow(2)	8.447	-0.2293	1
Second Goal	yellow(2)	27.17	-0.6564	2

Table 6.16: Navigation - Test 2: Results (Actual Destinations)

Type	AV Color	Error x-position	Error in Lane Number
First Goal	red(1)	-2.83	0
Second Goal	red(1)	-1.13	0
First Goal	yellow(2)	-2.99	1
Second Goal	yellow(2)	-1.62	0

Table 6.17: Navigation - Test 2: Errors in Results between desired goals and actual destinations

Analysis of Goals to check the nine points (1 and 2):

First Goal: Every AV will need to execute two lane changes to arrive to their destinations. In addition, the goal of the yellow AV lies on the path of the red AV, which means that the yellow AV will need to do feasibility checks in order to reach its goal. Furthermore, the goal of the yellow AV is closer which will drive the local planner to send it lane changing lane faster than the red AV that does not need to its lane fast.

In figure 6.34a, the yellow AV could perform its first lane change as it was feasible. However, it couldn't reach its destination because of the red AV in figure 6.34b.

Partially checked: 1, and 2.

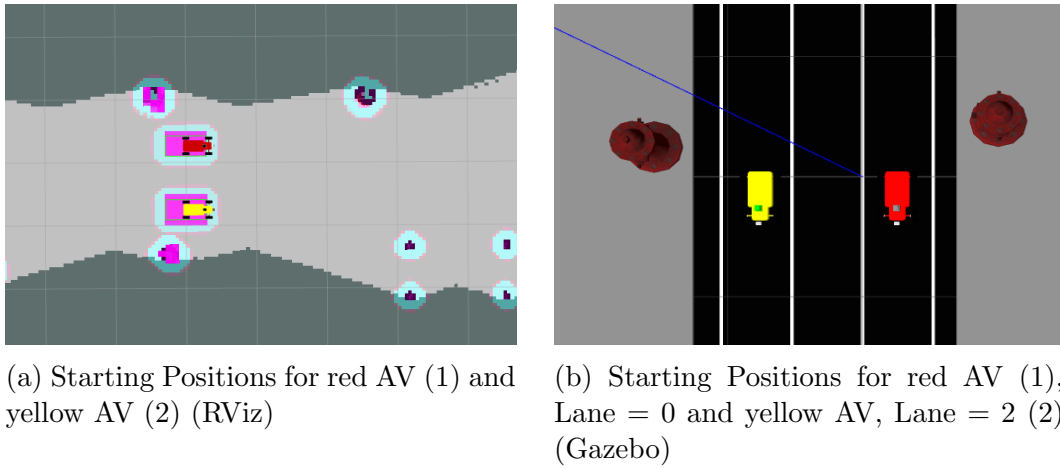


Figure 6.32: Test 2: Starting Positions

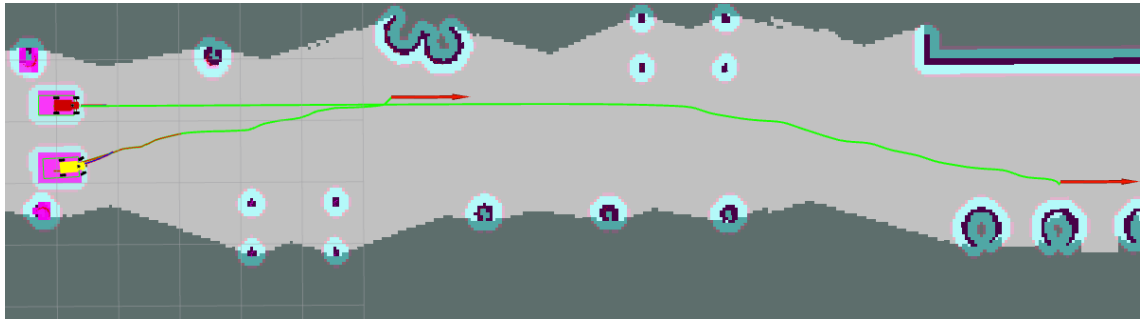
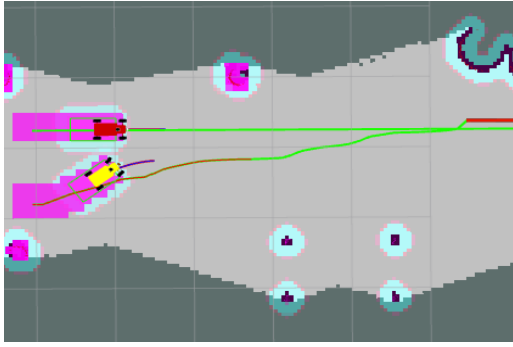
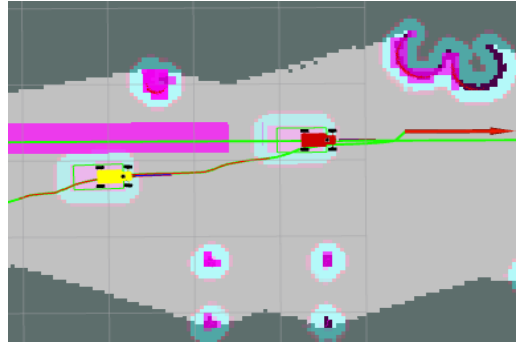


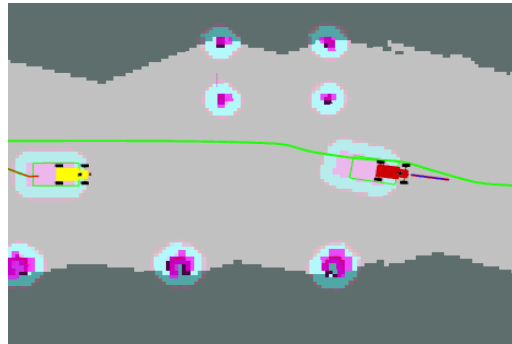
Figure 6.33: Test 2: First Desired Goals' Positions for both AVs (RViz)



(a) AV 2 Changing its Lane and AV 1 Keeping its lane (RViz)



(b) The 2 AVs are Keeping their Lanes (RViz)



(c) AV 2 Arrived Destination while AV 1 is Changing its lane (RViz)

Figure 6.34: Test 2: First Goal Maneuvers

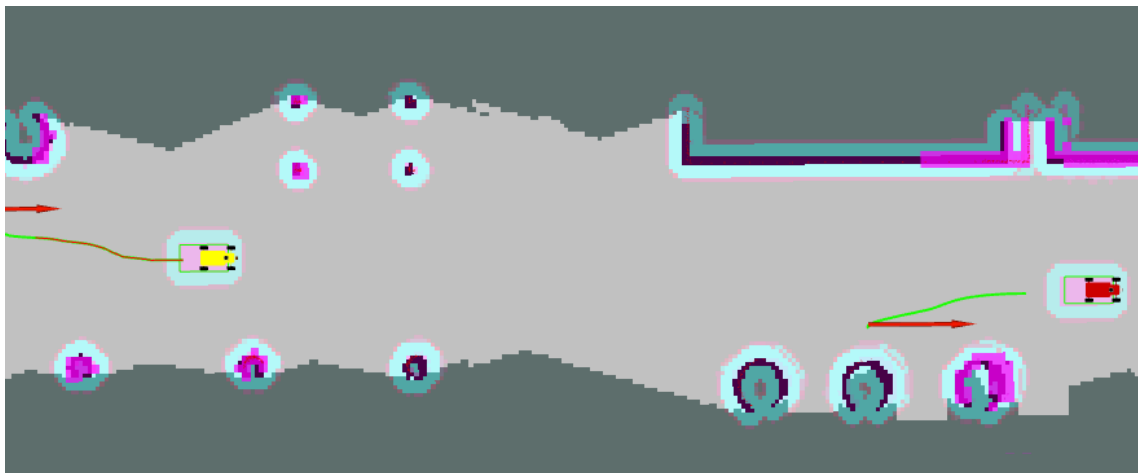


Figure 6.35: Test 2: First Goal Reached by both AVs (RViz)

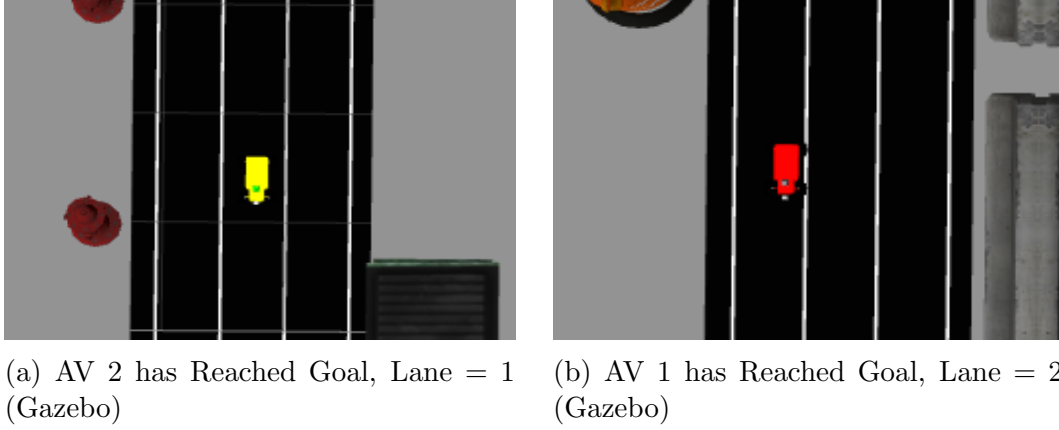


Figure 6.36: Test 2: First Goal Reached by both AVs, yellow AV's lane = 1, and red AV's lane = 2 (Gazebo)

Second Goal: The goal of the yellow AV is sent far earlier than the goal of the red AV. Therefore, the AV kept its lane until it became behind the red AV, and started decelerating in Figures 6.37 and 6.38a. After that, the red AV could safely change its lane in figures 6.38a, then the yellow AV can finally change its lane as well just after the red AV, in figure 6.38c.

Partially checked: 1, and 2.

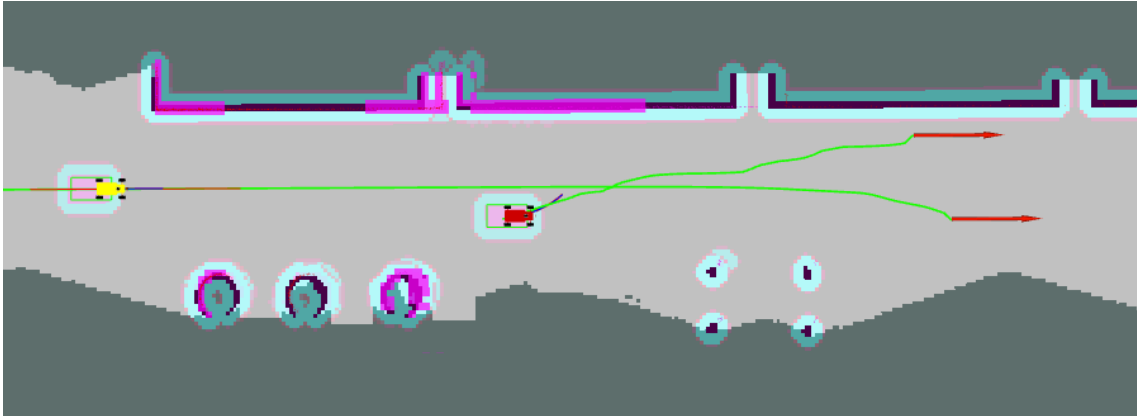


Figure 6.37: Test 2: Second Goal Positions (RViz)

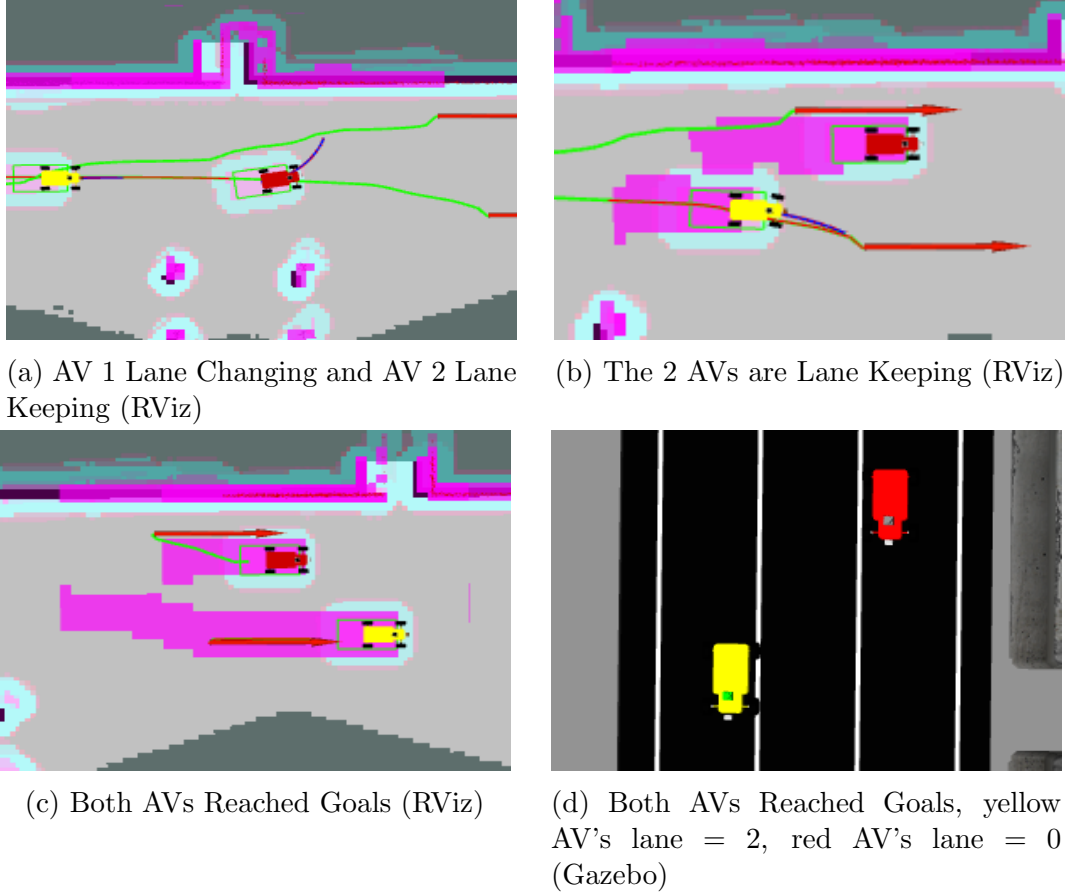


Figure 6.38: Test 2: Second Goal

The previous test cases indicated that the navigation module is integrated well, and most of the checks studied in the beginning of this section was true. However, working on the path planning and enhancing the goal tolerance will indeed make the system better.

6.1.8 External Commands Following

In order to test the external commands following module explained in 5.1.8, we coded a simple client that generates 10 random actions (Last one is flag) and send them as requests to the external commands following module.

- **Scenario:**

- The AV's starting position is at $(0, -0.750)$ in lane number 2.
- The AV's velocity range is $(0, 0.4)m/s$.
- The AV's maximum acceleration is $0.023m/s^2$.
- The required time in lane keeping and accelerate actions is $20sec$.
- Because of the AV's velocity and acceleration ranges, the AV cannot accelerate or decelerate more than $17.39sec$.
- Lane changing occurs in its own designed duration depending on its velocity as explained in section 5.1.5.

The 8 randomly generated actions requested are:

Order	Action Type	Acceleration m/s^2 .
1	Accelerate	0.023
2	Right Lane Changing	0
3	Lane Keeping	0
4	Left Lane Changing	0
5	Right Lane Changing	0
6	Left Lane Changing	0
7	Decelerate	-0.023
8	Right Lane Changing	0
9	Lane Keeping	0

Table 6.18: External Commands Following - Test: Requested Actions

- **Results and Discussion:** The AV's starting position is at (22.91,-0.8672) in lane number 2. Table 6.19 documents the returned responses from the external commands following module (server) to the simple client that generated 9 random requests.

Actions Analysis:

- **First action:** The time taken to accelerate from 0 to 0.4 m/s was 17.60 sec, this is close to the 17.39 sec with 0.21 error. The error comes from the processing time between sending a goal and receiving the result. Hence, the AV cannot continue an infeasible acceleration as explained in velocity controlling server algorithm 7.
- **Second action:** Lane changing was infeasible as the AV was in the outermost right lane. Therefore, the AV sent that it is an infeasible goal to the client in order to choose another action as in algorithm 8
- **Third action:** The AV could successfully maintain its maximum velocity for 20 seconds.
- **Fourth action to sixth action:** The AV changed its lane in fast as it preserved its velocity after actions 1 and 3. However, its velocity decreasing during lane changing.
- **Seventh action:** The AV decelerated in 14.17 sec instead of 17.39 as it lost momentum in lane changing.
- **Eighth and ninth Action:** The AV stopped after lane keeping for 20 sec indicating that the external commands following module got deactivated.

Refer to Figure 6.39 for visualizations

Order	Feasibility	Time Taken (sec)
1	Partially Feasible	17.60
2	Infeasible	0
3	Feasible	20
4	Feasible	4.862
5	Feasible	5.155
6	Feasible	5.613
7	Partially Feasible	14.17
8	Feasible	16.60
9	Feasible	20

Table 6.19: External Commands Following - Test: Responses

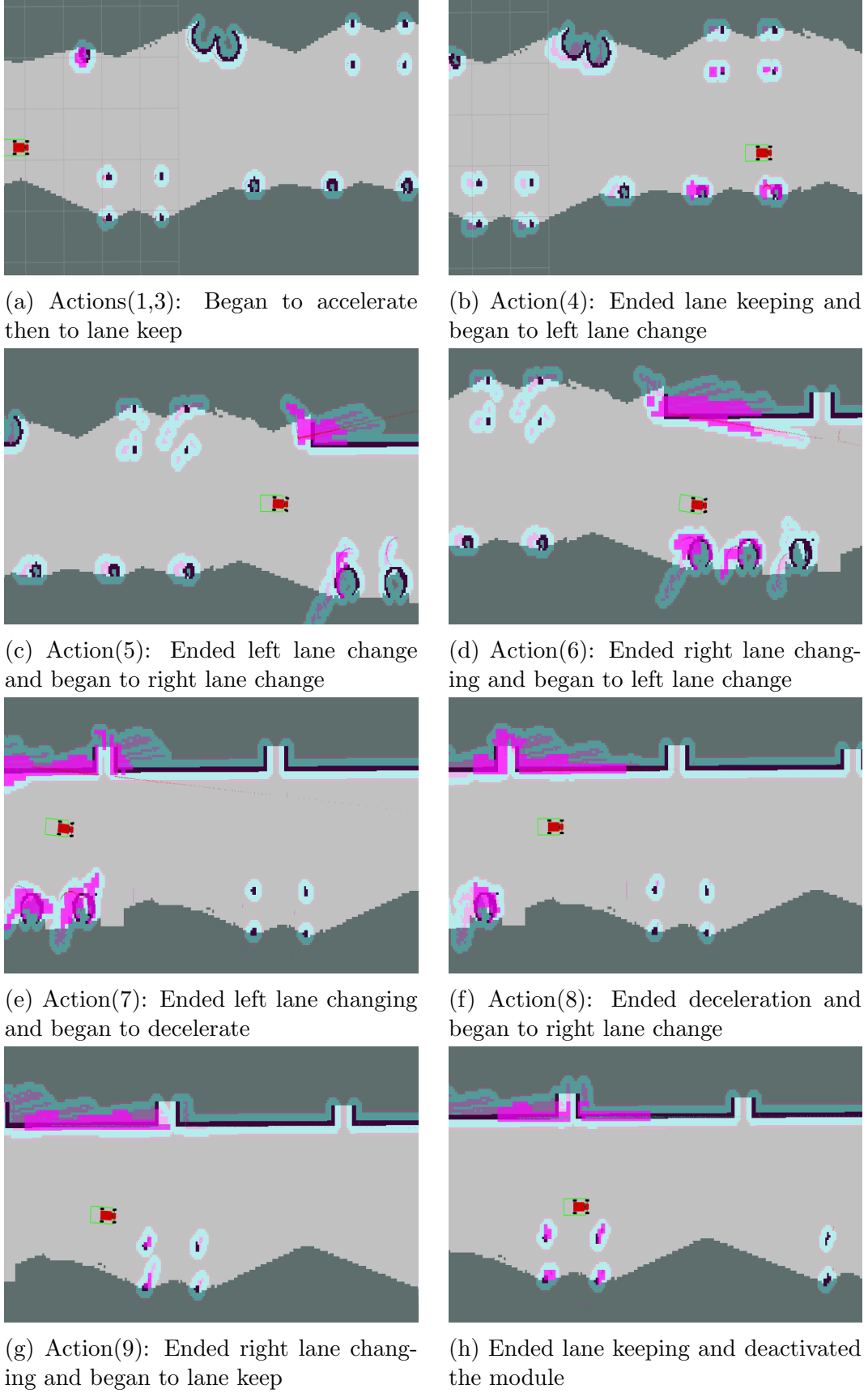


Figure 6.39: Test 2: External Commands Following - 9 randomly generated actions

6.1.9 High-Level Control Master

The main task of the high-level control master, explained in 5.1.9, is to activate and deactivate both: the navigation module and the external commands following module. Hence, in order to test the module, we used the same client that generates random actions as in section 6.1.8. We designed two tests that aim to verify the following: (both checks are included in all tests):

- The navigation gets deactivated once an external command is requested.
- The AV follows external commands and ignores the path generated by the navigation module.

Checks that are related to each test on its own:

1. **Test 1:**

- When the external commands module gets deactivated and the AV has already reached the goal, it stops. In other words, it doesn't try to go to the other lane's goal.

2. **Test 2:**

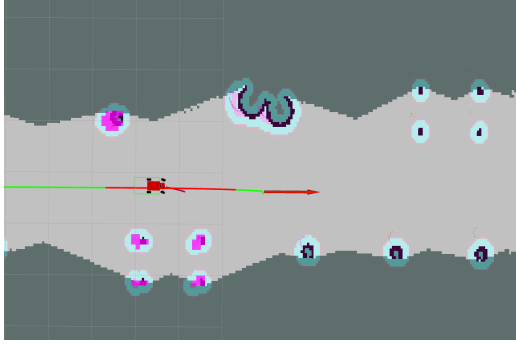
- If the goal is still not reached, after the re-activation of the navigation module, the AV should continue until it reaches it.

- **Test 1:**

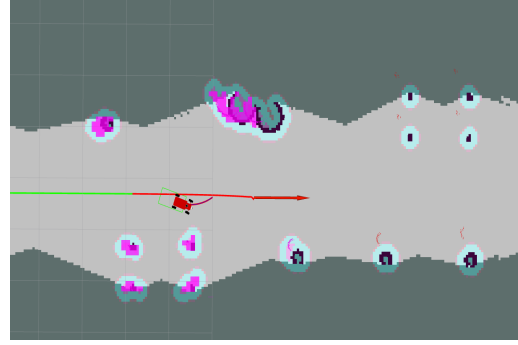
- **Scenario:**

1. The navigation module sends a near goal to the AV.
2. After a couple of seconds, the external commands following requests an action from the AV in the following sequence:
 - (a) Right lane change (Feasible)
 - (b) Right lane change (Infeasible)
 - (c) Left lane change (Feasible)
 - (d) Left lane change (Feasible)
 - (e) Left lane change (Infeasible)
 - (f) A flag to deactivate the external commands following module
3. The AV misses its goal during following external commands.

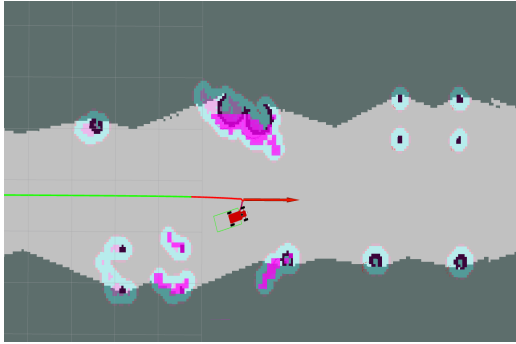
- **Results and discussion:**



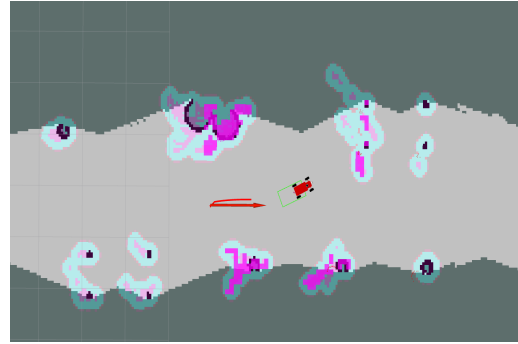
(a) The AV is following the path generated by the navigation module to reach its goal



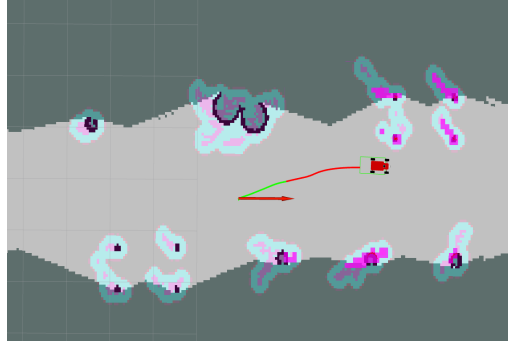
(b) The AV starts following an external command (right lane changing) and ignored the navigation's goal



(c) The AV is following the second external command (left lane changing) and is still ignoring the navigation's goal



(d) The AV is following the third external command (left lane changing) and is still ignoring the navigation's goal



(e) The AV followed the three feasible external commands then stopped as it missed its goal

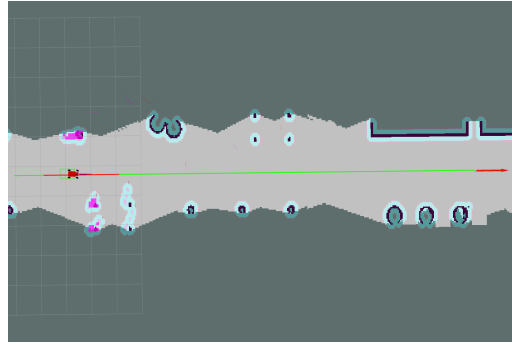
Figure 6.40: Test 1: High-Level Control Master

- **Test 2:**

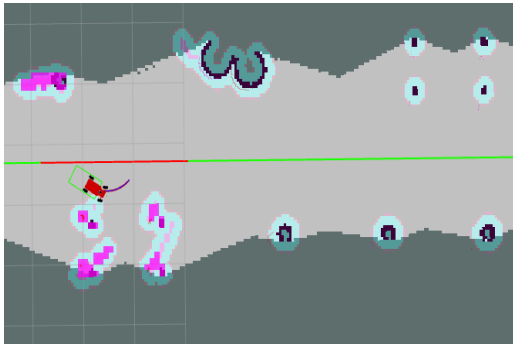
- **Scenario:**

1. The navigation module sends a far goal to the AV.
2. After a couple of seconds, the external commands following requests actions from the AV in the same sequence as in the first test.

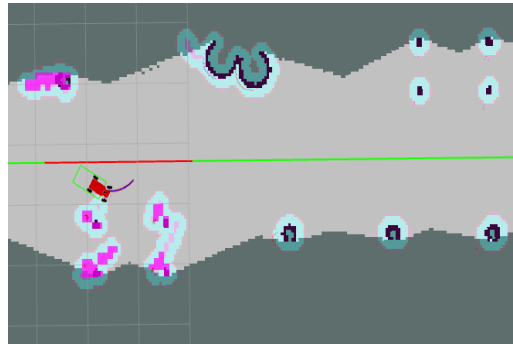
- **Results and discussion:**



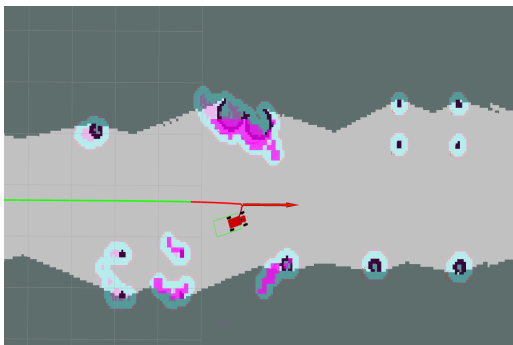
(a) The AV is following a goal generated by the navigation module



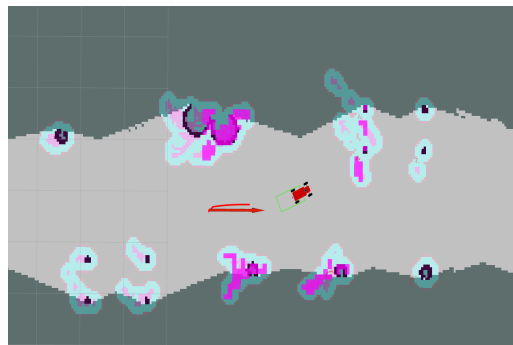
(b) The AV starts following an external command (right lane changing) and ignored the navigation's goal



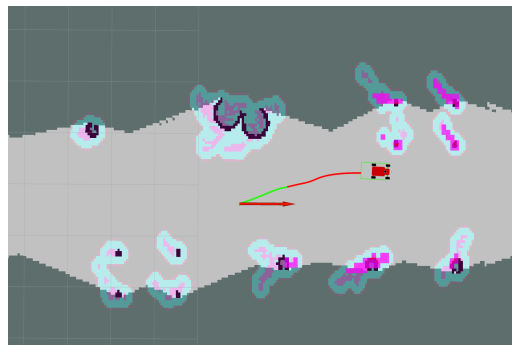
(c) The AV is following the second external command (left lane changing) and is still ignoring the navigation's goal



(d) The AV is following the third external command (left lane changing) and is still ignoring the navigation's goal



(e) External commands following module is deactivated and navigation is activated, the AV is right lane changing to follow the global path



(f) The AV reached the navigation goal

Figure 6.41: Test 2: High-Level Control Master

According to figures 6.40 and 6.41, all checks are verified.

6.2 V2V Communication

In order to verify that the communication has been properly established between vehicles, using our implementation previously discussed in 5.2, two tests were designed. In the first test, the track has four vehicles which are all within the communication ranges of one another. The second test is exactly the same as the first one except that the vehicles have a shorter communication range. This reduction results in some vehicles being beyond the communication ranges of some others. For each of the test cases, the list of IDs received by each of the four vehicles is recorded. The expected outcome in either of the tests is that each vehicle's list has the IDs of only the others which are within its communication range. To avoid the hassle of displaying too much redundant data, we concern ourselves with checking the list of IDs received by only one “ego” vehicle.

Figure 6.42 depicts the position of the vehicles in the track (same for both tests). For demonstration purposes, two of the AVs were assigned the type “racecar”, shown in red, while the two others were assigned the type “ambulance”, shown in yellow. With all values in metric units, the true IDs of the four vehicles are as follows:

- Vehicle 1 (right-most red vehicle, **ego** vehicle):
 {**vehicle num:** 1
 type: "racecar"
 x position: 26
 y position: - 0.2625
 lane num: 1
 footprint: [[25.72,-0.4125], [26.28,-0.4125], [26.28,-0.1125], [25.72,-0.1125]]
 velocity: 0
 max vel: 0.4
 max acc: 0.15}
- Vehicle 2 (right-most yellow vehicle):
 {**vehicle num:** 2
 type: "ambulance"
 x position: 27
 y position: 0.2625
 lane num: 0
 footprint: [[26.72,0.1125], [27.28,0.1125], [27.28,0.4125], [26.72, 0.4125]]
 velocity: 0
 max vel: 0.2
 max acc: 0.1}
- Vehicle 3 (left-most yellow vehicle):
 {**vehicle num:** 3
 type: "ambulance"
 x position: 24.5
 y position: - 0.2625
 lane num: 1

footprint: [[24.22,-0.4125], [24.78,-0.4125], [24.78,-0.1125], [24.22,-0.1125]]
velocity: 0
max vel: 0.5
max acc: 0.1}

- Vehicle 4 (left-most red vehicle):

{**vehicle num:** 4
type: "racecar"
x position: 24
y position: - 0.7875
lane num: 2
footprint: [[23.72,-0.9375], [24.28,-0.9375], [24.28,-0.6375], [23.72,-0.6375]]
velocity: 0
max vel: 0.3
max acc: 0.2}

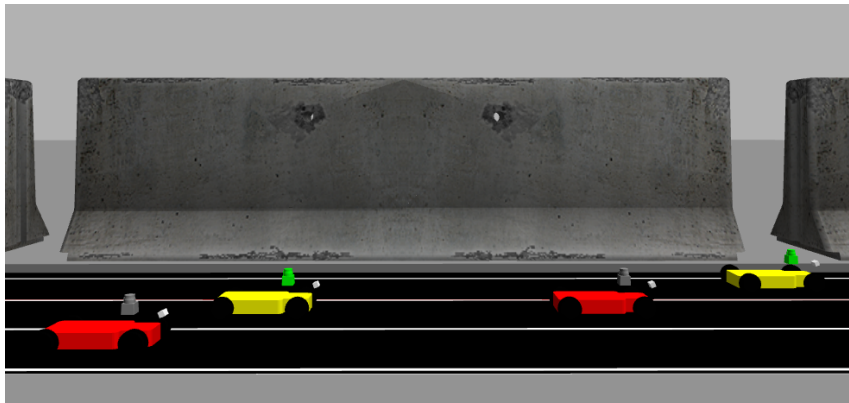


Figure 6.42: V2V Communication Test: Relative position of test vehicles in the track.

1. Test 1:

- **Scenario:**

- Vehicles are in their positions shown in 6.42.
- Communication range: 10 meters

- **Results and discussion:**

With all values in metric units, the list of IDs received by our ego vehicle is as follows:

```
[{vehicle num: 2  
  type: "ambulance"  
  x position: 26.9999370575  
  y position: 0.262505710125  
  lane num: 0  
  footprint: [[26.7199370575,0.1125057101], [27.2799370575,0.1125057101],  
  [27.2799370575,0.4125057101], [26.7199370575, 0.4125057101]]  
  velocity: 2.09938647799e-07
```

```
max vel: 0.20000000298
max acc: 0.10000000149},

{vehicle num: 3
 type: "ambulance"
 x position: 24.4999885559
 y position: -0.262500733137
 lane num: 1
 footprint: [[24.12123115115,-0.4125051555], [24.7784544485,-0.4125051555],
 [24.7784544485,-0.1125015555], [24.1223115115,-0.1125015555]]
 velocity: 1.70507689745e-07
 max vel: 0.50000000011
 max acc: 0.10000000149},

{vehicle num: 4
 type: "racecar"
 x position: 23.9999885559
 y position: -0.787500739098
 lane num: 2
 footprint: [[23.7199974111,-0.9375001255], [24.28,-0.9375001255],
 [24.28,-0.6375008511], [23.7199974111,-0.6375008511]]
 velocity: 1.63941436426e-07
 max vel: 0.300000011921
 max acc: 0.20000000298}]
```

Our ego vehicle received IDs from all the other three vehicles, and those IDs were equivalent to the vehicles' true ones. This matches the expected outcome of the test since all the other three vehicles were truly within our ego vehicle's communication range.

2. Test 2:

- **Scenario:**

- Vehicles are in their positions shown in 6.42.
- Communication range: 1.5 meters

- **Results and discussion:**

With all values in metric units, the list of IDs received by our ego vehicle is as follows:

```
[{vehicle num: 2
 type: "ambulance"
 x position: 26.9999980927
 y position: 0.262499958277
 lane num: 0
 footprint: [[26.71988970225,0.1125003101], [27.2799370575,0.1125003101],
 [27.2799370575,0.4125066601], [26.71988970225, 0.4125066601]]
 velocity: 7.3141836765e-06
 max vel: 0.20000000298
 max acc: 0.10000000149},
```

```
{vehicle num: 3
  type: "ambulance"
  x position: 24.4999980927
  y position: -0.262500017881
  lane num: 1
  footprint: [[24.12123118845,-0.4125087566], [24.7784884855,-0.4125087566],
[24.7784884855,-0.1125026569], [24.12123118845,-0.1125026569]]
  velocity: 7.31403542886e-06
  max vel: 0.50000000011
  max acc: 0.10000000149}}
```

Our ego vehicle received IDs from vehicle number 2 and 3 only, and the IDs were equivalent to the respective vehicles' true ones. Since the communication range was reduced from 10 meters in test 1 to 1.5 meters here, the results of the test match the expected outcome. This is because the true distance between our ego vehicle and the other three vehicles are 1.129, 1.5 and 2.068 meters respectively. Thus, only vehicles number 2 and 3 are within our vehicle's communication range.

6.3 Cooperation Technique

In this section we illustrate the effect of using our RL agents, whose implementation was thoroughly discussed in 5.3, on our main metric: EV Travel Time, and highlight key insights from our SUMO experiments that guided our current design and shall guide our later research.

6.3.1 Single Agent with Q-learning in a Single-Agent Environment

We have trained our agent with Q-learning algorithm according to problem formulation structure mentioned in Section 5.3.1 using 50,000 episodes. Then, we have tested it for 50,000 other episodes, in an environment that contains both the EV and the agent only. The EV travel time was always optimal i.e. EV has managed to travel with its maximum velocity all the way during its trip. Obviously, the agent has taken all necessary actions to evacuate road for it. However, the agent has experienced an oscillation which was not a good indication of safe driving behavior.

To illustrate the cause of oscillating behaviour, we will use the following scenario-specific case. If the road is divided into three lanes and the EV is driving in the first lane, then the agent could either drive in the second or third lane without blocking the EV's path. According to our first design of reward function, the reward was only a function of EV acceleration. As a result, the agent was taking any action as long as it does not decrease the EV acceleration.

To overcome oscillation behaviour, we have applied the following solutions :

1. We know that oscillation behaviour happens when the agent drives using our

RL model instead of car following model (its basic autonomous functionalities). RL model is basically used to allow cooperation between agent and the EV. If EV is outside agent window, it makes sense to disable RL model and let agent drive with its autonomous functionalities. Hence, the agent will not experience an oscillating behaviour as long as the EV is outside agent window and RL model is disabled. RL engagement(when EV is inside agent window) and disengagement(when EV is outside agent window) was a partial solution that decreases oscillating behaviour but it does not totally eliminate it. Since now the oscillatory behavior was absent when the EV was outside the agent window, and still present when the EV was inside it.

2. The second solution was to edit the reward to be a weighted function of the EV acceleration (larger weight) and agent actions (less weight). The fact that the EV acceleration has the larger weight makes the agent's first priority to evacuate the road for it. Every agent action has a reward associated with it. We have ordered agent actions from highest reward to lowest reward as the following :

- (a) Accelerate
- (b) Change Lane
- (c) Continue as it is
- (d) Decelerate

This order has made the agent always eager to increase its speed and not change lanes frequently unless lane change will enable it to increase its speed. Subsequently, the agent oscillating behaviour has totally disappeared while the EV travel time was still optimal.

6.3.2 Single Agent with Q-learning in a Multi-agent Environment

Our single-agent model (model trained with single agent and emergency vehicle only on the field) was put to test in an environment where other vehicles (other than the emergency vehicle and single-agent vehicle itself) were deployed. Results in terms of the model's performance on the main metric: the emergency vehicle travel time are retrieved and discussed.

6.3.2.1 Experiment Setup

Our experiments aimed at measuring two main aspects of the effect of the model, keeping the emergency vehicle travel time as our main observation: what its effect is on the travel time as the busyness increased, and what its effect is if not all the vehicles have the RL model (Q_LEARNING_SINGLE_AGENT, from 4.1.1.2) deployed, but only a percentage of them.

To measure this, we define experiments with a set of controllable parameters and one observation: EV Travel Time. The controllable parameters are:

1. Lane Busyness (0 to 1). At 0%, there are no vehicles on the field; at 100%, the maximum number of vehicles that each lane could contain is deployed to the field.
2. Percentage of RL vehicles(0 to 1). With 0% meaning that all vehicles present are autonomous vehicles, and 100% meaning that all vehicles present have the model (Q_LEARNING_SINGLE_AGENT) deployed. This does not have to do with how many vehicles are actually there on the field.

6.3.2.2 Averaging and Randomness

You can skip this point and go to the next subsection, this part mentions some technical details about the simulation process.

During training and testing in Section 6.3.1, the starting position for the agent vehicle and emergency start lane (0,1, or 2) -both- were randomized, to help the agent generalize. During testing in this section, only emergency vehicles start lane was chosen randomly each episode, and agents start position was kept constant.¹, this was done as the starting position was irrelevant to our RL model, and was only included in the training to help it generalize.

To implement the idea of lane busyness, a random float from 0 to 1 was generated a number of times= the maximum number of possible AVs, and when that number was less than the lane busyness, an AV was placed, else, no AV was placed.

The RL percentage was implemented in a similar manner to the lane busyness, but the random number decided whether a placed vehicle is an RL vehicle, or not, instead.

Because of this randomness, each observation in our table has been averaged over 100 runs of the experiment at otherwise constant parameters.

6.3.2.3 Important Note Before Results

In order for the reader to have an eye for the results, some information is in order. The ideal travel time of the emergency vehicle is 50 time steps (500 cells at its maximum velocity = 10 cells/sec). Presence of other vehicles (i.e. lane busyness > 0%) typically results in the delay of the emergency vehicle (i.e. time > 50 time steps). Improvement can be seen as reducing this travel time to be below what other (non-RL) autonomous vehicles would have achieved, at the same lane busyness.

6.3.2.4 Results

1. Model Performance Against an All Autonomous Vehicles Model, as Lane Busyness Increases

The two models described in 4.1.1.2 are put in comparison, while sweeping over the possible value of the lane busyness (0 to 1), with a step of 0.1. Results are shown in Figure 6.43. Some key insights and explanations to the figure are listed below:

¹Kept as the middle point between the closest-to-emergency possible starting position (40), and furthest-from-emergency possible starting position (250), to equal 149.

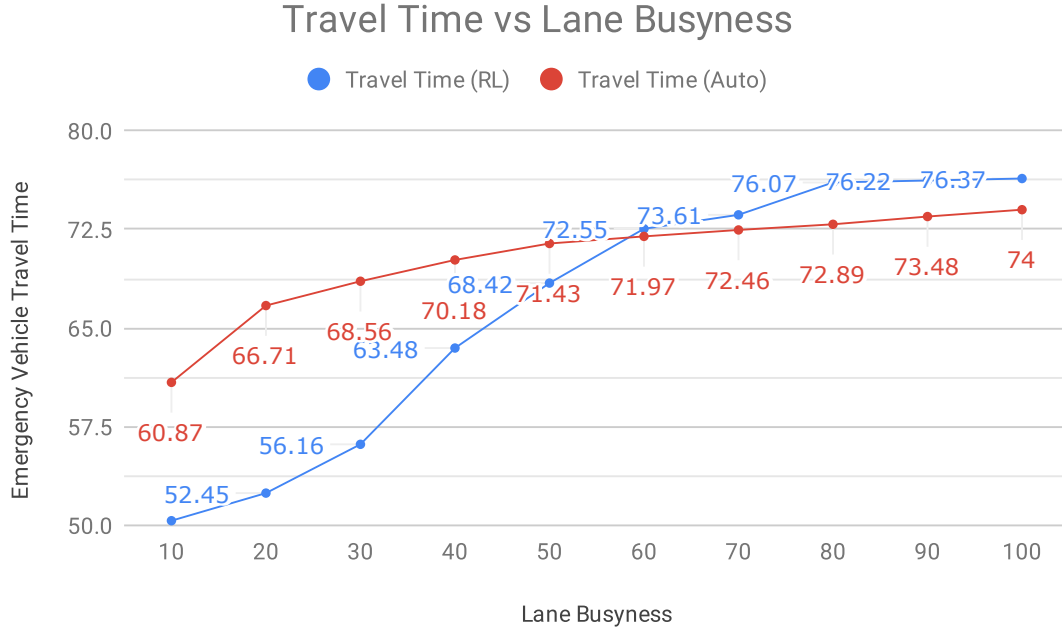


Figure 6.43: Travel Time vs Lane Busyness, comparing between two cases: RL (all vehicles on field have the model Q_LEARNING_SINGLE_AGENT) deployed, and Auto(all vehicles on field have the model SUMO_KRAUSS deployed). Both models described in 4.1.1.2.

- We see clear improvements upon using our model in the EV travel time at low lane busyness's, meeting the SUMO_KRAUSS model at around 60% busyness. After which the model almost does not improve the results, and actually results in minimal extra delay.
- Abstractly, this is because our approximation of the agent being the only entity responsible for the EV delay² works fine at low busyness's. At higher busyness's, however, the model is unable to change lane because of the absence of lateral free space, nor perform any action to help the EV accelerate because almost all actions it learned to choose are infeasible (mostly lane changing actions).
- Taking further looks at the experiment at higher busyness's, it is evident that the our model vehicle gets trapped trying to perform what it was trained to do: change lane to evacuate the EV's lane. However, because that is not possible and (other times) that is not the solution, it can not do this.
- Furthermore, what makes the performance even worse later is that it has entered unexplored territory: cases that it never sampled during the learning process³. In such cases, the agent performs random actions (one of which is slowing down), resulting in the extra delay of the EV.

²This approximation is implied by penalizing the agent (check problem formulation) whenever the EV does not used maximum acceleration.

³An example case is: the agent being far in front of the EV, but the EV still not having maximum velocity. This is a case that has not been sampled earlier during the learning. Another example is when the agent is just in front of the EV, on its lane, but another vehicle is blocking

- (e) A key insight due to improvement in lane busyness's: the problem of the lane evacuation is a big problem that communication along with even a simple solutions like this one can result in great improvements at. At 30% lane busyness for example, only 6.16 time steps (12.3% of the ideal 50 steps) delay were incurred by the EV when using the RL model, compared to 18.56 steps (37.1% of the ideal 50 steps) delay incurred by the EV when using the non-communicating⁴ autonomous vehicles.
- (f) Another key insight is that there is still free space to be utilized at high lane busyness's, with the only problem being other AVs blocking any helpful action a single AV could take. Following up with this idea, we believe an independent learner can learn to improve the performance even more, only if trained in presence of other independent learners/vehicles, as another simple solution that could arise would be for agents not on the EV's lane to slow down, allowing for agent's on other lanes to change lane easily.

2. Model Performance for Different Lane Busyness's, as RL Percentage Increases

In the last experiment, either all the vehicles used SUMO_KRAUSS model, or all of them used Q_LEARNING_SINGLE_AGENT model. In this experiment, start from the former cases (all SUMO_KRAUSS), and increase the percentage of vehicles deploying the Q_LEARNING_SINGLE_AGENT from 0 to 1, with a step of 0.1, to demonstrate the effect of having only some AVs implementing our model. Results are shown in Figure 6.44.

Inspecting Figure 6.44, we notice that increasing the percentage of AVs using our model typically monotonically decreases the vehicle travel time. This means that not all vehicles need to be RL vehicles for the system to produce beneficial results, but also that the true benefit of the model appears when more AVs deploy it.

6.4 System Integration

In this sub-section, we test that our implementation of the single-agent problem on Gazebo, ROS, based on the architecture discussed in 5.4 allows for a smooth, accurate training of the RL model on one AV which communicates and cooperates with an EV as appropriate.

To properly test our architecture, we need to verify the following:

1. The simulation is started, reset and closed successfully.
2. In case Gazebo dies, the system simulation does not stop and is able to successfully handle the situation.
3. The environment is able to get the agent's states accurately.

the lane change so it is infeasible. In the later case, the agent was seen to select random actions, of which one is slowing down.

⁴i.e. are not aware of the presence of the EV.

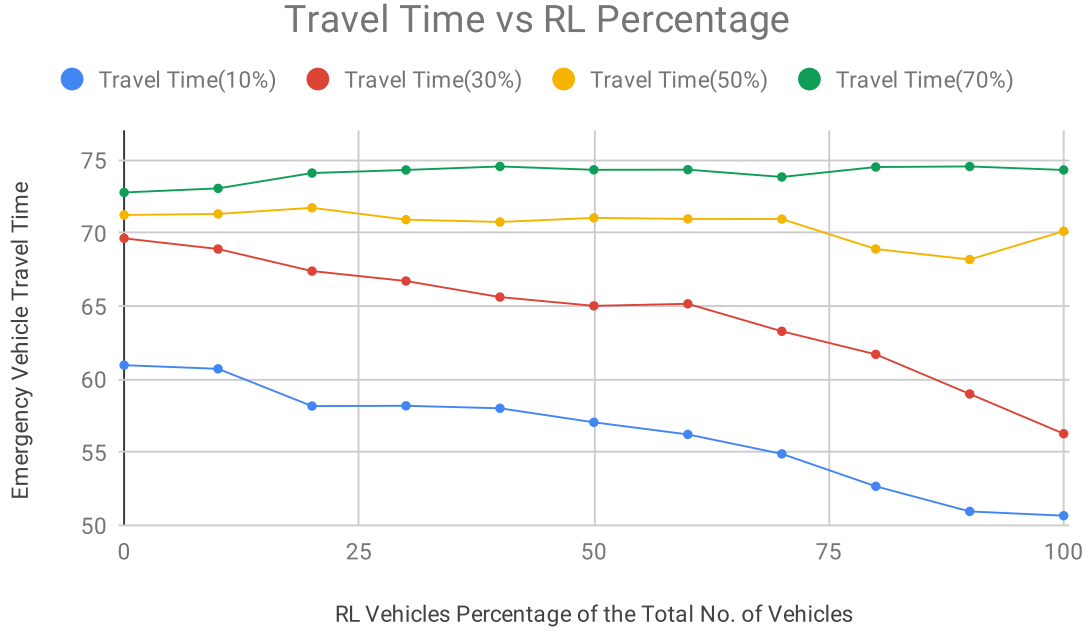


Figure 6.44: Travel Time vs RL Percentage for different Lane Busyness's.

4. The environment is able to calculate the agent's reward based on its chosen action's execution time.
5. The environment is able to detect when an episode should be terminated.
6. The environment is able to detect when the EV is inside/outside the agent's designed window.
7. Following the RL model is deactivated when the EV is outside the agent's window, and reactivated when it gets inside it.
8. The agent is not prompted to execute an infeasible action. In such case, a different action is chosen instead.

To perform the required verifications, the RL model was allowed to be trained for a few episodes in the Gazebo simulation environment. Some chosen parameters were as follows:

- The dimensions of the agent's window were set to $[-24, 9]$ meters relative to the agent's position.
- The maximum step reward is 0.
- Actions are always taken via exploration; since these are the first couple of episodes.

The results associated with the significant actions taken by the AV in each of the first 3 episodes were then found to be:

1. Episode 1:

- **Results:**
 - **Episode Termination Reason:** ambulance reached its goal
 - **Number of Executed Actions:** 9
 - **Final State:**
 - (a) Agent Velocity: 0.144442
 - (b) Agent Lane: 1
 - (c) EV Velocity: 0.998572
 - (d) EV Lane: 2
 - (e) Relative Position: 17.038090
 - **Initial State:**
 - (a) Agent Velocity: 0.417382
 - (b) Agent Lane: 0
 - (c) EV Velocity: 0.967884
 - (d) EV Lane: 2
 - (e) Relative Position: -23.068913

Requested Action	State Before	State After	Reward
1- Change left was infeasible, then it chose decelerate	Agent Velocity: 0.417382, Agent Lane: 0, EV Velocity: 0.967884, EV Lane: 2, Relative Position: -23.068913	Agent Velocity: 0.33806, Agent Lane: 0, EV Velocity: 0.999919, EV Lane: 2, Relative Position: -19.848728	-0.004744 in 5 sec
2- No acceleration	Agent Velocity: 0.333763, Agent Lane: 0, EV Velocity: 0.999919, EV Lane: 2, Relative Position: -19.864744	Agent Velocity: 0.336215, Agent Lane: 0, EV Velocity: 0.999720, EV Lane: 2, Relative Position: -16.504776	0 in 5 sec
3- Change left was infeasible, then it chose no acceleration	Agent Velocity: 0.336215, Agent Lane: 0, EV Velocity: 0.999736, EV Lane: 2, Relative Position: -16.485779	Agent Velocity: 0.336477, Agent Lane: 0, EV Velocity: 0.999372, EV Lane: 2, Relative Position: -13.077777	0 in 5 sec
4- Change left was infeasible, then it chose change right	Agent Velocity: 0.336477, Agent Lane: 0, EV Velocity: 0.999372, EV Lane: 2, Relative Position: -13.077777	Agent Velocity: 0.334250, Agent Lane: 1, EV Velocity: 0.999839, EV Lane: 2, Relative Position: -9.206767	0 in 5.7 sec

Table 6.20: TITLE: Episode 1 - Results

- **Discussion:**

- It can be noticed in Table 6.20 that all actions took place while the relative position between the EV and the agent is greater than -23 and less than -9.3; this validates that the RL model was followed only when the EV is inside the agent's window.
- In actions 1, 3 and 4, the first chosen action was lane changing to the left. It was then found to be infeasible (which is true since the agent is already in the left-most lane of the track) and another feasible action was chosen and executed.
- The transition between the values of the states is reasonable relative to the taken action.
- The state after taking an action in one time step is close to the state before taking an action in the following time step; which means that

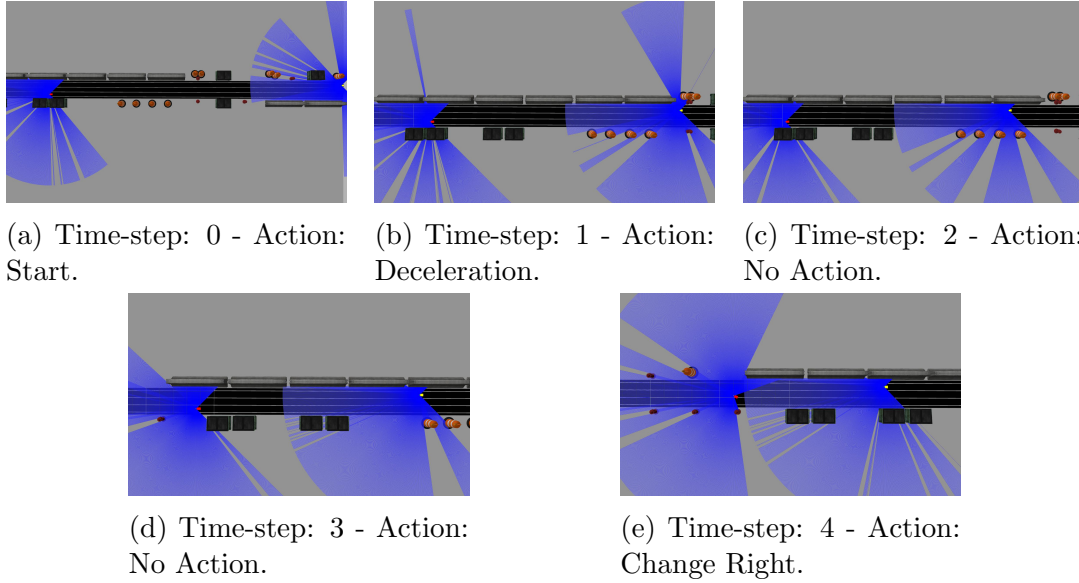


Figure 6.45: First four time-steps in episode 1.

the designed integrated system architecture takes care of synchronization.

- The rewards of actions 2, 3 and 4 are equivalent to the maximum step reward; which is as expected since the velocity of the EV before taking the actions was already maximum and remained so after the actions execution. The reward of action 1 is very close to that of maximum step reward; which is reasonable since the EV performed maximum acceleration.

2. Episode 2:

- **Results: Episode Termination Reason:** simulation died
- **Discussion:** Although the Gazebo simulator crashed, the system did not freeze. Instead, the episode was forced to terminate and the simulation was prompted to restart.

3. Episode 3:

- **Results:**
 - **Episode Termination Reason:** agent reached its goal
 - **Number of Executed Actions:** 9
 - **Final State:**
 - (a) Agent Velocity: 0.015078
 - (b) Agent Lane: 1
 - (c) EV Velocity: 0.102960
 - (d) EV Lane: 1
 - (e) Relative Position: -0.795270
 - **Initial State:**
 - (a) Agent Velocity: 0.416254

- (b) Agent Lane: 1
- (c) EV Velocity: 0.996863
- (d) EV Lane: 1
- (e) Relative Position: -22.961435

Requested Action	State Before	States After	Reward
1- Decelerate	Agent Velocity: 0.249905, Agent Lane: 1, EV Velocity: 1.000747, EV Lane: 1, Relative Position: -16.362143	Agent Velocity: 0.149595, Agent Lane: 1, EV Velocity: 0.919836, EV Lane: 1, Relative Position: -12.401335	-0.584236 in 5 sec
2- No acceleration	Agent Velocity: 0.149595, Agent Lane: 1, EV Velocity: 0.919836, EV Lane: 1, Relative Position: -12.4013354	Agent Velocity: 0.149030, Agent Lane: 1, EV Velocity: 0.778528, EV Lane: 1, Relative Position: -8.794250	-0.655530 in 5 sec
3- Decelerate	Agent Velocity: 0.149030, Agent Lane: 1, EV Velocity: 0.778528, EV Lane: 1, Relative Position: -8.794250	Agent Velocity: 0.048788, Agent Lane: 1, EV Velocity: 0.600992, EV Lane: 1, Relative Position: -5.818396	-0.790863 in 5 sec
4- Decelerate	Agent Velocity: 0.048788, Agent Lane: 1, EV Velocity: 0.601142, EV Lane: 1, Relative Position: -5.788945	Agent Velocity: 0.015384, Agent Lane: 1, EV Velocity: 0.563688, EV Lane: 1, Relative Position: -4.905980	-0.861758 in 5 sec

Table 6.21: Episode 3 results

- **Discussion:**

- Apart from comments similar to those in 1, it is clear in Table 6.21 that when the agent was preceding the EV in the same lane, the EV has to reduce its velocity in order to keep a safe distance between the agent and itself. As a result of such a decrease in EV velocity, the agent was given negative reward.

4. General Discussion:

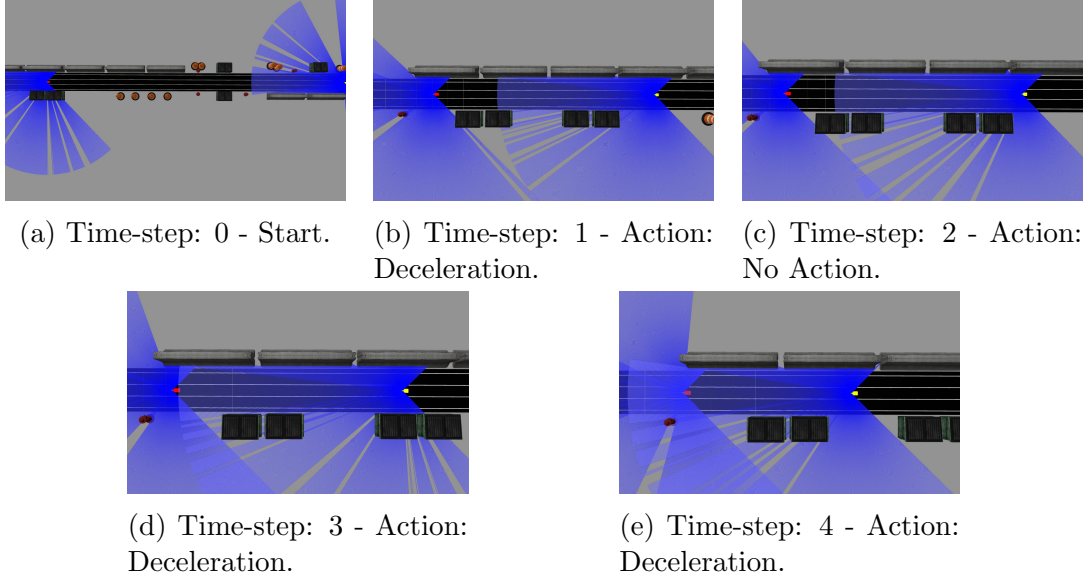


Figure 6.46: First four time-steps in episode 3.

- Between episodes, the simulation was smoothly restarted and the vehicles' initial positions and final goals were reset correctly.
- Although the different system components were tested and found to be working, one important aspect to further test before fully training the RL model is the effect of the reward normalization based on action execution times. Thorough testing is needed in order to make sure that no undesirable behaviors accordingly arise.

Chapter 7

Conclusion and Future Work

This chapter concludes by commenting on our results, illustrates our main contributions, and mentions our future plans.

7.1 Conclusion

Our work throughout the past year, as illustrated in this document, demonstrates the benefits of using reinforcement learning alongside autonomous functionalities to reduce the travel time delay of approaching emergency vehicles in a multi-agent environment where vehicles are allowed to communicate. Autonomous functionalities were implemented on ROS, traffic simulations were implemented on SUMO. From our experiments on both platforms, we conclude that using RL on top of autonomous functionalities¹ is a practically-feasible and safe approach that can be used to solve our problem. Further (more complex) solutions could be implemented, however, in this work, the problem was dragged from the ideas realm to the practically quantifiable realm, with demonstrated benefits. We hope that building on this work, other researchers will be able to apply the same idea to more complex tasks through our developed modular environments and simplified interfaces.

7.2 Main Contributions

Our main contributions to the Zewail City and scientific communities are:

1. A modular playground for developing Communicating Autonomous Vehicles on Gazebo, ROS, with a fairly accessible interface.
2. A practical implementation and problem formulation of the Emergency Vehicle delay problem.
3. A Multi-Agent Reinforcement Learning environment on SUMO, and a benchmark problem for MARL algorithms.
4. An extendable Realistic Platform for Training and Testing RL Models on Autonomous Vehicles in Gazebo, ROS

¹(as opposed to: using RL for autonomous functionalities)

7.3 Future Work

Our goals for the future of this project are:

1. Despite not taking a great space inside this document, big part of our work pre-COVID-19 was dedicated to implementing the project on actual hardware. We had all the hardware delivered from USA, the body 3D-printed, and everything ready for work. We hope that (as soon as we are able to get back to our labs) we can either work on the hardware ourselves or help younger students implement the project on it. We believe it will be of great benefit to have a toy autonomous car in the lab, as it will facilitate participating in future competitions for students from Zewail City (including F1/10 [9] itself).
2. Continue RL research with Dr. Elshenawy on the same problem, implementing a more complex Multi-Agent model, capable of detecting its surrounding vehicles.
3. Work with students from younger batches on enhancing the autonomous functionalities, module by module. We believe the currently present infrastructure will provide a good educational playground for students willing to contribute.

Bibliography

- [1] Nadeem Esmail Bacchus Barua and Taylor Jackson. *The Effect of Wait Times on Mortality in Canada*. 1st. Fraser Institute, 2014. ISBN: 978-0-88975-294-8.
- [2] Gary Nuss and George L. Kelling. *Police Response Time, Its Determinants and Effects*. 1st. The Police Foundation, Washington, DC, 1976. ISBN: UNK.
- [3] *Cairo Traffic Congestion study- Executive Note*. <https://www.worldbank.org/en/country/egypt/publication/cairo-traffic-congestion-study-executive-note>. Accessed: 2020-07-19.
- [4] Pablo Alvarez Lopez et al. “Microscopic Traffic Simulation using SUMO”. In: *The 21st IEEE International Conference on Intelligent Transportation Systems*. IEEE, 2018. URL: <https://elib.dlr.de/124092/>.
- [5] Nathan Koenig and Andrew Howard. “Design and Use Paradigms for Gazebo, An Open-Source Multi-Robot Simulator”. In: *In IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2004, pp. 2149–2154.
- [6] Morgan Quigley, Brian Gerkey, and William D. Smart. *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*. 1st. O’Reilly Media, Inc., 2015. ISBN: 1449323898.
- [7] X. Wu et al. “Vehicular Communications Using DSRC: Challenges, Enhancements, and Evolution”. In: *IEEE Journal on Selected Areas in Communications* 31.9 (2013), pp. 399–408.
- [8] “VANET Based Communication for Emergency Vehicles”. In: ().
- [9] Matthew O’Kelly et al. *F1/10: An Open-Source Autonomous Cyber-Physical Platform*. 2019. arXiv: 1901.08567 [cs.R0].
- [10] B Ravi Kiran et al. *Deep Reinforcement Learning for Autonomous Driving: A Survey*. 2020. arXiv: 2002.00444 [cs.LG].
- [11] Marco Wiering and Martijn van Otterlo. “Reinforcement Learning: State-of-the-Art”. In: Springer, 2012.
- [12] Richard S. Sutton and Andrew G. Barto. “Reinforcement Learning: An Introduction”. In: MIT press, 1998.
- [13] In: *IEEE 2020* ().
- [14] Subhadeep Patra Seilendria A. Hadiwardoyo. “An Android ITS Driving Safety Application Based on Vehicle-to-Vehicle (V2V) Communications”. In: *IEEE Conference Publication* (Nov. 2017). DOI: 10.1109/VNC.2009.5416384.
- [15] Frank Kargl A. Buchenscheit Florian Schaub and Michael Weber. “A VANET-based emergency vehicle warning system”. In: *Vehicular Networking Conference (VNC), 2009 IEEE* (Nov. 2009). DOI: 10.1109/VNC.2009.5416384.

- [16] Kapileswar Nellore and Gerhard P. Hancke. “Traffic Management for Emergency Vehicle Priority Based on Visual Sensing”. In: (Nov. 2016). DOI: 10.3390/s16111892.
- [17] B.M. Farheena S.; Chandak. “An approach towards traffic management system using density calculation and emergency vehicle alert”. In: *IOSR J. Comput. Sci* (Apr. 2014).
- [18] J.Baranitharam Baharadawaj R. Deepak. “Efficient dynamic traffic control system using wireless sensor networks”. In: *IEEE International Conference on Recent Trends in Information Technology (ICRTIT)* (July 2013).
- [19] andGerhard P. Hancke Kapileswar Nellore. “Traffic Management for Emergency Vehicle Priority Based on Visual Sensing”. In: (Nov. 2016).
- [20] David Lenz, Tobias Kessler, and Alois Knoll. “Tactical Cooperative Planning for Autonomous Highway Driving using Monte-Carlo Tree Search”. In: *2016 IEEE Intelligent Vehicles Symposium (IV)* (June 2016).
- [21] Karen Simonyan David Silver Julian Schrittwieser. “Mastering the Game of Go without Human Knowledge”. In: *nature* (Jan. 2016).
- [22] Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou Daan Wierstra Martin Riedmiller. “Playing Atari with Deep Reinforcement Learning”. In: (Dec. 2013).
- [23] Jakob N. Foerster. “Deep Multi-Agent Reinforcement Learning”. In: *Magdalen College at University of Oxford* (2018).
- [24] Audrunas Gruslys Peter Sunehag Guy Lever. “Value-Decomposition Networks For Cooperative Multi-Agent Learning”. In: (June 2017).
- [25] Adam Haber, Matthew McGill, and Claude Sammut. “jmeSim: An Open Source, Multi Platform Robotics Simulator”. In: 2012.
- [26] Varundev Suresh Babu and Madhur Behl. “ROS F1/10 Autonomous Racecar Simulator”. In: Macau, China, Oct. 2019.
- [27] *ROS Components*. URL: <https://www.roscomponents.com/en/visible-price/85-ust-101x.html>.
- [28] *Trampa Boards*. URL: <https://trampaboards.com/vesc-6-mkiii-in-cnc-t6-silicone-sealed-aluminium-box-with-genuine-xt90-connectors--vedder-electronic-speed-controller-trampa-special-p-27536.html>.
- [29] Benjamin Binder Eugen Kaltenegger and Markus Bader. “Controlling and Tracking an Unmanned Ground Vehicle with Ackermann drive”. In: *Proceedings of the Austrian Robotics Workshop (ARW-16)*. Wels, Austria, May 2016.
- [30] Roland Siegwart and Illah R. Nourbakhsh. *Introduction to Autonomous Mobile Robots*. USA: Bradford Company, 2004. ISBN: 026219502X.
- [31] Michael Behrisch Daniel Krajzewicz Jakob Erdmann. “Recent Development and Applications of SUMO - Simulation of Urban MObility.” In: 2012. URL: http://sumo.sourceforge.net/pdf/sysmea_v5_n34_2012_4.pdf.
- [32] Jie Song et al. “Research on car-following model based on SUMO”. In: *Proceedings of 2014 IEEE 7th International Conference on Advanced Infocomm Technology, IEEE/ICAIT 2014* (Jan. 2015), pp. 47–55. DOI: 10.1109/ICAIT.2014.7019528.

- [33] Jakob Erdmann. “SUMO’s Lane-Changing Model”. In: *Modeling Mobility with Open Data*. Ed. by Michael Behrisch and Melanie Weber. Cham: Springer International Publishing, 2015, pp. 105–123. ISBN: 978-3-319-15024-6.
- [34] G. Grisetti, C. Stachniss, and W. Burgard. “Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters”. In: *IEEE Transactions on Robotics* 23.1 (2007), pp. 34–46.
- [35] *OpenSlam - Gmapping*. URL: <https://openslam-org.github.io/gmapping.html>.
- [36] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic robotics*. Cambridge, Mass.: MIT Press, 2005. ISBN: 0262201623 9780262201629. URL: <http://www.amazon.de/gp/product/0262201623/102-8479661-9831324?v=glance&n=283155&n=507846&s=books&v=glance>.
- [37] Aaron Martinez and Enrique Fernandez. *Learning ROS for Robotics Programming*. Packt Publishing, 2013. ISBN: 1782161449.
- [38] Lentin Joseph. *ROS Robotics Projects*. Packt Publishing, Mar. 2017. ISBN: 978-1-78355-471-3.
- [39] Rachid Attia, Rodolfo Orjuela, and Michel Basset. “Longitudinal Control for Automated Vehicle Guidance”. In: Oct. 2012. DOI: 10.3182/20121023-3-FR-4025.00049.
- [40] Stefan Krauß. “Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics”. In: Apr. 1998.
- [41] Gabriel M. Hoffmann et al. “Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing”. In: *IN TO APPEAR IN THE PROCEEDINGS OF THE 26TH AMERICAN CONTROL CONFERENCE*. 2007.
- [42] Jarrod Snider. “Automatic Steering Methods for Autonomous Automobile Path Tracking”. In: (Apr. 2011).
- [43] M. L. Ho, P. T. Chan, and A. B. Rad. “Lane change algorithm for autonomous vehicles via virtual curvature method”. In: *Journal of Advanced Transportation* 43.1 (2009), pp. 47–70. DOI: 10.1002/atr.5670430104. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/atr.5670430104>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/atr.5670430104>.
- [44] Eitan Marder-Eppstein et al. “The Office Marathon: Robust Navigation in an Indoor Office Environment”. In: *International Conference on Robotics and Automation*. 2010.
- [45] C. Rösmann, F. Hoffmann, and T. Bertram. “Kinodynamic trajectory optimization and control for car-like robots”. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 5681–5686.
- [46] David Lu, Dave Hershberger, and William Smart. “Layered costmaps for context-sensitive navigation”. In: *IEEE International Conference on Intelligent Robots and Systems* (Oct. 2014), pp. 709–715. DOI: 10.1109/IRoS.2014.6942636.
- [47] Kaiyu Zheng. “ROS Navigation Tuning Guide”. In: *CoRR* abs/1706.09068 (2017). arXiv: 1706.09068. URL: <http://arxiv.org/abs/1706.09068>.

- [48] Higor Barbosa Santos et al. “Control of Mobile Robots Using ActionLib”. In: May 2017, pp. 161–189. DOI: 10.1007/978-3-319-54927-9_5.
- [49] J. B. Kenney. “Dedicated Short-Range Communications (DSRC) Standards in the United States”. In: *Proceedings of the IEEE* 99.7 (2011), pp. 1162–1182.
- [50] F. Wang. “Big challenges of vehicle communication and application”. In: *2015 IEEE 4th Asia-Pacific Conference on Antennas and Propagation (APCAP)*. 2015, pp. 380–383.
- [51] Tabish Rashid et al. “Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning”. In: *arXiv e-prints*, arXiv:2003.08839 (Mar. 2020), arXiv:2003.08839. arXiv: 2003.08839 [cs.LG].
- [52] Alexey Dosovitskiy et al. “CARLA: An Open Urban Driving Simulator”. In: *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, pp. 1–16.
- [53] Néstor Subirón. *CARLA 0.9.0: Multi-client multi-agent support*. <http://carla.org/2018/07/30/release-0.9.0/>. July 2018.
- [54] Guodong Rong et al. *LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving*. May 2020.
- [55] J. Gonzales et al. “Autonomous drifting with onboard sensors”. In: 2016.
- [56] Brian Goldfain et al. “AutoRally: An Open Platform for Aggressive Autonomous Driving”. In: *IEEE Control Systems Magazine* 39.1 (2019), pp. 26–55.
- [57] P.W.D. Charles. *HyphaROS Workshop*. https://github.com/Hypha-ROS/hypharos_racecar. 2017.
- [58] *Self Driving Cars, Medium*. URL: <https://medium.com/@subodh.malgonde/building-an-actual-self-driving-car-53f67ca41566>.
- [59] Siddhartha S. Srinivasa et al. *MuSHR: A Low-Cost, Open-Source Robotic Racecar for Education and Research*. 2019. arXiv: 1908.08031 [cs.R0].
- [60] Roxana Rădulescu et al. “Deep Multi-agent Reinforcement Learning in a Homogeneous Open Population”. In: *Artificial Intelligence*. Ed. by Martin Atzmueller and Wouter Duivesteijn. Cham: Springer International Publishing, 2019, pp. 90–105. ISBN: 978-3-030-31978-6.
- [61] *Machine Learning - Unity*. URL: <https://unity3d.com/machine-learning>.
- [62] Greg Brockman et al. “OpenAI Gym”. In: *CoRR* abs/1606.01540 (2016). arXiv: 1606.01540. URL: <http://arxiv.org/abs/1606.01540>.
- [63] J. M. Santos, D. Portugal, and R. P. Rocha. “An evaluation of 2D SLAM techniques available in Robot Operating System”. In: *2013 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. 2013, pp. 1–6.
- [64] G. Grisetti et al. “A Tutorial on Graph-Based SLAM”. In: *IEEE Intelligent Transportation Systems Magazine* 2.4 (2010), pp. 31–43.
- [65] ROS Perception Michael Ferguson. *slam_karto*. https://github.com/ros-perception/slam_karto. 2018.
- [66] CCNY Robotics Lab Isaac I.Y. Saito. *scan_tools*. https://github.com/ccny-ros-pkg/scan_tools/. 2016.

- [67] A. Censi. “An ICP variant using a point-to-line metric”. In: *2008 IEEE International Conference on Robotics and Automation*. 2008, pp. 19–25.
- [68] Martin Vajnar. “Model car for the F1/10 autonomous car racing competition”. Supervisor: Ing. Michal Sojka, Ph.D. MA thesis. Czech Technical University in Prague: Faculty of Electrical Engineering Department of Control Engineering, May 2017.
- [69] *ROS Answers*. URL: https://answers.ros.org/question/237580/hector_slam-vs-gmapping/.
- [70] *The Geduino Foundation*. URL: <http://www.geduino.org/site/archives/36#:~:text=Those%20two%20algorithms%20differ%20from,uses%20the%20laser%20scan%20only..>
- [71] Martin Treiber, A Hennecke, and Dirk Helbing. “Congested traffic states in empirical observations and microscopic simulations”. In: *Physical review. E, Statistical physics, plasmas, fluids, and related interdisciplinary topics* 62 (Sept. 2000), pp. 1805–24.
- [72] Venkatesan Kanagaraj et al. “Evaluation of Different Vehicle Following Models Under Mixed Traffic Conditions”. In: vol. 104. Dec. 2013. DOI: 10.1016/j.sbspro.2013.11.132.
- [73] Mitra Pourabdollah et al. “Calibration and Evaluation of Car Following Models Using Real-World Driving Data”. In: Oct. 2017. DOI: 10.1109/ITSC.2017.8317836.
- [74] SUMO. *Definition of Vehicles, Vehicle Types, and Routes*. https://sumo.dlr.de/docs/Definition_of_Vehicles,_Vehicle_Types,_and_Routes.html. June 2020.
- [75] P. Falcone et al. “Predictive Active Steering Control for Autonomous Vehicle Systems”. In: *IEEE Transactions on Control Systems Technology* 15.3 (2007), pp. 566–580.
- [76] *Lane Detection with Deep Learning*. URL: <https://towardsdatascience.com/lane-detection-with-deep-learning-part-1-9e096f3320b7>.
- [77] David Ball et al. “OpenRatSLAM: an open source brain-based SLAM system”. In: *Autonomous Robots* 34 (Apr. 2013), pp. 1–28. DOI: 10.1007/s10514-012-9317-9.
- [78] Parth Mehta et al. “A Review on Algorithms for Pathfinding in Computer Games”. In: Mar. 2015.
- [79] Liang Hu et al. “Optimal Position and Velocity Estimation for Multi-USV Positioning Systems with Range Measurements”. In: (July 2018). DOI: 10.1155/2018/5452723. URL: <https://doi.org/10.1155/2018/5452723>.
- [80] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: a modern approach*. 3rd ed. Pearson, 2009.
- [81] The GIMP Development Team. *GIMP*. Version 2.10.12. June 12, 2019. URL: <https://www.gimp.org>.
- [82] Houssam Abbas Christopher Kao Kim Luong. *Skeleton codes for F110 Fall 2018 course at UPenn*. <https://github.com/mlab-upenn/f110-fall2018-skeletons>. 2018.

- [83] *2D navigation stack*. <http://wiki.ros.org/navigation>. Accessed: 2020-07-19.
- [84] *teb_local_planner*. http://wiki.ros.org/teb_local_planner. Accessed: 2020-07-19.
- [85] Chang Ye et al. “Rotation, Translation, and Cropping for Zero-Shot Generalization”. In: *ArXiv* abs/2001.09908 (2020).
- [86] HyeongRyeol Kam et al. “RViz: a toolkit for real domain data visualization”. In: *Telecommunication Systems* 60 (Oct. 2015), pp. 1–9. DOI: 10.1007/s11235-015-0034-5.